The maya Book

A complete tour of a modern C++ terminal-UI library

From ANSI bytes to compile-time UI trees

agentty docs

a companion volume to the agentty sources

This book is a deep guide to maya, the header-mostly C++26 terminal UI library that powers agentty. We start with the terminal as a machine, work up through the C++ language features maya relies on, and then walk every layer of the library — the DSL, elements, the layout engine, the SIMD-accelerated renderer, the signal graph, the Elm-style runtime, and the widget catalogue. Each chapter ends with a working program.

Everything in this book is grounded in the actual source under maya/. Where the code disagrees with the prose, the code wins — report the mismatch as a bug.

Contents

Part I

Foundations

Hello, maya

This book teaches you maya — a library for building programs that draw their interface inside a terminal window — assuming you are new to almost everything involved. We will not skip steps. By the end of this first chapter you will know what kind of program maya is, why anyone bothers building such a thing in 2025, what is happening on your screen *right now* as you read this in a terminal, and how to compile and run your first maya program. None of that requires prior knowledge of C++; we will treat the example as a thing to look at, and Chapter 3 will teach you the language behind it.

1.1 What is a terminal? (the honest answer)

You have almost certainly used a terminal: it is the black-or-dark window that says things like `$` or `>` and lets you type commands. It might be called Terminal, iTerm, kitty, Alacritty, GNOME Terminal, Konsole, Windows Terminal, or just *the console*.

Here is what the terminal actually is, in plain words:

1. A **window on your screen** that shows a rectangular grid of text. Every position in the grid holds *one character* — a letter, a digit, a symbol — possibly with a colour.
2. A **program** (the *terminal emulator*) that owns that window. The terminal emulator's job is twofold:
   - When the user types on the keyboard, send those keystrokes to whatever program is running inside.
   - When that program prints text, draw the text into the window.
3. A **program running inside it**. Usually this is a *shell* like `bash` or `zsh` — the thing that shows the `$` prompt — but it can be *any* program. When you run `ls`, the shell starts `ls`, which prints file names, then exits, and you are back at the shell.

So the terminal emulator is a kind of two-way pipe with a screen attached: text goes in (from the keyboard), text comes out (from the program), and the rectangle of cells on screen is updated to reflect what the program has printed so far.

Idea

A terminal is a window that displays a grid of characters, and a program that pumps text between your keyboard and whatever else is running.

1.1.1 Why does it look so old?

Because it is. The protocol the terminal speaks — the rules for "move the cursor to row 5, column 12" and "print this in red" — was finalised on a physical device called the DEC VT100

in 1978, and every modern terminal still understands it. Programs written today talk the same byte language as programs written for an actual VT100. This is unusual in computing and turns out to be very valuable: any program that follows the rules will work on every machine you can `ssh` into, today, forever.

1.2 What is a TUI?

A **TUI** is a *Text User Interface*: a program that does not just dump output line by line, but *takes over* the terminal window and draws a proper interface inside it. Menus, panels, scrollable lists, progress bars, even charts — all rendered out of characters.

You have used TUIs without thinking of them as such:

- `vim` or `nano` when you edit a file.
- `htop` when you watch processes.
- `git log` (the pager part), `less`, `man`.
- Installer screens on a Linux ISO.

A **GUI** (Graphical User Interface) program asks the operating system to paint individual pixels: *draw a blue rectangle from* $(100, 80)$ *to* $(300, 120)$. A TUI program does something more modest: it asks the terminal emulator to put specific characters into specific grid cells, possibly with colour. The terminal emulator itself decides how each character looks (the font, the cell size); the TUI program only chooses what character goes where.

Idea

GUI = pixels you place anywhere. TUI = characters you place inside a fixed grid of cells.

That restriction — one character per cell — is the entire reason TUIs are smaller, faster, and easier to write than GUIs. There is no font engine, no anti-aliasing, no input method editor; just a grid and some bytes.

1.2.1 Why bother with TUIs in 2025?

You might reasonably ask: why not just write a web page? Several reasons:

- **They run anywhere there is a terminal.** A developer on a server they reached by `ssh` has no browser, no desktop, sometimes not even a graphical session. They do have a terminal.
- **They are very fast.** A typical TUI redraw is a few kilobytes of bytes. A web page is megabytes of HTML, CSS, JS, images, and a browser.
- **They are scriptable.** A TUI program's output is still text, so it composes with pipes, `grep`, `ssh`, `tmux`, and the rest of the Unix toolbox.
- **They are pleasant to use for developers.** Keyboard first, no mouse round-trips, low latency, look good in any monospace font.

The trade-off is the visual restriction: no images (well, mostly), no freeform layouts, no animations beyond what a grid of glyphs can do. For a large class of programs, that is exactly the right trade.

1.3 One frame of a TUI: what literally happens

To make the rest of this book concrete, let us walk what happens in one "frame" of a typical TUI — the unit of work between two updates of the screen. We will use a counter app as the example: it shows a number, and the keys `+` and `-` change it.

Suppose the screen currently shows the number `7`. The user presses `+`.

1. The keyboard sends a byte to the kernel. When you press `+`, the OS kernel receives the byte $0x2B$ (ASCII for `+`) from your keyboard driver and places it into the input buffer of whichever terminal program currently has focus — your terminal emulator.

2. The terminal emulator forwards the byte. Your terminal emulator (kitty, iTerm, GNOME Terminal, ...) writes the byte into one end of the *pseudo-terminal* — the kernel byte-pipe between the emulator and the program running inside it. Chapter 2 covers the pseudo-terminal in detail.

3. The maya program reads the byte. The maya program is sitting in a loop, asking the OS "is there any input ready?". Now there is. It reads $0x2B$ from its standard input.

4. Maya turns the byte into an Event. A single byte is not yet a useful concept. maya's input parser decides "this is a printable character, namely the character `+`" and constructs a structured value: `Key{.kind = KeyKind::Char, .character = '+'}`. That value is then wrapped in the broader `Event` type and handed to your application code.

5. Your code computes a new state. Your application receives the `Event`, decides it means "increment", and produces the new state: the counter is now 8. In an Elm-style maya program this is one pure function: `update(state=7, event=Inc) = 8`.

6. Maya turns the new state into an Element tree. Your `view` function is called with the new state (8) and returns an `Element`: a description of the UI, "a box with a rounded border containing the text `Count:  8`". This is a value in memory, not yet bytes on the wire.

7. The layout engine assigns positions. Given the current terminal size — say 80 columns by 24 rows — maya walks the element tree and decides where every piece goes: the outer box at $(0, 0)$ of size 14×3, the inner text at $(1, 1)$, and so on. The output is a list of styled cells, one per position.

8. The renderer builds a virtual screen. Those cells are written into a 2-D grid in memory — the *canvas*. Every position on the screen now has a known character, foreground colour, background colour, and attribute set.

9. The differ finds what changed. A copy of last frame's canvas is still around. maya compares the two canvases cell by cell. For our counter, almost everything is identical; only one cell changed — the `7` became an `8`.

10. Maya emits ANSI bytes for only that cell. A short string of bytes is produced: "move cursor to (col, row) of the changed cell; set the right colour; print `8`". Even though there might be hundreds of cells on screen, only a dozen bytes are sent.

11. Those bytes flow back through the pty to the emulator. The emulator reads them, parses them, and updates its window. The user sees the digit change. From keypress to pixel update is typically a couple of milliseconds.

That is one frame. The rest of the book is, in a sense, just detailed coverage of each of those eleven steps. The DSL (Chapter 5) and Element tree (Chapter 6) cover steps 5–6; the layout engine (Chapter 7) is step 7; the canvas (Chapter 8) is step 8; the differ (Chapter 9) is step 9; ANSI (Chapter 10) is step 10; the runtime that orchestrates the whole loop (Chapter 4) is what sits around steps 3–6.

Idea

One frame: a byte arrives, it becomes an event, you compute new state, maya builds the new screen, compares it to the old one, and sends bytes for just the differences. That tight loop is what every TUI library is, including maya.

1.4 What is maya?

maya is a C++ **library** — a bundle of code you include in your own program — whose job is to make writing a TUI pleasant. Without a library, writing a TUI means manually printing escape sequences (we will look at the bytes in Chapter 2), manually tracking what is on screen, manually parsing keyboard input, and manually re-laying-out everything when the window resizes.

With maya, you describe what you *want* on screen as a tree of boxes and text, and maya works out the bytes to send. Concretely, the work the library does for you is:

1. Take your description (*"a box containing the text 'Hello' with a rounded border"*) and figure out the position and size of every piece given the current window size.
2. Build a virtual grid of cells representing what the screen should look like.
3. Compare that grid to what is *already* on screen, and emit just enough escape sequences to update the cells that changed.
4. Read keyboard, mouse, paste, and resize events and feed them back to your program.
5. Restore the terminal to a clean state when your program exits, even on crashes.

This book covers each of those jobs in detail. For now, treat maya as a black box that turns "description of a UI" into "the correct bytes on the wire".

1.5 What is a tree, and why a tree?

In computing, a **tree** is a data structure that looks like a family tree turned upside down: one thing at the top, called the **root**, which has zero or more **children**, each of which may have its own children, and so on. The end of every branch — a node with no children — is called a **leaf**.

A terminal UI is naturally tree-shaped, and once you see it you cannot unsee it. Consider a chat application:

```
+-----------------------------------------+
| Title: my chat                          |
+-----------------------------------------+
| sidebar  |  message list                 |
|  - alice |   alice: hi                   |
|  - bob   |   me   : hello                |
|          |   bob  : evening              |
+----------+-------------------------------+
| type here...                            |
+-----------------------------------------+
```

Described as a tree, this UI is:

```
Root: vertical stack
|-- Title bar (text)
|-- Body: horizontal split
|   |-- Sidebar: vertical list
|   |   |-- "alice"  (text)
|   |   `-- "bob"    (text)
|   `-- Messages: vertical list
|       |-- "alice: hi"      (text)
|       |-- "me   : hello"   (text)
|       `-- "bob  : evening" (text)
`-- Input: bordered text box
```

Four kinds of node appear: “vertical stack”, “horizontal split”, “list”, “text”. The first three are *container* nodes — they have children that they arrange. Text nodes are leaves: they have no children, just a string.

A tree is the right structure because UIs are recursive: a sidebar *is* a UI; a message list *is* a UI; both are pieces of a bigger UI. Trees compose. You can reuse the message-list subtree in a completely different program by grafting it onto a different root.

maya’s `Element` type is exactly the tree representation of a UI. Container nodes (`BoxElement`, `ElementList`) hold `std::vector<Element>` of children; leaf nodes (`TextElement`) hold the bytes of a string. The DSL is the surface syntax that lets you spell that tree out as readable C++ code. Chapter 6 dissects the `Element` type in full.

Idea

A UI is a tree of containers and text. Containers arrange their children; text leaves carry the actual characters. maya’s `Element` type *is* that tree.

1.6 Why does maya exist when there are already others?

There are many TUI libraries. `ncurses` (C, ancient and respected), `ratatui` (Rust), `bubbletea` (Go), `textual` (Python), `ink` (Node.js). Each is good in its own way. maya picks three positions that distinguish it.

1. Wrong UI code does not compile. Most libraries accept your code happily and let you discover the bug when you run the program: a border colour set without a border becomes a no-op, a negative padding silently becomes zero, a layout meant to be a row accidentally becomes a column. maya is built on C++'s *type system* so that a lot of these mistakes refuse to compile. The compiler tells you, at build time, exactly which line is wrong.

As a tiny taste: in maya you might write

```
auto ui = t<"Hello"> | bcol<60, 65, 80>;   // border colour, no border yet
```

A library written in Python or Go would accept that and silently do nothing with the border colour at run-time. maya refuses to compile it, because the `bcol` ("border colour") modifier is defined to only apply to nodes that have already been given a border. The compiler error names the constraint that failed; you fix the code by adding `| border_<Round>` first.

You will not see why this is possible until Chapter 5; the short version is that the description you write is stored in the *types* of the C++ values, and maya's API states the rules between those types.

2. Re-drawing is wired to be fast. Once maya has decided what the next frame should look like, it compares that virtual grid of cells with the grid that is currently on screen. Only the cells that changed are sent to the terminal. That comparison is done with the CPU's **SIMD** instructions — special instructions that operate on many bytes at once. The effect is that even on a slow `ssh` link, a maya program redraws smoothly. We will dissect the SIMD code in Chapter 9; you do not need to know what SIMD is yet.

3. The runtime is borrowed from a language called Elm. maya organises a whole program around one idea: your program's state is a single value, your program's events are a single closed list of cases, and the function that updates the state given an event is *pure* — meaning it computes a new value and does nothing else (no printing, no opening files, no network calls). Anything that needs to "happen in the world" is described as data and handed to the library to perform. This is unusual for C++, and the payoff is programs that are easy to test, easy to replay, and easy to reason about. Chapter 4 is dedicated to this idea.

If those three sentences mean nothing to you yet, that is fine. The book is structured so each one is earned, slowly, before you have to use it.

1.7 The layers of a maya program

It will help to have a picture of how the work flows, even if many of the words are unfamiliar now. Read it once; you do not need to memorise it.

Your code	describe a UI; say what should happen on each event
DSL	C++ syntax (`|`, `t<"...">`) that builds your description
Element	in-memory tree of boxes and text
Layout	gives every box a position and size
Canvas	a grid of cells with characters, colours, attributes
Diff	finds which cells changed since last frame
ANSI	escape-sequence bytes for the changed cells
Writer	sends those bytes to the terminal
Terminal	draws characters into the window

A few words on the unfamiliar ones:

- **DSL** stands for *Domain-Specific Language*. It is not a separate language — it is just normal C++ — but the surface looks specialised. You write `t<"Hi"> | Bold | border_<Round>` and that is valid C++ that produces a UI description. The DSL is what makes maya code short.
- **Element** is the name we give to the in-memory shape of your UI. It is a tree because boxes contain other boxes (and text). The element tree is the central data structure of maya.
- **ANSI** is the byte language the terminal understands. `ANSI` is short for *American National Standards Institute*, who blessed the standard in 1976. We will look at individual ANSI bytes in Chapter 2.

Each layer knows only about the one immediately below it. Your code talks to the DSL, the DSL produces an Element tree, the layout engine walks the tree, and so on. This separation is what makes the library both testable and replaceable: someone could swap out the SIMD diff for a slower scalar one without anything above noticing.

1.8 Your first maya program

Here is a complete, real maya program. It compiles, runs, prints a small banner, and exits.

```cpp
#include <maya/maya.hpp>
using namespace maya;
using namespace maya::dsl;

int main() {
    constexpr auto ui = v(
        t<"Hello, maya"> | Bold | Fg<100, 180, 255>,
        t<"a tiny banner"> | Dim
    ) | border_<Round> | pad<1>;

    print(ui.build());
}
```

Before we decode it line by line, here is roughly what you should see in your terminal after running this:

Terminal

```
.----------.
| Hello, maya    |
| a tiny banner  |
'----------'
```

A two-line text stack, the first line bold and pale blue, the second dimmed, surrounded by a rounded border with one cell of padding inside it. The corners are drawn with the Unicode box-drawing glyphs the terminal renders as smooth round corners.

You are not expected to understand every token of the program. You *are* expected to be able to point at each piece and at least name what it is. Let us go line by line.

1.8.1 Line by line

`#include <maya/maya.hpp>`. `#include` is the C++ way of saying "copy and paste the contents of this file here before compiling". *maya/maya.hpp* is a single header that pulls in the public parts of maya. After this line, the names `v`, `t`, `Bold`, `print`, and so on are available.

`using namespace maya;` and `using namespace maya::dsl;`. A **namespace** is a labelled container of names. maya's public API lives inside the namespace `maya`, and the DSL pieces live in `maya::dsl`. The two `using namespace` lines say "for the rest of this file, names from those namespaces are visible without their prefix". Without them you would have to write `maya::print(maya::dsl::v(...))`. Verbose; we elide it for examples.

`int main()`. Every C++ program starts at a function called `main`. The `int` says `main` returns an integer (the exit status); we do not write a `return` statement, and C++ treats that as `return 0` which means "success".

`constexpr auto ui = ...`. Three things to unpack:

- `auto` means "figure out the type of this for me, based on the right-hand side". C++ normally requires you to spell the type; `auto` lets the compiler infer it. The actual type of `ui` here is something elaborate that we do not care to write out.
- `constexpr` means "this value is computed at *compile time*, not at runtime". The whole tree — the border, the colour, the text — is baked into the binary. When the program starts, the tree is already there.
- `ui` is just the name of the variable. We could have called it anything.

`v(...)`. `v` is maya's function for "stack these things vertically". The arguments inside the parentheses are the things to stack — in this case, two pieces of text. There is also `h(...)` for horizontal.

`t<"Hello, maya">`. `t` is a **template** parameterised by a compile-time string. The angle brackets `<...>` are the template-argument syntax. `t<"Hello, maya">` is a value representing the literal text "Hello, maya". The peculiar thing here is that the text itself lives inside the type — swapping `"Hello"` for `"World"` gives a different type. This is one of the tricks that lets the compiler check things. Chapter 3 explains how it works in detail.

`| Bold | Fg<100, 180, 255>`. The vertical bar `|` is the **pipe operator** in C++. By default it means bitwise-or on integers, but you can teach it to mean anything for your own types. maya has taught it to mean *compose this style onto the thing on the left.* `Bold` is a tag value meaning "apply bold"; `Fg<R, G, B>` is a template that takes three numbers in the range 0–255 and represents a foreground colour.

So `t<"Hello, maya"> | Bold | Fg<100, 180, 255>` reads left-to-right: "the text 'Hello, maya', made bold, given a foreground colour of $(100, 180, 255)$". A pleasant pale blue.

`| Dim`. Same idea: applies the *dim* attribute (terminals usually render this as a faded foreground). One of the few attributes whose appearance varies most across terminals.

`| border_<Round> | pad<1>.` After the inner `v(...)`, two more pipes wrap the result. The first puts a rounded border around the whole stack; the second adds one cell of padding (space) inside the border. `Round` is a choice from a small set (`Round`, `Square`, `Double`, `Thick`, etc.); `pad<1>` carries the number 1 in its type.

`print(ui.build()).` Finally, the call that does something. `ui.build()` converts the compile-time description `ui` into a runtime `Element` — the in-memory tree we mentioned earlier. `print(...)` is the simplest of four output modes maya provides; it paints the element once, to standard output, and returns. No event loop, no alternate screen, no clearing the terminal. For a static banner, that is what we want. Bigger programs use `run<P>(...)`, which we will meet in Chapter 4.

1.8.2 What you should take away

Three observations are worth carrying forward:

1. The structure of the UI lives in the C++ source as a *value*: `ui` is a thing you can pass to a function, return from a function, store in an array. It is not strings of markup, not a separate file format, not XML.
2. Because the value is `constexpr`, the compiler builds it. If you wrote `Fg<300, 0, 0>` (out of range), the compiler would refuse. If you wrote `border_<Squiggle>` (not a real style), the compiler would refuse. These checks are free at runtime — they have already happened.
3. Reading code with `|` is like reading English left to right: subject first, modifiers afterwards. Once you get used to it the syntax is quite pleasant to scan.

1.9 Building and running

The maya repository builds with **CMake**, a tool that generates build files for your platform. You need a recent C++ compiler — GCC 15 or newer, or a recent Clang — because maya uses C++26 features. On Linux:

```
git clone --recursive https://github.com/1ay1/agentty
cd agentty/maya
cmake -B build
cmake --build build -j$(nproc)
```

What each command does, briefly:

- `git clone -recursive ...` downloads the project and its sub-projects (called *submodules*).
- `cd agentty/maya` moves into the maya directory.
- `cmake -B build` reads maya's *CMakeLists.txt* file and produces build files in a new directory called *build*.
- `cmake -build build -j$(nproc)` actually compiles things, using as many parallel jobs as you have CPU cores.

If it succeeds, *build/* contains around two dozen example programs — *build/counter*, *build/dashboard*, *build/doom_fire* — one for each file in *examples/*. Run any of them by typing its path:

```
./build/counter
```

A small UI appears. Press the keys it documents (often `q` to quit). Resize the terminal window while it runs; watch the UI re-flow.

1.9.1 Adding your own example

The easiest way to try the program above is to drop it into *maya/examples/hello.cpp*, re-run CMake, and rebuild:

```
# from inside maya/
$EDITOR examples/hello.cpp        # paste the program
cmake -B build                    # picks up the new file
cmake --build build -j$(nproc)
./build/hello
```

This loop — edit, build, run — is the loop you will spend the rest of the book in.

Pitfall

If the build complains about `constexpr`, `concept`, or unknown standard-library headers, your compiler is too old. `g++ -version` should be 15 or newer. On Ubuntu/Debian: `sudo apt install g++-15`. On macOS, install a recent Homebrew `llvm`. On Windows, use WSL with a current Ubuntu.

1.10 What the rest of the book looks like

The book has four parts.

Part I, Foundations, is the part you are reading. After this chapter, Chapter 2 explains what a terminal really is at the byte level — you will be able to write "Hello" in red without any library at all by the end of it. Chapter 3 is a careful, from-the-ground-up tour of the C++ features maya uses; if you are new to C++, this is the chapter that pays for everything that follows.

Part II, Architecture, is about the way a maya program is structured. The Elm runtime (4), the DSL that produces UI descriptions (5), the in-memory tree that is the result (6), and the layout engine that decides where things go (7).

Part III, Rendering, is the bytes-on-the-wire half. Cells and canvases (8), SIMD comparison (9), the ANSI language (10), and the special mode where maya renders *inline* without taking over the terminal (11).

Part IV, Runtime and widgets, is everything that happens while a program runs. Fine-grained reactivity with signals (12), input parsing (13), the full effect algebra (14), the widget catalogue (15), and how to ship a program built on maya (16).

You can read straight through. You can also skip Chapter 3 on first pass if you already know modern C++ well, and come back to it as a reference.

1.11 How maya compares to other TUI libraries

It helps to know what other people are doing. Here is the same static banner — text, bold, colour, border — in four other popular TUI libraries, alongside the maya version, so you have a felt sense of where this book is taking you.

```c
#include <ncurses.h>
int main(void) {
    initscr();
    start_color();
    init_pair(1, COLOR_CYAN, COLOR_BLACK);
    box(stdscr, 0, 0);
    attron(A_BOLD | COLOR_PAIR(1));
    mvprintw(1, 2, "Hello, maya");
    attroff(A_BOLD | COLOR_PAIR(1));
    mvprintw(2, 2, "a tiny banner");
    refresh();
    getch();
    endwin();
}
```

Imperative. You position every character by row and column. There is no layout; if the terminal is smaller than your hard-coded numbers, the UI breaks.

```python
from textual.app import App
from textual.widgets import Static

class Hello(App):
    def compose(self):
        yield Static("[bold cyan]Hello, maya[/]\n[dim]a tiny banner[/]",
                     classes="box")
    CSS = ".box { border: round cyan; padding: 1; }"

Hello().run()
```

Declarative, pleasant, but markup is strings (the bold/colour spans), and the CSS is a separate sub-language the Python compiler cannot check.

```rust
let block = Block::default()
    .borders(Borders::ALL)
    .border_type(BorderType::Rounded)
    .border_style(Style::default().fg(Color::Cyan));
let text = Text::from(vec![
    Line::from(Span::styled("Hello, maya",
        Style::default().fg(Color::Cyan).add_modifier(Modifier::BOLD))),
    Line::from(Span::styled("a tiny banner",
        Style::default().add_modifier(Modifier::DIM))),
]);
frame.render_widget(Paragraph::new(text).block(block), frame.size());
```

Close to maya in spirit — a tree of widgets, styles as values — but more verbose, and styles are runtime values, so e.g. an out-of-range colour is caught only at runtime (or in your tests).

```
constexpr auto ui = v(
    t<"Hello, maya"> | Bold | Fg<100, 180, 255>,
    t<"a tiny banner"> | Dim
) | border_<Round> | pad<1>;
print(ui.build());
```

The whole description is a `constexpr` value. The compiler builds the tree at translation time; bad styles, bad borders, and out-of-range colours are caught before the program runs.

The trade is roughly: more verbose than maya in dynamic languages; more strictly checked in maya than anywhere else; broadly similar mental model across the declarative tier (textual, ratatui, maya). Once you can read one, you can read all of them.

1.12 What is a "binary"?

maya (C++26). We keep mentioning "the binary" — the thing your compiler produces. One sentence on what it is: a **binary** (or **executable**) is a single file on disk full of CPU instructions plus the data your program needs. When you run `./hello`, the operating system loads that file into memory and begins executing the instructions at a fixed entry point (typically a function the compiler links in, which then calls your `main`).

The binary is built once and run many times. Anything that can be computed at compile time can live inside the binary directly — text strings, constant tables, style values — without any cost at startup. That is what maya exploits with `constexpr`: the UI tree of a static banner is, literally, encoded into the bytes of the executable file.

1.13 How this book is laid out

A few visual conventions you will see in the chapters:

Idea

Blue boxes are **ideas** — a single sentence summarising the key intuition. If you only remember one thing from a section, this is what it should be.

Theory

Purple boxes are **theory** — the background concept from computer science or programming-language theory behind what the section just explained. Skippable on first read; useful on second.

Pitfall

Amber boxes are **pitfalls** — a mistake people new to the material commonly make, and how to avoid it.

Recap

Green boxes at the end of a section summarise it in two or three sentences.

Terminal

```
Black boxes contain literal terminal bytes or output, the way the terminal sees
or shows them.
```

Code in monospace with a coloured rule on the left is C++ (or occasionally shell). Lines are numbered. Long examples are broken into pieces and explained.

Recap

A TUI is a program that draws into the grid of cells inside a terminal window. maya is a C++26 library that lets you describe a TUI as a tree of styled boxes and text and renders it for you. Its three distinctive ideas — compile-time correctness, SIMD-fast diffing, and a pure-functional runtime — are each given a chapter of their own later. The first maya program is a five-line static banner; the loop you spent the rest of the book in is edit, build, run.

Workshop

You should not yet expect to do these without looking things up. They are calibrated so that doing them takes you exactly through the machinery the chapter introduced.

1. **Build the repository.** Get a clean `cmake -build build -j$(nproc)` succeeding on your machine. If the build fails, the error message is the assignment — read it, look up the missing tool, install it.
2. **Run three examples** from *build/*. Suggested: *counter* (smallest), *dashboard* (mid-sized), *doom_fire* (visually striking). Resize the window while each is running.
3. **Compile the Hello-banner.** Drop the program from earlier into *maya/examples/hello.cpp*, re-run CMake, rebuild, run.
4. **Mutate the banner.** Change the foreground colour to something else. Swap `Round` for `Double` and then `Thick`. Change `pad<1>` to `pad<3>`. Each change is one number or one identifier; each requires a rebuild to take effect.
5. **Provoke a compile error.** Change `Fg<100, 180, 255>` to `Fg<100, 180, 999>`. Read the compiler's error message. You do not need to fix it elegantly — just observe that it caught the mistake.

Glossary of terms introduced in this chapter

Keep this list handy; every term here is referenced freely in the rest of the book.

ANSI
: The byte language a program writes to a terminal to control colour, cursor position, and screen modes. Standardised in 1976. Chapter 2 covers it.

Binary, executable
: A file on disk holding compiled CPU instructions for your program. `./hello` runs the binary called *hello*.

Canvas
: maya's in-memory 2-D grid of styled cells; the working representation of the next screen. Chapter 8.

Cell One position in the terminal grid. Holds one character (sometimes two-cell-wide), foreground colour, background colour, and attribute bits.

`constexpr`
: C++ keyword meaning "compute this at compile time". maya's DSL leans on it heavily.

Diff The pass that compares the new canvas to the previous one and decides which cells changed. Chapter 9.

DSL
: Domain-Specific Language. The pleasant `t<"..."> | Bold | ...` syntax in maya is a DSL embedded inside C++. Chapter 5.

Element
: maya's in-memory UI tree. The output of your `view` function; the input to the layout engine. Chapter 6.

Elm runtime
: The Elm-inspired loop (`init`–`update`–`view`–`subscribe`) that maya programs are organised around. Chapter 4.

Escape sequence
: A short string of bytes beginning with `ESC` (`0x1B`) that tells the terminal to do something. The grammar of ANSI.

Frame
: One complete update cycle: read events, compute new state, build new canvas, diff, emit bytes.

GUI
: Graphical User Interface — pixel-based. Contrast with TUI.

Layout
: The step that assigns each Element a position and size given the terminal dimensions. Chapter 7.

Leaf, node, root
: Tree vocabulary. The root is the top of a tree; leaves are nodes with no children; everything else is an internal node.

NTTP
: Non-Type Template Parameter — a template argument that is a value, not a type. The trick that lets `t<"Hello">` encode a string into a type. Chapter 3.

Pseudo-terminal, pty
: The kernel-managed byte pipe between a terminal emulator and the program running inside it. Chapter 2.

Raw mode
: A configuration of the terminal where the kernel passes every keystroke through immediately, with no line editing or echo. The mode every TUI runs in.

SIMD
: Single Instruction Multiple Data: CPU instructions that operate on many bytes at once. maya's diff pass uses SIMD. Chapter 9.

stdin / stdout / stderr
: The three default file descriptors — 0, 1, 2 — a process inherits. In a TUI all three are wired to the pty.

Tree
: A data structure: a root with children, each of which may have children. A UI is naturally a tree.

TUI
: Text User Interface. A program that draws into the grid of cells inside a terminal window.

UTF-8
: The standard variable-length encoding of Unicode characters into bytes. Modern terminals expect UTF-8.

The terminal, byte by byte

Chapter 1 described a terminal in plain words. This chapter looks underneath. We will see exactly what bytes travel between your program and the terminal emulator, what those bytes mean, and why the protocol is the way it is. By the end you will be able to read raw terminal output, write "Hello" in red without any library, and trust your mental model of what maya is doing one floor below the API.

There is nothing magical here. The terminal is one of the simplest abstract machines in computing. It has a small alphabet and a small list of rules; we will learn both.

2.1 What is a "state machine"?

The chapter is titled "the terminal as a state machine", so it is worth saying what that phrase means before we use it.

A **state machine** (formally, a *finite-state automaton*) is a model of computation with three parts:

1. A finite set of **states** — things the system can *be in* right now.
2. An **alphabet** — the kinds of *input* the system reacts to.
3. A **transition table** — for each (state, input) pair, the state the machine moves to next, and any output it produces along the way.

The machine sits in a state, reads an input, looks up the transition, and moves. That is the entire model. A vending machine is one: state is "how much money has been inserted"; alphabet is "coin", "button press"; transitions either accept the coin (raising the balance) or, if the button is pressed and balance is sufficient, dispense a snack and return to zero.

A terminal emulator is a state machine. Its states include:

- *Ground* — the default, awaiting the next byte.
- *Escape* — the previous byte was `ESC`; expecting a follow-up.
- *CSI* — inside a Control Sequence; reading parameter digits.
- *OSC* — inside an Operating System Command; reading a string up to a terminator.
- *DCS* — inside a Device Control String (rare).

Its alphabet is "one byte". Its transitions read like:

State	Input byte	Action / next state
Ground	printable	emit the glyph; stay in Ground
Ground	`0x1B` (ESC)	move to Escape
Ground	`0x0A` (LF)	move cursor down; stay in Ground
Escape	`[`	move to CSI
Escape	other	dispatch a one-byte escape; back to Ground
CSI	digit	accumulate into current parameter
CSI	`;`	start the next parameter
CSI	letter	dispatch the sequence; back to Ground

This is a tiny excerpt of what a real terminal emulator implements (Paul Williams's well-known VT500 state diagram has a few more rows), but it is the whole essence. Every byte the emulator receives moves it through these states; the side effects — drawing glyphs, changing colours, moving the cursor — happen on the transitions.

maya's input parser is also a state machine, but in reverse: it reads bytes from the keyboard and assembles them into structured `Key` events. Its states are "Ground", "after ESC", "inside CSI", "maybe Alt+key". The grammar is the same; only the direction of the byte flow is reversed.

Idea

The terminal is a small state machine that reads bytes and updates a grid. Your program is a state machine in the other direction: structured intent in, bytes out.

2.2 Three things called "terminal"

The word *terminal* is overloaded. Pinning down which one we mean is half the battle. There are three distinct things, often confused.

The hardware terminal. A physical device with a keyboard and a screen that connected to a mainframe over a serial cable. Digital Equipment Corporation's **VT100** (released 1978) is the famous one. Nobody owns one today, but the byte protocol it spoke survives essentially unchanged on every modern system. When we say a modern emulator "speaks ANSI", we mean it speaks VT100's grandchildren.

The terminal emulator. A program on your machine — iTerm2, kitty, Alacritty, GNOME Terminal, Konsole, Windows Terminal, xterm — that pretends to be a VT100 (or a richer descendant). It owns a window on your desktop, draws characters into it, and forwards your keystrokes to whatever program is running "inside" it.

The pseudo-terminal, or *pty*. A pair of file descriptors in the operating-system kernel that hooks the emulator up to the program. We will look at it in a moment. The pty is the part most people never hear named, and yet it is the part doing the actual work of being a terminal.

When this book says "maya writes ANSI to the terminal", it means: maya calls the operating-system primitive `write` on the file descriptor numbered 1 (called `stdout`); the bytes flow through the pty; the emulator on the other end reads them and updates the window.

2.2.1 What is a file descriptor?

If you have not run into the word before: in Unix, every open "thing you can read from or write to" is named by a small integer called a **file descriptor**. Three are conventional:

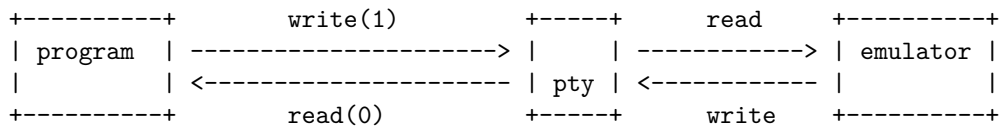
0 `stdin` – bytes the program reads as input.
1 `stdout` – bytes the program writes as ordinary output.
2 `stderr` – bytes the program writes as error output.

When you run a program inside a terminal, all three are wired to the pty. When you write `ls > out.txt`, the shell rewires the program's file descriptor 1 to a freshly opened file before starting it. The program is none the wiser: it calls `write(1, ...)` as always, and bytes happen to land in a file. This is the famous Unix "everything is a file" design, and it is what makes terminal programs so flexible.

2.3 The pseudo-terminal: a kitchen-pass-through

A useful mental model for the pty: imagine a kitchen with a pass-through window. The cook (your program) puts plates of bytes on the counter; the server (the terminal emulator) picks them up and delivers them to the diner (the window on your screen). When the diner says something to the server, the server sets a slip of paper (keystrokes) on the counter for the cook to pick up. The cook and the server never see each other; they only exchange bytes via the counter.

In reality the "counter" is a chunk of kernel memory — a buffer with two ends. The cook (your program) sees one end as a normal file: `write` to send bytes into it, `read` to take bytes out of it. The server (the emulator) sees the other end the same way. The kernel shuffles bytes between the two ends, with a few useful behaviours layered on (input modes, signal handling — we will get to those).



This picture is enough for the rest of the chapter. We do not need to look at the implementation of the pty itself.

Idea

The pty is a two-way byte pipe between your program and the terminal emulator. `stdout` goes *to* the emulator; `stdin` comes *from* it.

2.4 The byte language: ASCII plus ANSI

Now the central question: what bytes does the program send, and what do they do?

2.4.1 Ordinary bytes: ASCII

Most bytes you write are interpreted literally. The byte `0x48` is the letter `H`; `0x69` is `i`; writing the two-byte sequence `0x48 0x69` prints `Hi`. This mapping is **ASCII**, a standard from 1963 covering the bytes from 0 to 127. Every English letter, every digit, every common punctuation mark has a fixed ASCII byte.

Characters outside ASCII (accented letters, CJK, emoji) are sent as **UTF-8** sequences of two-to-four bytes per character. Modern terminals all understand UTF-8; `maya` produces UTF-8 throughout. We will return to the question of *cell width* for non-ASCII characters in the canvas chapter.

A quick word on UTF-8

UTF-8 is the variable-length encoding of Unicode that the world standardised on. The rule, in one paragraph: code points 0–127 (the old ASCII range) are encoded as a single byte, exactly as in ASCII; code points 128–2047 take two bytes; 2048–65535 take three bytes; the remainder (including emoji and obscure scripts) take four. The first byte of a multi-byte sequence has a specific high-bit pattern identifying the length (`110xxxxx` for two-byte, `1110xxxx` for three-byte, `11110xxx` for four-byte), and subsequent bytes all begin with `10xxxxxx`.

Example. The letter `H` is `0x48` (one byte). The Greek letter mu, μ (U+03BC), takes two: `0xCE 0xBC`. The Chinese character zhōng (U+4E2D) takes three: `0xE4 0xB8 0xAD`. A grinning emoji (U+1F600) takes four: `0xF0 0x9F 0x98 0x80`.

Why this matters for a TUI: the number of *bytes* in a string is not the number of *characters*, and the number of characters is not the number of *cells* the terminal will use to display it. Three different quantities. `maya`'s text layer distinguishes them carefully; you usually do not have to.

2.4.2 Control bytes

A handful of bytes in the range 0–31 have *control* meanings: they do not print a glyph, they tell the terminal to do something. The interesting ones:

Byte (hex)	Name	Effect
`0x07`	BEL	beep / bell
`0x08`	BS	backspace — cursor left one cell
`0x09`	HT	horizontal tab — advance to next tab stop
`0x0A`	LF	line feed — cursor down one row
`0x0D`	CR	carriage return — cursor to column 1
`0x1B`	ESC	escape — start of an escape sequence
`0x7F`	DEL	delete — treated as backspace on input

You have met most of these. `LF` is the newline. `CR` is why old-school progress bars work — print `CR`, then a fresh line of text, and you overwrite the previous one in place.

The interesting byte is `ESC`, hex `0x1B`, decimal 27. It is the door into the ANSI escape sequence language.

2.4.3 Anatomy of an escape sequence

An ANSI escape sequence has a strict shape:

`ESC [` *parameters* *final-byte*

- `ESC` is the single byte `0x1B`.
- `[` is the literal ASCII byte `0x5B`. Together, `ESC [` (two bytes) form the **Control Sequence Introducer**, abbreviated **CSI**.
- *parameters* is a list of decimal numbers separated by semicolons. Each number is written as its ASCII digits — so the number 31 is two bytes, `0x33 0x31`, not the binary value 31.
- *final-byte* is a single ASCII letter that says which command this sequence is. The letter chooses which family the command belongs to.

Here is a complete example, byte by byte. Suppose we want to apply the colour "bold red":

Terminal

```
ESC [ 1 ; 3 1 m 0x1b 0x5b 0x31 0x3b 0x33 0x31 0x6d
```

Seven bytes. The first two are CSI. `1 ; 3 1` is the parameter list "1, 31". The final byte `m` says "this is an **SGR** command — Select Graphic Rendition". Parameter 1 within SGR means "bold"; 31 means "red foreground". The terminal emulator parses the seven bytes, updates its internal style for *future* text, and waits. The next ordinary bytes you send are drawn bold and red.

If we now write `Hi`, then send `ESC [ 0 m` to reset, the full transcript is:

Terminal

```
\x1b[1;31mHi\x1b[0m
```

This is what maya ultimately produces. Every layer of the library exists to spare you from writing those bytes by hand.

Idea

An escape sequence is `ESC [`, then a few numbers with semicolons, then one letter. The letter picks the family; the numbers parameterise it.

2.4.4 The families

The final byte categorises the command. The families you will see in maya's output, in rough order of frequency:

Family	Final byte	Purpose
SGR	`m`	graphic rendition: colour, bold, italic, etc.
CUP	`H` or `f`	move cursor to (row, column)
CUU / CUD	`A` / `B`	cursor up / down by n
CUF / CUB	`C` / `D`	cursor forward / back by n
CHA	`G`	cursor to absolute column
ED	`J`	erase part of the display
EL	`K`	erase part of the line
DECSET / DECRST	`h` / `l`	set / reset private modes (with `?`)

Three additional things to know.

ANSI columns and rows are 1-based. `ESC [ 5 ; 12 H` moves the cursor to row 5, column 12, where the top-left cell of the screen is $(1, 1)$, *not* $(0, 0)$. This is a constant source of off-by-one bugs; maya converts to/from 0-based internally and `move_to`'s signature is documented as 1-based at the boundary.

Private modes use `?`. DEC's extensions to ANSI use a question mark inside the parameter list to distinguish "private" parameters that only DEC-style terminals understand. `ESC [ ? 25 h` (show cursor) and `ESC [ ? 1049 h` (enter alternate screen) are private; the plain SGR `ESC [ 1 m` is not.

Numbers default to 1. If you write `ESC [ A` (just the family with no parameter), the parameter is implicitly 1 — "cursor up one row". Most families follow this convention; maya explicitly emits the number anyway, because being explicit avoids surprises across emulators.

2.4.5 SGR: colours and attributes

SGR is the family you will see most. It composes: `ESC [ 1 ; 31 ; 4 m` applies bold, red, and underline together. Then ordinary text is drawn with all three until another SGR command changes them.

The parameters you will care about:

```
ESC [ 0 m              // reset every attribute
ESC [ 1 m              // bold
ESC [ 2 m              // dim
ESC [ 3 m              // italic
ESC [ 4 m              // underline
ESC [ 7 m              // inverse (swap fg / bg)
ESC [ 9 m              // strikethrough

ESC [ 22 m             // bold + dim off
ESC [ 23 m             // italic off
ESC [ 24 m             // underline off
ESC [ 27 m             // inverse off
ESC [ 29 m             // strikethrough off

ESC [ 30..37 m         // 3-bit foreground (black..white)
ESC [ 40..47 m         // 3-bit background
ESC [ 90..97 m         // bright fg
ESC [ 100..107 m       // bright bg
ESC [ 39 m             // default fg
ESC [ 49 m             // default bg

ESC [ 38 ; 5 ; N m     // 256-colour fg, palette index N
ESC [ 38 ; 2 ; R ; G ; B m   // 24-bit truecolour fg
ESC [ 48 ; 2 ; R ; G ; B m   // 24-bit truecolour bg
```

Modern terminals all support truecolour (24 bits = 16,777,216 colours). maya's `Fg<R, G, B>` compiles to exactly `ESC [ 38 ; 2 ; R ; G ; B m`. If you read *maya/include/maya/terminal/ansi.hpp* you will find these as `constexpr std::string_view` constants:

```
inline constexpr std::string_view bold      = "\x1b[1m";
inline constexpr std::string_view italic    = "\x1b[3m";
inline constexpr std::string_view underline = "\x1b[4m";
inline constexpr std::string_view reset     = "\x1b[0m";
```

`constexpr` means the string literally lives in the binary; no allocation, no construction at startup. We will explain `string_view` and `constexpr` properly in Chapter 3.

2.4.6 Cursor movement and erasing

A short worked example: print "A" at row 3, column 5, then "B" at row 3, column 20:

Terminal

```
ESC [ 3 ; 5 H A ESC [ 3 ; 20 H B
```

Two CUP sequences sandwich the printable letters. Each is six bytes (seven, with the two-digit column): the cost of cursor positioning is exactly the length of the decimal numbers plus four (`ESC`, `[`, `;`, `H`). This is why diff-based rendering is so effective: most cursor moves are tiny.

For erasing, the three useful ED forms:

```
ESC [ 0 J     // erase from cursor to end of screen
ESC [ 1 J     // erase from start of screen to cursor
ESC [ 2 J     // erase entire screen
```

And EL, line-erasing:

```
ESC [ 0 K     // erase from cursor to end of line
ESC [ 1 K     // erase from start of line to cursor
ESC [ 2 K     // erase entire line
```

The cursor itself does not move when you erase — the cells under the cursor change to blank but the position stays. If you want the classic *clear-screen-and-home* effect, you send both: `ESC [ 2 J ESC [ H` (the `H` with no parameters means "move to row 1, column 1").

Pitfall

Erasing does not reset the colour. A cell "erased" to blank keeps its background colour. If you erased after setting a red background, you get a red rectangle. To truly blank the screen to the default, send `ESC [ 0 m` (reset colours) *before* erasing.

2.4.7 Modes: cursor, alt-screen, mouse, paste

A different style of command toggles *modes*: terminal-wide flags the emulator carries between sequences. These use the DEC private form with a `?`:

```
ESC [ ? 25 l          // hide the cursor
ESC [ ? 25 h          // show the cursor
ESC [ ? 1049 h        // enter alternate screen
ESC [ ? 1049 l        // leave alternate screen
ESC [ ? 1000 h        // mouse press/release events
ESC [ ? 1002 h        // mouse + drag motion events
ESC [ ? 1006 h        // SGR-encoded mouse coordinates (no 223-col limit)
ESC [ ? 2004 h        // bracketed paste (paste arrives wrapped)
ESC [ ? 2026 h        // begin synchronised update
ESC [ ? 2026 l        // end synchronised update
```

A few of these deserve names of their own.

The alternate screen. When you run `vim` or `less`, you notice that quitting restores whatever you had on screen *before* the editor started — the editor's contents do not pollute your scrollback. This is the alternate screen. The emulator keeps two screen buffers; `?1049 h` switches to the secondary one (saving the primary), `?1049 l` switches back. maya programs in fullscreen mode emit the enter sequence at startup and the leave sequence at exit, which is why they behave the same way.

Bracketed paste. When the user pastes a chunk of text, the emulator wraps it with the sentinels `ESC [ 200 ~` (before) and `ESC [ 201 ~` (after). Without this, a pasted multi-line block looks identical to the user typing very fast, and a TUI editor cannot tell the difference. With it, the editor sees: "a paste begins, here are the bytes, the paste ends" — and can treat the whole block atomically (no auto-indent per line, for example). maya surfaces this as a `Paste` variant of its `Event` type.

Synchronised update. A new and very useful mode. When `?2026 h` is sent, the terminal buffers all output until `?2026 l`. The frame is then drawn atomically — no tearing, no half-painted UI even on a slow connection. Modern terminals (kitty, WezTerm, foot, Ghostty, recent xterm) support this; older ones ignore the sequence harmlessly. maya's `env_supports_synchronized_output` in *terminal/ansi.hpp* detects this from environment variables and decides whether to wrap each frame in sync brackets.

2.4.8 Encoding non-ASCII keys: the Kitty Keyboard Protocol

Plain ANSI cannot tell `Ctrl+M` from `Enter` (both arrive as byte $0x0D$), or `Shift+Tab` from a regular escape sequence. This has been a chronic source of bugs in TUI editors for thirty years. The **Kitty Keyboard Protocol** (KKP) and the older `modifyOtherKeys=2` extension fix this by sending fully disambiguated key reports.

maya opts in on startup with two sequences:

```
ESC [ > 1 u          // push KKP flag 1 ("disambiguate")
ESC [ > 4 ; 2 m      // xterm modifyOtherKeys=2
```

and opts out on exit with `ESC [ < u` and `ESC [ > 4 m`. Terminals that do not understand the sequences ignore them silently; the worst case is the older behaviour. We discuss the input side of this in Chapter 13.

2.5 Cooked mode, raw mode, and `termios`

Now we turn from output to input. In a normal shell, what happens when you type a line of text?

The kernel sits between your keyboard and any program reading `stdin`. By default it is in **canonical** (cooked) mode:

- Keystrokes are buffered until you press `Enter`, then delivered as a whole line.
- Backspace edits the buffer in place (the program never sees the deletion).
- `Ctrl-C` is intercepted and sent as a *signal* (`SIGINT`) rather than as data.

- `Ctrl-D` on an empty line is interpreted as end-of-input.
- Each typed character is also *echoed* back to the screen automatically.

This is exactly the behaviour you want at a shell prompt. It is exactly the wrong behaviour for a TUI. A TUI wants every keystroke delivered immediately, byte for byte, with no automatic echo and no special handling of control characters.

The opposite mode is **raw mode**, configured via the `termios` structure — a C `struct` the kernel exposes through `tcgetattr` and `tcsetattr`.

2.5.1 The `termios` struct

`termios` has four flag fields plus a small array of special characters. Each flag field is a bitmask: bits enabled mean different behaviours active.

Field	Meaning
`c_iflag`	input-processing flags
`c_oflag`	output-processing flags
`c_cflag`	control flags (baud rate, parity — mostly serial relics)
`c_lflag`	local flags (canonical mode, echo, signals)
`c_cc[]`	special characters (which byte is INTR? EOF? ...)

To enter raw mode, you turn off a specific list of bits:

```c
#include <termios.h>
#include <unistd.h>

termios cooked, raw;
tcgetattr(STDIN_FILENO, &cooked);   // save the current settings
raw = cooked;                       // copy, mutate the copy

raw.c_lflag &= ~(ICANON | ECHO | ISIG | IEXTEN);
raw.c_iflag &= ~(IXON | ICRNL | INPCK | ISTRIP);
raw.c_oflag &= ~(OPOST);
raw.c_cc[VMIN]  = 0;     // non-blocking reads
raw.c_cc[VTIME] = 0;
tcsetattr(STDIN_FILENO, TCSAFLUSH, &raw);

// ... TUI runs here ...

tcsetattr(STDIN_FILENO, TCSAFLUSH, &cooked);   // restore
```

The bits, decoded:

- `ICANON` — canonical mode (line buffering). Off means every keystroke comes through immediately.
- `ECHO` — echo input. Off means typed characters do not appear on screen automatically; the program decides.
- `ISIG` — signal generation on `Ctrl-C` / `Ctrl-Z`. Off means those become normal bytes ($0x03$, $0x1A$).
- `IEXTEN` — implementation-defined extensions (`Ctrl-V` as "literal next", etc.). Off, just to be safe.

- `IXON` — flow control (`Ctrl-S` freezes the terminal, `Ctrl-Q` unfreezes). Off; you want those bytes yourself.
- `ICRNL` — translate `CR` ($0x0D$) into `LF` ($0x0A$) on input. Off, so you see exactly what was typed.
- `INPCK`, `ISTRIP` — parity checking and 7-bit stripping. Both relics from serial-line days.
- `OPOST` — output processing, mostly translating `LF` into `CR LF`. Off, because you will be writing full escape sequences yourself.
- `VMIN` $= 0$, `VTIME` $= 0$ — "return from `read` immediately if there is no input". The TUI loop polls; it does not block.

Pitfall

You must restore `cooked` on exit. If the program crashes before restoring, the user's shell will appear broken — typed characters will not echo, `Enter` will not commit, `Ctrl-C` will not work. The remedy is the magic command `stty sane`, which resets `termios` to sensible defaults. maya uses a RAII guard (see Chapter 3) so the destructor restores cooked mode even on exceptions.

2.5.2 What keystrokes look like in raw mode

In raw mode, a single keypress arrives as one or more bytes.

- Printable keys: one byte, their ASCII value. `a` arrives as $0x61$.
- Control keys: `Ctrl-A` is $0x01$, `Ctrl-B` is $0x02$, ..., `Ctrl-Z` is $0x1A$. (The pattern: clear the top three bits of the letter.)
- `Enter`: $0x0D$ (CR).
- `Tab`: $0x09$ (HT).
- `Backspace`: $0x7F$ (DEL).
- `Escape`: $0x1B$. By itself *or* as the start of a sequence.
- Function and arrow keys: an escape sequence. `Up` arrives as the three bytes `ESC [ A`; `Down` as `ESC [ B`; `Right` `ESC [ C`; `Left` `ESC [ D`.
- `Alt-`x: the bytes `ESC` then x.

The clash is obvious: `ESC` alone is a real keypress (some people bind it), but `ESC` is *also* the first byte of every arrow-key sequence. The standard trick is to read `ESC`, *wait a few milliseconds*, and if no more bytes arrive, treat it as a bare `Escape`. If they do, parse the sequence. maya's input parser (*src/terminal/input.cpp*) implements exactly this; the timeout is around 50 ms by default.

2.6 The screen as a grid

We have looked at bytes; now look at what those bytes update.

The emulator maintains, in its own memory, a 2-D grid of **cells**. Each cell holds:

- One Unicode *grapheme* (for most languages, one code point — a letter or a digit; for emoji and some scripts, a short cluster of code points that displays as one glyph).
- A foreground colour and a background colour.
- A set of attribute bits: bold, italic, underline, inverse, strikethrough, dim.

The cursor has a position (row, column) and a visibility flag. A **frame** of UI is a complete specification of every cell that should be on screen.

When the user resizes the window, the emulator sends the program a notification: on Linux/macOS, the `SIGWINCH` signal; on Windows, a console-resize event. The program re-queries the new size with `ioctl(TIOCGWINSZ)` and re-lays-out its UI. maya does this inside the runtime; you do not need to call anything.

2.6.1 Wide characters

A subtle wrinkle: not every Unicode character takes *one* cell.

- CJK ideographs (U+4E2D *zhōng*, U+3042 *hiragana a*) take **two** cells. Their glyph is roughly square; in a monospace font each one is twice as wide as a Latin letter.
- Many emoji also take two cells (U+1F600 smiley face).
- Combining marks — a `COMBINING ACUTE ACCENT` on top of an `e` to render *é* — take **zero** cells; they decorate the previous cell.

If the program writes a wide character but counts it as one cell, the emulator and the program disagree about the cursor position after, and the layout drifts. The Unicode Consortium publishes *EastAsianWidth.txt* listing the width of every code point; maya ships a generated table at *include/maya/text/unicode_width_table.hpp* and consults it when laying out text. You do not call into the table directly — maya's text widget does it for you.

2.7 Signals: `SIGWINCH`, `SIGINT`, and friends

A **signal** on Unix is a tiny asynchronous notification the kernel delivers to a process. The process can install a handler — a function that runs when the signal arrives — or it can use the default behaviour (often: "terminate"). Two signals matter to a TUI.

`SIGWINCH` — window-size change. When the user resizes the terminal window, the kernel delivers `SIGWINCH` to whichever process owns the foreground of the pty. The signal carries no payload — the process must re-ask the kernel for the new size. The mechanism is the system call `ioctl`, broadly "do a non-standard thing on a file descriptor". The request `TIOCGWINSZ` ("terminal IO control: get window size") fills a small struct with row and column counts:

```c
#include <sys/ioctl.h>
struct winsize ws;
ioctl(STDOUT_FILENO, TIOCGWINSZ, &ws);
// ws.ws_row, ws.ws_col, ws.ws_xpixel, ws.ws_ypixel
```

maya's runtime installs a `SIGWINCH` handler that simply sets an atomic flag; the main loop checks the flag, re-queries the size, and re-runs layout. Doing real work inside a signal handler is dangerous (most library functions are not safe to call from one); flagging-then-handling is the standard idiom.

`SIGINT` — interrupt. Delivered by the kernel when the user presses `Ctrl-C` *in cooked mode*. In raw mode (which maya sets), `Ctrl-C` becomes the byte `0x03`, which the program sees as ordinary input and may interpret however it likes. maya also installs a backup `SIGINT` handler so that if something forces cooked mode back on mid-run (a misbehaving sub-process, for example), a Ctrl-C still cleans up the terminal state before exiting.

`SIGTERM`, `SIGHUP`, `SIGSEGV`. maya's terminal driver installs handlers for all three so that catastrophic exits still restore the cooked-mode terminal before the process dies. Without this, killing a TUI with `kill <pid>` or letting it crash would leave the user's shell in raw mode. RAII is not enough alone; the destructor only runs if the stack unwinds, and `SIGSEGV` does not unwind. The signal handler is a belt-and-braces backup.

2.7.1 What `ioctl` actually is

The "standard" file operations are `open`, `read`, `write`, `close`. `ioctl` ("IO control") is the escape hatch for everything else: "perform request `X` on file descriptor `fd`, with these arguments". Hundreds of requests exist; each one is essentially a separate API hiding behind a common name. You meet `ioctl` mostly for terminals (`TIOCGWINSZ`), sockets, and device drivers.

In portable code you would prefer dedicated APIs; in practice no such dedicated API exists for "get terminal size", so `ioctl` with `TIOCGWINSZ` is the only path.

2.8 Mouse reporting, on the wire

Mouse input is also delivered as escape sequences. When mouse reporting is enabled (`ESC [ ?1000 h` or one of its richer variants), every mouse event arrives as a CSI sequence the program must parse.

The modern format is **SGR mouse reporting** (`?1006 h`):

`ESC [ < button ; column ; row M` (press / drag)
`ESC [ < button ; column ; row m` (release)

`button` is a small bitfield: low bits encode left/middle/right (0/1/2); higher bits flag `Shift`, `Alt`, `Ctrl` modifiers (4, 8, 16); a bit at position 32 marks "motion event" (this is a drag, not a click); a bit at position 64 marks wheel scroll. `column` and `row` are 1-based.

Example. A left-click at (col 12, row 5):

Terminal

```
ESC [ < 0 ; 12 ; 5 M
```

A Shift-drag of the same button to (15, 5):

Terminal

```
ESC [ < 36 ; 15 ; 5 M
```

(Button code $36 = 0 + 4$ (Shift) $+ 32$ (motion).)

A wheel-scroll down at (40, 10):

Terminal

```
ESC [ < 65 ; 40 ; 10 M
```

(Button code $65 = 1 + 64$ (wheel); the low bit on wheel means "down". Wheel up would be 64; left/right wheel use 66/67.)

maya's event parser decodes these into structured `Mouse` events. The widget layer then routes them by position to the widget under the cursor.

Why the SGR form?

The old X10 mouse-reporting format (`?1000 h`) encoded the column and row as *single bytes*, with an offset of 32 (so the byte for column 1 was 33, the byte for column 12 was 44). It broke on column or row numbers above $223 - 32 = 191$, because the bytes were only 7-bit-safe and the offsetting overflowed. Modern wide terminals routinely exceed 200 columns; the SGR form, with decimal coordinates, has no such limit. maya enables both `?1000 h` and `?1006 h` on startup, the latter overrides; emulators that do not understand `?1006 h` fall back gracefully to the older encoding.

2.9 Bracketed paste, on the wire

When `?2004 h` is enabled and the user pastes a block of text into the terminal, the emulator wraps the pasted bytes in two sentinel sequences:

Terminal

```
ESC [ 200 ~  ...  pasted bytes ...  ESC [ 201 ~
```

The content between can be arbitrary, including newlines and other control characters — the program should treat the whole block as one atomic event.

Example. Pasting the two lines

Terminal

```
foo
bar
```

produces on the wire:

Terminal

```
ESC [ 200 ~ f o o LF b a r ESC [ 201 ~
```

Without bracketed paste, that arrival is indistinguishable from a very fast user typing `f`, `o`, `o`, `Enter`, `b`, `a`, `r`. With it, an editor knows to disable auto-indent, treat the whole block as a single undo unit, and so on. maya's input parser produces a single `Paste` event carrying the entire content as a `std::string`.

2.10 The input parser as a state machine

We sketched the input parser earlier as "a state machine in the other direction". Concretely: it reads bytes from `stdin` and builds structured `Event` values. Here is its core loop in pseudo-code:

```
enum State { Ground, AfterEsc, InCsi };
State s = Ground;
std::string params;          // accumulator for CSI parameter bytes
for (;;) {
    byte b = read_one();
    switch (s) {
      case Ground:
        if (b == 0x1B) { s = AfterEsc; start_esc_timeout(50ms); }
        else           emit_key(b);
        break;
      case AfterEsc:
        if (b == '[')  { s = InCsi; params.clear(); }
        else if (timed_out) { emit_key(Escape); s = Ground; pushback(b); }
        else           { emit_key_alt(b); s = Ground; }
        break;
      case InCsi:
        if (is_param_byte(b)) params.push_back(b);
        else if (is_final_byte(b)) {
            dispatch_csi(params, b);   // → emit Arrow, Mouse, Paste, ...
            s = Ground;
        }
        break;
    }
}
```

A single keystroke can therefore become a single `Event` (`A` arrives, `KeyKind::Char 'A'` emitted) or take a few bytes (`ESC [ A` becomes `KeyKind::Up`) or many (a paste of a hundred bytes is one `Paste` event). The 50 ms timeout on `ESC` is the only piece that is not a pure function of the byte stream — it disambiguates a bare `Escape` from an incomplete escape sequence.

maya's real parser is more elaborate (it handles modified arrows, the Kitty Keyboard Protocol's CSI-u format, `modifyOtherKeys=2`, function keys, focus events, OSC responses), but every additional case follows the same state-machine shape.

2.11 Putting it together: a no-library Hello world

We now know enough to write a complete TUI "Hello, red world" with no library at all. It is a useful exercise: you will see exactly what maya is doing for you.

```
#include <stdio.h>
#include <termios.h>
#include <unistd.h>
```

```c

int main(void) {
    termios cooked, raw;
    tcgetattr(STDIN_FILENO, &cooked);
    raw = cooked;
    raw.c_lflag &= ~(ICANON | ECHO);
    tcsetattr(STDIN_FILENO, TCSAFLUSH, &raw);

    // 1. Hide the cursor
    // 2. Clear the screen
    // 3. Move to row 1, column 1
    // 4. Set bold red foreground
    // 5. Write "Hi"
    // 6. Reset attributes
    // 7. Show the cursor again
    write(STDOUT_FILENO,
          "\x1b[?25l"          // hide cursor
          "\x1b[2J"            // erase entire screen
          "\x1b[1;1H"          // cursor to (row 1, col 1)
          "\x1b[1;31m"         // bold red foreground
          "Hi"
          "\x1b[0m"            // reset attributes
          "\x1b[?25h",         // show cursor
          /*len=*/ 33);

    char c;
    read(STDIN_FILENO, &c, 1);   // wait for any keypress

    tcsetattr(STDIN_FILENO, TCSAFLUSH, &cooked);
    return 0;
}
```

Save it as *hello.c*, build with `cc hello.c -o hello`, run `./hello`. The screen clears; "Hi" appears in bold red at the top-left; press any key; the screen returns. You have just written a TUI by hand.

This is also a faithful caricature of what maya does *every frame*. The whole library is in service of generating exactly this style of byte stream, only longer, more carefully diffed, and described to you as a tree of styled elements instead of a literal string.

2.12 A real maya frame, traced byte by byte

To close the chapter, here is a hand-traced version of one incremental redraw in a maya program. The screen currently shows the word `Count:  7` on row 5, starting at column 3, in the default white foreground. The user pressed `+`; maya wants the screen to now read `Count:  8`. Only one cell changed — column 10 of row 5 — so maya emits only what is needed to update it.

The bytes maya writes:

Terminal

```
\x1b[?2026h  ESC [ ?2026 h   begin synchronised update
\x1b[5;10H  ESC [ 5;10 H   move cursor to row 5, col 10
\x1b[39;49m  ESC [ 39;49 m   default fg & bg (style reset)
8                    the new digit
\x1b[?2026l  ESC [ ?2026 l   end synchronised update
```

Byte-by-byte that is:

Terminal

```
1B 5B 3F 32 30 32 36 68 1B 5B 35 3B 31 30 48 1B
5B 33 39 3B 34 39 6D 38 1B 5B 3F 32 30 32 36 6C
```

32 bytes in total to redraw the change. Compare to what a naive renderer would do: clear the screen, redraw every cell. For an 80 by 24 terminal that is roughly $80 \times 24 \times 10$ bytes (most cells take around ten bytes including SGR and the character), or close to 20 kB. Two orders of magnitude saved by the diff.

The sync wrapper (`?2026 h` ... `?2026 l`) is optional and omitted by terminals that do not advertise it; maya's renderer checks an environment-derived hint (`env_supports_synchronized_output` in *terminal/ansi.hpp*) and elides the wrapper if the terminal will not honour it.

The cursor positioning (`ESC [ 5;10 H`) and the style transition (`ESC [ 39;49 m`) are produced by maya's `StyleApplier::transition_to` helper, which writes the minimal SGR change from the previous cell's style to the new one (in this case, the previous cell on the row was already styled the same, so the SGR transition would actually be empty — we include it here for illustration).

Idea

The whole maya rendering pipeline exists to make this incremental byte stream short and correct. Everything from the canvas to the SIMD diff to the ANSI builder is in service of: "write the minimum bytes needed to update only the cells that actually changed".

Recap

A terminal is a state machine on the far end of a pty. Bytes flowing into it are mostly literal text, with a small number of control characters and the ANSI escape language (`ESC [` *params letter*) for everything else. Cooked mode lets the kernel pre-process input; raw mode hands bytes through unchanged. The screen is a 2-D grid of styled Unicode cells. Every layer above this in maya speaks in C++ values; this one alone speaks in bytes.

Workshop

1. **Inspect your terminal's flags.** Run `stty -a` in a normal shell. Find the lines beginning with `iflag` and `lflag`. Identify `icanon` and `echo` (with or without a leading `-`, which means "negated"). Those are the bits a TUI clears.

2. **Identify maya's constants.** Open *maya/include/maya/terminal/ansi.hpp* (we read it above). Find:

   - the constant for entering the alternate screen;
   - the constant for enabling SGR mouse reporting (1006);
   - the constant for the Kitty Keyboard Protocol push.

   Cross-check each one against the table earlier in this chapter.

3. **Drive a terminal by hand.** In a scratch C++ file:

```cpp
#include <cstdio>
#include <unistd.h>
int main() {
```

```
    std::fputs("\x1b[?1049h", stdout);     // enter alt-screen
    std::fputs("\x1b[1;1HHello", stdout);  // print at top-left
    sleep(2);
    std::fputs("\x1b[?1049l", stdout);     // leave alt-screen
}
```

Build and run. Observe that your shell is exactly as you left it afterwards.

4. **Type the no-library example.** Compile and run the C program in this chapter. Then mutate it: change `31` (red) to `32` (green), `1` (bold) to `4` (underline). Each change is one digit.

5. **Provoke a raw-mode mistake.** Comment out the final `tcsetattr` that restores cooked mode. Recompile and run. Your shell will behave strangely after. Recover by typing `stty sane`, then `Enter` (you may need to type blind). You now appreciate the RAII guard.

Modern C++ for maya

This chapter is a self-contained course in the slice of C++ that maya uses. It assumes no C++ background. If you have written C++11 or C++14 it will still be useful, because maya leans on features that arrived in C++20, C++23, and C++26; some of the patterns will be new even to experienced C++ programmers.

Read it linearly the first time. The sections build on each other: later ones (templates, concepts, variants) lean on earlier ones (values, references, classes). When you finish, you will be able to read every line of maya source code.

3.1 What a C++ program is

A C++ program is one or more **source files** compiled together into an **executable**. The smallest possible one fits on one line, although by convention we spread it out:

```
#include <print>

int main() {
    std::println("Hello, world");
}
```

Four things are happening, and each deserves a paragraph.

The preprocessor runs first. Before the compiler proper looks at your code, a small program called the **preprocessor** does textual substitution. Lines beginning with `#` are its instructions. `#include <print>` literally means "find the file named *print* in the standard library's directories and paste its contents here". After the preprocessor finishes, your source file has grown to several thousand lines, all the standard-library declarations now visible.

The compiler reads what is left. The expanded text is parsed, type-checked, and translated into **object code** — machine instructions in a not-yet-runnable format. Each source file becomes one *object file*, also called a **translation unit**. A small project has one; maya has several hundred.

The linker stitches object files together. The **linker** takes all object files plus any libraries you depend on and produces a single executable. Its main job is resolving **symbols**: when one file uses a function defined in another file, the linker matches the use to the definition.

The operating system runs the result. When you type `./hello`, the OS loads the executable into memory and jumps to a specific named function: `main`. The integer `main` returns is the process's *exit status* (zero = success). The whole program ends when `main` returns.

Idea

The preprocessor pastes text. The compiler turns text into object files. The linker joins object files into an executable. The OS runs the executable from `main`.

3.1.1 Headers, source files, and the one-definition rule

A **header file** (suffix *.hpp* or *.h*) is a file you `#include` into other files. It typically contains *declarations* — statements about what exists, but not how it works. A **source file** (suffix *.cpp*) typically contains *definitions* — the actual function bodies and data.

```cpp
// math.hpp --- declaration
int add(int a, int b);
```

```cpp
// math.cpp --- definition
#include "math.hpp"
int add(int a, int b) { return a + b; }
```

```cpp
// main.cpp --- user
#include "math.hpp"
int main() { return add(2, 3); }
```

Every use of `add` in any source file sees the same declaration (from *math.hpp*), and there is exactly one definition in the whole program (in *math.cpp*). That is the **One Definition Rule**, ODR: every entity must have exactly one definition across all translation units linked together.

maya is a *header-mostly* library. Almost all its code lives in headers, with most functions and classes defined `inline` or as templates. This is unusual and has trade-offs (compile times can be long), but it lets the compiler aggressively inline and constant- fold across the library boundary. We discuss the trade-off in Chapter 16.

3.1.2 `#pragma once` and include guards

If two headers include the same third header, the preprocessor would paste it twice, producing duplicate declarations. The traditional fix is an **include guard**:

```cpp
// math.hpp
#ifndef MATH_HPP
#define MATH_HPP
int add(int a, int b);
#endif
```

The second inclusion finds `MATH_HPP` already defined and skips the contents. Modern code uses the shorter and equivalent:

```cpp
// math.hpp
#pragma once
int add(int a, int b);
```

Every file in maya starts with `#pragma once`.

3.1.3 How to actually compile and run a C++ file

You will get more out of the rest of this chapter if you can try things as you read. The minimum tool set is a compiler and a terminal. On Linux or WSL:

```
sudo apt install g++-15        # or your distro's equivalent
echo 'int main() {}' > t.cpp
g++-15 -std=c++26 t.cpp -o t   # compile
./t                            # run
```

On macOS install a recent Homebrew `llvm` and use `clang++` with `-std=c++26`. On Windows the easiest path is WSL with a current Ubuntu.

The useful compiler flags for learning:

- `-std=c++26` — enable C++26. `maya` requires it; C++20 or C++23 will not compile the DSL.
- `-Wall -Wextra` — turn on most warnings. Treat warnings as bugs while learning.
- `-g` — include debug information, so a stack trace in `gdb` or `lldb` points at your code.
- `-O2` — ask for optimisation. Off by default; turn it on when you measure performance.
- `-I path/to/headers` — add a directory to the include search path. `maya` consumers pass `-I maya/include`.
- `-fsanitize=address,undefined` — compile in run-time checks for memory bugs and undefined behaviour. Free safety net while learning.

For a one-liner sanity check that your compiler is recent enough:

```
echo '#include <print>\nint main(){std::println("ok");}' \
  | g++-15 -std=c++26 -x c++ - -o ok && ./ok
```

If that prints `ok`, you can run every example in this book.

3.1.4 A first build session, narrated

For concreteness, here is what a real first session at the terminal looks like. We will write a tiny C++ file, compile it, deliberately introduce an error to see what the compiler says, fix it, and run.

```
$ cat > greet.cpp <<'EOF'
#include <print>

int main() {
    std::println("hi, world");
}
EOF

$ g++-15 -std=c++26 -Wall -Wextra greet.cpp -o greet
$ ./greet
hi, world
```

Good. Now break it on purpose — forget the semicolon after `println`:

```
$ cat > greet.cpp <<'EOF'
#include <print>

int main() {
    std::println("hi, world")
}
EOF
```

```
$ g++-15 -std=c++26 -Wall -Wextra greet.cpp -o greet
greet.cpp: In function 'int main()':
greet.cpp:4:32: error: expected ';' before '}' token
    4 |     std::println("hi, world")
      |                                ^
      |                                ;
    5 | }
      | ~
```

Note the shape: file name, line number, column, error kind, the offending line printed with a carat pointing where the compiler thinks the fix goes. Reading those four pieces of information is a skill; most beginners panic at the wall of text and miss that the compiler is being quite specific.

Fix the semicolon, rebuild, run. That is the entire edit-build-run loop. The rest of this chapter explains what to write between `int main() {` and `}`.

3.2 Statements, blocks, control flow

Before we go further into types, a quick survey of the syntactic furniture every C++ program is built from. If you have written any imperative language — Python, JavaScript, Java, C, Go — almost nothing here is new; if you have not, read carefully.

3.2.1 Statements and blocks

A **statement** is one unit of work. Most statements end with a semicolon:

```
int x = 0;
x = x + 1;
std::println("x is {}", x);
```

A **block** is a sequence of statements wrapped in braces. It introduces a new scope — names declared inside the block disappear at the closing brace:

```
int x = 0;
{
    int y = 5;      // y exists only inside this block
    x += y;
}
// y is gone here; x is 5
```

Function bodies, loop bodies, `if` branches — all are blocks.

3.2.2 `if` and `else`

```
if (x > 0) {
    std::println("positive");
} else if (x < 0) {
    std::println("negative");
} else {
    std::println("zero");
}
```

The condition must be a `bool` (or something that converts to one — an `int`, a pointer). C++17 added **if-init**: you can declare a variable in the condition itself:

```cpp
if (auto it = map.find(key); it != map.end()) {
    use(it→second);
}
// it is not in scope here
```

The form `if (init; cond)` keeps the helper variable confined to the `if`/`else` blocks. maya uses this heavily.

3.2.3 Loops: `while`, `do-while`, `for`

```cpp
while (cond) {
    // body, repeated as long as cond is true
}

do {
    // body, runs at least once, then while-loops
} while (cond);

for (int i = 0; i < 10; ++i) {
    // classic C-style for: init; cond; step
}
```

The modern **range-for** (C++11) iterates over any container or range:

```cpp
std::vector<int> xs = {1, 2, 3};
for (int x : xs)        std::println("{}", x);   // copy each element
for (int& x : xs)       ++x;                    // mutate in place
for (const auto& x : xs) std::println("{}", x);  // read-only, no copy
```

The `const auto&` form is the most common in maya: no copy, no surprises.

3.2.4 `break` and `continue`

`break` exits the innermost loop. `continue` skips the rest of the current iteration and goes to the next.

```cpp
for (int i = 0; i < 10; ++i) {
    if (i == 3) continue;   // skip 3
    if (i == 7) break;      // stop entirely
    std::println("{}", i);   // prints 0 1 2 4 5 6
}
```

3.2.5 `switch`

```cpp
switch (kind) {
    case Kind::Red:    handle_red();    break;
    case Kind::Green:  handle_green();  break;
    case Kind::Blue:   handle_blue();   break;
    default:           handle_other();  break;
}
```

The argument must be an integral or enum value. Each `case` label falls through to the next unless terminated with `break` (or `return`). C++ accepts the fall-through; modern compilers warn about unintended ones, and the attribute `[[fallthrough]];` silences the warning for intentional cases.

In maya, `switch` is used on small `enum class` kinds (error kinds, key kinds). For closed sum types, `std::visit` on a variant is preferred — it forces exhaustiveness in a way `switch` cannot.

3.2.6 Expressions and statements

Many things look like statements but are actually **expressions** — they produce a value. `x + 1`, `f(3)`, `a == b`, `x ? y : z` (the ternary conditional). Assignment is also an expression; `(x = 5)` has value 5. This rarely matters until you meet a chained assignment or a condition like `while ((c = read()))`.

3.3 Built-in types and integer widths

C and C++ have a long list of arithmetic types, mostly inherited from 1970s hardware. You only need a few.

Type	Typical size	Use for
`bool`	1 byte	truth values
`char`	1 byte	a byte / ASCII letter
`int`	4 bytes	ordinary integers
`long`	4 or 8	avoid; use a fixed-width type
`long long`	8	avoid; use `std::int64_t`
`float`	4	single-precision floating-point
`double`	8	default floating-point
`std::size_t`	pointer-sized	sizes, indices
`std::ptrdiff_t`	pointer-sized	signed sizes, pointer differences
`std::uint8_t`	1	a byte you do arithmetic on
`std::int32_t`	4	an exactly-32-bit signed integer
`std::int64_t`	8	an exactly-64-bit signed integer

Two rules:

- Use `std::size_t` for sizes and indices into containers. `std::vector::size()` returns it; comparing it against a signed `int` warns.
- Use the fixed-width types (`std::int32_t`, etc.) when you care about the exact bit count — file formats, hash functions, packed cell layouts.

maya's canvas cell, for example, is a `std::uint64_t` — exactly 64 bits, the same on every platform. The diff pass loads it into a SIMD register; that requires a known size.

3.3.1 Signed vs unsigned

`int` is signed; `unsigned int` is not. The difference matters for two reasons:

1. **Overflow.** Signed overflow is *undefined behaviour*: the compiler may produce nonsense if you cause it. Unsigned overflow is defined: it wraps modulo 2^N.
2. **Comparison.** Mixing signed and unsigned in a comparison surprises everyone. `-1 < 0u` is *false*: the `-1` is converted to `unsigned` first and becomes a huge positive number.

Use signed types unless you genuinely need the wrap-around semantics of unsigned — bit twiddling, hashing, modular arithmetic, sizes.

3.3.2 Character types

`char` is one byte. Whether it is signed or unsigned is implementation-defined — never assume. `signed char` and `unsigned char` explicitly specify. For UTF-8 byte buffers, `std::uint8_t` is the clearest choice; for ASCII text, `char` is fine.

C++ also has `char8_t`, `char16_t`, `char32_t` for wider Unicode — you will see them rarely.

3.3.3 Literals

Number literals have suffixes that fix their type:

```
42          // int
42u         // unsigned int
42L         // long
42LL        // long long
42ULL       // unsigned long long
3.14        // double
3.14f       // float
0x2A        // hex (= 42)
0b101010    // binary (= 42)
0'042       // digit separator allowed --- = 042 (octal = 34)
1'000'000   // a million, with separators for readability
```

String literals: `"hello"` is a `const char[6]` (the five letters plus a trailing `\0`). `"hello"sv` (with the `sv` suffix, after `using namespace std::string_view_literals`) is a `std::string_view`.

3.4 Namespaces

A **namespace** is a labelled container of names. It exists to prevent collisions: every library can put its types inside its own namespace, and no one tramples on anyone else's identifiers.

```
namespace mylib {
    int add(int a, int b) { return a + b; }
    struct Point { double x, y; };
}

int main() {
    int n = mylib::add(2, 3);
    mylib::Point p{1.0, 2.0};
}
```

The scope-resolution operator `::` reaches into a namespace. `std::vector` means "`vector` inside the `std` namespace". The standard library lives entirely in `std`.

Namespaces nest:

```
namespace maya {
    namespace ansi {
        constexpr std::string_view bold = "\\x1b[1m";
    }
}

// or, equivalently (C++17):
namespace maya::ansi {
    constexpr std::string_view bold = "\\x1b[1m";
}
```

You use it as `maya::ansi::bold`.

3.4.1 `using` declarations

Typing `maya::dsl::` in front of every DSL identifier would be tedious. Three ways to abbreviate:

```
using maya::dsl::Bold;          // bring just one name into scope
using namespace maya::dsl;      // bring every name into scope
namespace md = maya::dsl;       // alias the whole namespace
```

`using namespace X` is convenient at the top of a `.cpp` file but **never** at namespace scope in a header — you would leak every name into every file that includes the header.

Maya's public namespaces:

- `maya` — public API (`print`, `run`, `Element`, `Signal`, `Cmd`).
- `maya::dsl` — the compile-time DSL pieces (`t<"...">`, `Bold`, `Fg<...>`, `v`, `h`, the pipe operators).
- `maya::ansi` — raw escape sequence constants and builders.
- `maya::widget` — the widget catalogue.

Example programs begin with `using namespace maya; using namespace maya::dsl;` for brevity.

3.5 Initialisation: the many forms

C++ has *four* syntaxes for initialising a variable. This is the single most baffling piece of beginner C++. We will lay them out, and then give you a rule.

```
int a;            // default initialisation: uninitialised garbage!
int b = 5;        // copy initialisation
int c(5);         // direct initialisation
int d{5};         // direct list initialisation ("brace init")
auto e = 5;       // auto + copy form
auto f = int{5};  // auto + brace form
```

For a built-in `int`, the second through fifth are equivalent. For classes, the difference matters in two places.

3.5.1 Default initialisation can leave garbage

`int a;` declares `a` but does *not* give it a value. Reading `a` before you assign to it is undefined behaviour. This is one of the most common beginner bugs.

Use `int a{};` or `int a = 0;` to be explicit. `T x{}` **value-initialises** `x` for any `T` — zero for built-ins, default-constructor for classes.

3.5.2 Brace init avoids narrowing

```
int    a = 3.7;     // ok, silently truncates to 3
int    b{3.7};      // ERROR: narrowing conversion not allowed
```

Braces forbid lossy conversions. This catches real bugs.

3.5.3 The "most vexing parse"

```
Widget w();         // looks like a default construction, but...
                    //   ... it is actually a FUNCTION DECLARATION
                    //   for a function w taking nothing,
                    //   returning Widget.
Widget w;           // correct default construction
Widget w{};         // correct, explicit
```

Use braces or empty parentheses where you really mean to construct.

3.5.4 The rule

For the rest of this book, the rule is: **prefer braces**.

```
int            n{0};
std::string    s{"hello"};
std::vector<int> v{1, 2, 3};
Point          p{3.0, 4.0};
auto           m = std::map<int, std::string>{{1, "a"}, {2, "b"}};
```

The one common exception: when calling a constructor whose arguments should *not* be interpreted as a list, you sometimes need parentheses. `std::vector<int> v{10, 0}` is a vector of two elements $[10, 0]$; `std::vector<int> v(10, 0)` is a vector of ten zeros. Read the API docs when this comes up.

3.6 Values, references, pointers, `const`

C++ forces you to be precise about *where* a piece of data lives and *who can change it*. This precision is what makes C++ fast and what makes beginners stumble. We will walk it slowly.

3.6.1 Values

A **value** is just data: `int x = 42;` creates a variable `x` of type `int` holding the value 42. The variable owns its storage. When `x` goes out of scope (the block ends), the storage is gone.

```
int x = 42;          // x owns 4 bytes holding 42
int y = x;           // y owns its own 4 bytes, copy of x
y = 100;             // x is still 42
```

This is the **value semantics** model: every assignment is a copy, every variable is its own thing. The same applies to classes:

```
std::string a = "hello";
std::string b = a;   // b is its own string, copy of a\'s characters
b += " world";
// a == "hello", b == "hello world"
```

For a `std::string` this is a real copy of every character. That sounds wasteful and sometimes is; we will see `std::move` shortly as the way to opt out of copies when you no longer need the original.

3.6.2 References

A **reference** is an alias for an existing value. It is *not* a separate variable; it is another name for the same storage.

```
int x = 42;
int& r = x;        // r is an alias for x
r = 100;
// x is now 100
```

References must be bound when created and cannot be re-seated. `int& r;` alone is not legal — a reference always names something.

References are the standard way to pass large values into a function without copying:

```
void print_long(const std::string& s) {
    std::println("{}", s);     // s is the caller\'s string
}                                  // no copy, no destruction
```

The `const` qualifier on a reference promises "I will not modify the thing you let me look at". `const T&` is the idiomatic way to receive read-only inputs of any non-trivial type.

3.6.3 Pointers

A **pointer** is a value that holds an address — the location of another value in memory. The syntax `T*` means "pointer to a `T`". You dereference with `*`.

```
int x = 42;
int* p = &x;         // p holds the address of x
*p = 100;            // write through the pointer
// x is now 100
```

Unlike references, pointers *can* be re-seated, can be null (`nullptr`), and can point at nothing meaningful (a real source of bugs). `maya` avoids raw pointers in user-facing APIs. Internally, it uses them for things that genuinely model "maybe points to something": linked lists, lookup tables, parent links in a tree.

When you do see a pointer in `maya`, prefer the modern alternatives: `std::unique_ptr<T>` (single owner, automatically destroyed), `std::shared_ptr<T>` (reference-counted shared owner), `std::optional<T>` (a value that may or may not be present, no heap allocation), `std::span<T>` (a non-owning view of a contiguous range), `std::string_view` (the same for character data).

3.6.4 `const` on values and references

A `const` variable cannot be reassigned:

```
const int n = 5;
// n = 10;          // error: cannot assign to const
```

A `const` reference promises read-only access:

```
void f(const std::string& s) {
    // s.push_back('!');  // error: s is const here
    std::println("{}", s);
}
```

A `const` method on a class promises not to modify the object:

```cpp
struct Counter {
    int n = 0;
    int get() const { return n; }   // const method, ok to call on const objects
    void inc()       { ++n; }
};

const Counter c;
c.get();           // ok
// c.inc();        // error: cannot call non-const method on const object
```

`const`-correctness is a deceptively powerful discipline: it lets the compiler check at compile time that your read-only paths really are read-only.

Recap

A value owns its storage. A reference is an alias; a pointer holds an address. `const` marks read-only at every level: variable, reference, method.

3.7 Stack and heap

When you write `int x = 42;`, where does `x` actually live? C++ programs have two main regions of memory you should know about.

The stack. A region of memory managed implicitly, like a stack of plates. Every function call pushes a new **stack frame** on top: storage for the function's parameters, its local variables, and bookkeeping for the return address. When the function returns, its frame is popped off. Allocation and deallocation are essentially free — a single register adjustment.

Local variables go on the stack:

```cpp
void f() {
    int x = 42;            // x is in f's stack frame
    std::array<int, 8> a;  // also on the stack: 32 bytes here
}                          // both gone when f returns
```

The stack is small — typically 1 MB on Linux, 512 kB on macOS by default. You cannot put very large objects on it (a million-element array of `int` would not fit), and you cannot have data outlive the function it was created in.

The heap. A region of memory managed explicitly. You ask the OS for a chunk with `new T(...)` or `std::make_unique<T>(...)` or any standard-library container that allocates internally (`std::vector`, `std::string`, `std::map`, ...). The chunk persists until you free it (with `delete`, or by letting the smart pointer / container destructor do it).

```cpp
void f() {
    auto p = std::make_unique<int>(42);      // 4 bytes on the heap
    auto v = std::vector<int>(1'000'000);    // 4 MB on the heap, owned by v
}                                             // both freed when p and v go away
```

The heap is large (gigabytes), and data on it can live as long as you like. Allocation and deallocation are real work — the OS needs to find a block, mark it used, eventually mark it free again. A program that allocates millions of small heap blocks will be noticeably slower than one that does not.

The rule. Use the stack for everything you can (locals, parameters, small fixed-size containers like `std::array`, small classes). Use the heap for things that are large, dynamically-sized, or need to outlive the function that created them. Standard containers automatically use the heap for their internal storage; you do not have to call `new` explicitly.

maya's hot path tries hard to keep allocations down. The canvas is allocated once and reused; the renderer's output buffer is allocated once and reused; the diff loop touches no heap at all. This is what makes maya feel fast on a slow link.

3.8 Functions, overloads, default arguments

A function takes inputs, may return an output, and may have side effects:

```
int square(int n) { return n * n; }
void greet(std::string_view name) {
    std::println("hi, {}", name);
}
```

The return type goes first, then the name, then the parameters in parentheses. `void` means "returns nothing".

3.8.1 Parameter passing

A parameter declared by value (`int n`) is a fresh copy. A parameter declared by reference (`const T&`) shares storage with the caller's value. The rule of thumb:

- Trivial small types (`int`, `double`, `std::string_view`, `std::span<T>`) — by value. Copying is cheap.
- Larger types you only read — by `const` reference. No copy.
- Types you intend to modify in place — by non-`const` reference. The caller sees the change.
- Types you want to take ownership of — by value (and let the caller `std::move` into the call).

3.8.2 Overloading

Multiple functions can share a name if their parameter lists differ. This is **overloading**:

```
void log(int n);
void log(double d);
void log(std::string_view s);

log(42);          // calls log(int)
log(3.14);        // calls log(double)
log("hello");     // calls log(string_view)
```

The compiler picks the best match. If two are equally good, you get a compile error and you spell out the choice.

3.8.3 Default arguments

A parameter can have a default:

```cpp
void greet(std::string_view name, std::string_view greeting = "hi") {
    std::println("{}, {}", greeting, name);
}

greet("world");            // "hi, world"
greet("world", "hello");   // "hello, world"
```

Defaults are part of the declaration; later overloads of the same name cannot redeclare them.

3.9 Strings: three kinds, and when to use each

Text handling has three core types in C++. Choosing the right one for each parameter and return type matters more than beginners realise.

`const char*` — the C string. A pointer to a null-terminated array of bytes. The `\0` byte signals the end. This is what `"hello"` actually *is* in C++ (a `const char[6]`, which decays to `const char*` when passed around).

```cpp
const char* name = "alice";   // points at static read-only memory
std::printf("%s\n", name);     // C-style I/O still takes const char*
```

Use `const char*` when interfacing with C APIs. Avoid for new C++ code.

`std::string` — the owning string. A class that owns a heap-allocated, dynamically resizable buffer of bytes. It is the workhorse:

```cpp
#include <string>
std::string s = "hello";        // copy of the literal, heap-allocated
s += ", world";                  // grows the buffer if needed
std::println("{}", s);
```

Use `std::string` when you need to own and mutate string data. It is a real allocation; do not copy it casually.

`std::string_view` — the non-owning view. Two pointers under the hood: "start" and "end" (or "start" and "length"). Refers to a contiguous range of bytes *owned by someone else*.

```cpp
#include <string_view>
void greet(std::string_view name) {
    std::println("hi, {}", name);
}

greet("alice");                  // works: literal becomes string_view
std::string s = "bob";
greet(s);                        // works: string converts to string_view
```

A `string_view` does not allocate, does not copy, and does not own. The catch: the data it points at must outlive the view. Storing a `string_view` as a class member is risky; using one as a function parameter is almost always safe (the caller's data outlives the call).

The rule.

- Function takes a string parameter, read-only? `std::string_view`.

- Function returns or stores a string it owns? `std::string`.
- Interfacing with a C API? `const char*`, often via `s.c_str()` on an existing `std::string`.

In maya, you will see `std::string_view` everywhere a function reads a string, and `std::string` where the type owns the data (e.g. in `Cmd<Msg>::SetTitle`, which owns the title string).

3.10 enum class: typed enumerations

An **enumeration** is a small named set of integer constants. The old C-style form was `enum Color { Red, Green, Blue };`, which defined three constants in the surrounding scope — `Red`, `Green`, `Blue` — of an implicit integer type. The names leaked, and the values implicitly converted to and from `int`.

C++11 introduced `enum class`, which fixes both problems:

```
enum class Color { Red, Green, Blue };

Color c = Color::Red;          // scoped: must write Color::
int   n = c;                   // ERROR: no implicit conversion
int   n = static_cast<int>(c); // ok
```

The enumerators are scoped inside the `Color` type. They do not implicitly convert to or from integers. Two unrelated `enum class`es are unrelated types.

You can fix the underlying integer width:

```
enum class Kind : std::uint8_t { A, B, C };  // each value fits in a byte
```

Maya uses `enum class` everywhere a value is one of a small set:

```
enum class HttpErrorKind { Tls, Timeout, Refused, Status, Cancel };
enum class BorderStyle   { None, Square, Round, Double, Thick };
enum class FlexDirection { Row, Column };
enum class KeyKind       { Char, Enter, Tab, Backspace, Escape,
                           Up, Down, Left, Right, ... };
```

When the value is one of more than a handful of cases, or when each case carries data, prefer `std::variant` (Section 3.28). `enum class` is for plain tagged states.

3.11 Type aliases: using

Long type names are tedious to repeat. The old C way was `typedef`:

```
typedef std::vector<std::pair<std::string, int>> ScoreList;
```

C++11 introduced a cleaner form, `using`:

```
using ScoreList = std::vector<std::pair<std::string, int>>;
```

It reads left-to-right ("`ScoreList` is…") and supports templates, which `typedef` does not:

```
template <typename T>
using Vec2 = std::vector<std::vector<T>>;

Vec2<int> matrix;   // std::vector<std::vector<int>>
```

Maya defines aliases for every public type that has a long template expansion (`using HttpResult = std::expected<Response, HttpError>;` for example). Prefer `using` over `typedef` everywhere; `typedef` is in the language only for C compatibility.

3.12 Attributes and `noexcept`

C++'s **attributes** are double-bracketed annotations the compiler reads to enable warnings, optimisations, or sanity checks. They do not change semantics; ignoring them is always safe.

- `[[nodiscard]]` on a function says "do not silently throw away my return value". The compiler warns if you call the function and ignore the result.
- `[[noreturn]]` says "this function never returns" (it throws, calls `std::abort`, loops forever). Lets the compiler treat code after the call as unreachable.
- `[[deprecated("reason")]]` marks an API as old; uses of it warn.
- `[[maybe_unused]]` on a variable or parameter silences the "unused" warning when the variable is intentionally unused.
- `[[likely]]` / `[[unlikely]]` on a branch hints to the compiler which way the branch usually goes.

In maya you will see `[[nodiscard]]` on almost every function returning a value. Ignoring the result of `ansi::fg(c)` is a bug; the attribute makes the bug a warning.

`noexcept` is not an attribute — it is part of the function signature, before the body:

```
int  add(int a, int b) noexcept { return a + b; }
void f() noexcept { /* must not throw */ }
```

It promises the function will not throw an exception. The compiler can generate slightly faster code for `noexcept` functions (no unwinding tables needed at the call site). `std::move`, `std::swap`, and many standard-library operations behave better on types whose key operations are `noexcept`; in particular, `std::vector` reallocations move (not copy) elements only when the move constructor is `noexcept`.

Rule: mark a function `noexcept` when you are sure it will not throw. Mark destructors `noexcept` essentially always (the language already implicitly does this).

3.13 Classes, constructors, destructors

A **class** (introduced with `class` or `struct` — the only difference is the default access level) is a user-defined type that bundles data and behaviour.

```
struct Point {
    double x;
    double y;

    double length() const {
        return std::sqrt(x*x + y*y);
    }
};

```

```
Point p{3.0, 4.0};
std::println("{}", p.length());   // 5
```

`struct` members are public by default; `class` members are private by default. Most maya types are `struct`.

3.13.1 Constructors

A **constructor** is a special function that runs when an object is created. It has the same name as the class, no return type, and may take parameters.

```
struct File {
    FILE* fp;

    explicit File(const char* path) {
        fp = std::fopen(path, "r");
    }
};

File log{"app.log"};   // calls the constructor with "app.log"
```

`explicit` prevents implicit conversion: without it, you could write `File f = "app.log";` and C++ would silently call the constructor. Mark single-argument constructors `explicit` unless you genuinely want the implicit form.

3.13.2 Member initialiser lists

The preferred way to set members from a constructor is the **initialiser list**, which appears between a colon and the body:

```
struct File {
    FILE* fp;

    explicit File(const char* path) : fp(std::fopen(path, "r")) {}
};
```

Members are initialised in the order they are declared in the class, *not* in the order they appear in the initialiser list. Modern compilers warn if you write them in the wrong order; pay attention to the warning.

3.13.3 Destructors

A **destructor** is the constructor's counterpart: it runs when the object is about to be destroyed. Its name is the class name prefixed with `~`.

```
struct File {
    FILE* fp;

    explicit File(const char* path) : fp(std::fopen(path, "r")) {}
    ~File() { if (fp) std::fclose(fp); }
};

void load() {
    File log{"app.log"};   // fopen runs here
    // ... use log.fp ...
}                          // ~File runs here, fclose runs automatically
```

The destructor runs at the end of the scope where the object was created, even if the function returns early, even if an exception is thrown. This is the foundation of the next idea.

3.14 RAII — the central pattern

Resource Acquisition Is Initialization, or **RAII**, is the single most important idiom in C++. It is the principle that every resource your program holds — memory, file descriptor, terminal mode, mutex lock, network connection — should be tied to the lifetime of an object. The constructor acquires; the destructor releases. You cannot forget to release because the language runs the destructor for you.

The `File` class above is RAII: constructing a `File` opens the file; the destructor closes it. There is no `close()` method to forget to call.

3.14.1 A `TerminalGuard` for raw mode

Let us write the RAII type that fixes the bug from Chapter 2, where forgetting to restore cooked mode left the user's shell broken.

```
class TerminalGuard {
    termios cooked_;
public:
    TerminalGuard() {
        tcgetattr(STDIN_FILENO, &cooked_);
        termios raw = cooked_;
        raw.c_lflag &= ~(ICANON | ECHO | ISIG | IEXTEN);
        raw.c_iflag &= ~(IXON | ICRNL | INPCK | ISTRIP);
        raw.c_oflag &= ~(OPOST);
        raw.c_cc[VMIN]  = 0;
        raw.c_cc[VTIME] = 0;
        tcsetattr(STDIN_FILENO, TCSAFLUSH, &raw);
    }

    ~TerminalGuard() {
        tcsetattr(STDIN_FILENO, TCSAFLUSH, &cooked_);
    }

    TerminalGuard(const TerminalGuard&)            = delete;
    TerminalGuard& operator=(const TerminalGuard&) = delete;
};

int main() {
    TerminalGuard guard;   // raw mode on
    // ... run a TUI ...
}                          // raw mode off, automatically
```

The two `= delete` lines forbid copying. Copying would let two `TerminalGuard` objects fight over the terminal state. We will see `= delete` again shortly.

maya's actual terminal driver is more elaborate (it also handles the alt-screen, mouse mode, KKP push, signal restoration on crash), but the shape is exactly this: a class whose lifetime is the lifetime of the terminal being in TUI mode.

Idea

RAII: every resource is owned by an object. Acquire in the constructor, release in the destructor. Forgetting becomes a compile-time impossibility, not a runtime risk.

3.14.2 The rule of zero, three, five

If a class manages a resource, you usually need to define five special member functions: destructor, copy constructor, copy assignment, move constructor, move assignment. This is the **rule of five**.

If you do not manage any resource yourself — you only hold other RAII types as members (`std::string`, `std::vector`, `std::unique_ptr`) — you should define *none* of them. The compiler generates correct versions automatically. This is the **rule of zero**, and it is the rule you should follow first.

The **rule of three** (C++98) is the pre-move-semantics version: define destructor, copy constructor, copy assignment together.

Most maya classes follow the rule of zero. The few that hold raw resources (the terminal guard, signal handlers, file descriptor wrappers) follow the rule of five.

3.15 `static` at three different scopes

The keyword `static` means three different things in C++ depending on *where* you write it. This is a long-standing source of confusion. Untangle them once:

1. `static` on a local variable. Makes the variable persist between calls of the function:

```
int next_id() {
    static int counter = 0;   // initialised once, on first call
    return ++counter;
}

next_id();   // 1
next_id();   // 2
next_id();   // 3
```

The `counter` variable lives in the program's data segment — not on the stack. It is initialised the first time control flows through the declaration, in a thread-safe way (C++11 guarantees this), and survives until program exit.

Useful for cheap, deterministic singletons. Maya uses it to lazily build lookup tables on first access.

2. `static` on a class member. Makes the member shared across all instances of the class, not owned by any individual instance:

```
struct Counter {
    static int total;        // declaration; shared by all Counters
    int      individual;
};
int Counter::total = 0;       // definition, exactly once, outside the class

```

```
Counter a, b;
Counter::total = 5;
// a.total and b.total are both 5; they refer to the same storage
```

For `static constexpr` members of fundamental type, you can initialise inline and skip the out-of-class definition. C++17's `inline` on a `static` member makes it definable in the header without ODR violations.

3. `static` on a free function or namespace-scope variable. Gives the function or variable **internal linkage** — it is visible only inside the translation unit that defines it. Other files cannot use it even if they declare it.

```
// in foo.cpp
static int helper(int x) { return x + 1; }   // local to this .cpp
int visible_helper(int x) { return x + 1; }  // visible everywhere
```

This is the C++'s pre-namespace way of saying "do not leak this name". Modern style prefers an anonymous namespace, which has the same effect and reads more clearly:

```
namespace { // anonymous namespace --- everything here has internal linkage
    int helper(int x) { return x + 1; }
}
```

Three different things, one keyword. When you read maya source, check the scope first: inside a function (case 1), inside a class (case 2), or at file scope (case 3).

3.16 Inheritance and `virtual` (and why maya avoids them)

A class can **inherit** from another, picking up its members:

```
struct Shape {
    virtual double area() const = 0;   // pure virtual: subclasses must override
    virtual ~Shape() = default;        // virtual destructor: required for safe deletion
};

struct Circle : Shape {
    double r;
    double area() const override { return 3.14159 * r * r; }
};

struct Square : Shape {
    double s;
    double area() const override { return s * s; }
};

Shape* p = new Circle{2.0};
std::println("{}", p→area());   // 12.56 --- runtime dispatch to Circle::area
delete p;
```

Key words:

- `struct Circle : Shape` — `Circle` inherits publicly from `Shape`; `Circle*` converts to `Shape*` implicitly.
- `virtual` on a method — the method can be overridden by subclasses and dispatched at runtime through a vtable.

- `= 0` — pure virtual: this class is *abstract* (cannot be instantiated) and subclasses must implement.
- `override` — a tag on the subclass method telling the compiler "I am overriding a base method". If the signature drifts and you no longer match, you get an error. Always write it.
- `virtual ~Shape()` — destructor must be virtual if you ever `delete` a derived object through a base pointer.

This is the classic object-oriented model: polymorphism through inheritance, runtime dispatch through vtables.

3.16.1 Why maya barely uses it

Inheritance has costs:

- Every `virtual` call is an indirect jump through a function pointer in the vtable. The branch predictor cannot always foresee where it goes, and a missed prediction is many cycles wasted. The diff loop runs hundreds of thousands of times per frame; an indirect call per cell would be ruinous.
- Polymorphism through pointers requires heap allocation in most uses (`new Circle...`, `std::unique_ptr<Shape>`). Allocation is a real cost.
- The set of subclasses is open: code adding a new `Shape` can appear in any translation unit, and no compiler check forces every consumer of `Shape` to handle it. Closed sum types do not have this problem.

Where a polymorphism model is needed, maya prefers `std::variant` + `std::visit`:

```
using Shape = std::variant<Circle, Square, Triangle>;

double area(const Shape& s) {
    return std::visit(overload{
        [](const Circle& c)   { return 3.14159 * c.r * c.r; },
        [](const Square& s)   { return s.s * s.s; },
        [](const Triangle& t) { return 0.5 * t.base * t.height; },
    }, s);
}
```

No vtable, no heap allocation, *closed* (adding a new alternative makes every visitor a compile error until updated). This is the pattern for `Element`, `Event`, `Cmd<Msg>`, `Sub<Msg>`, and every other "one of these things" in maya.

Inheritance survives in maya only for two narrow uses:

- Empty base classes used as compile-time tags (CRTP, mixin patterns). No runtime cost.
- Standard-library types we cannot avoid (`std::exception` is a base class).

Idea

Closed sum types over runtime polymorphism. The compiler can check exhaustiveness; the CPU does not pay for a vtable.

3.17 Operator overloading

C++ lets you teach operators new meanings for your types. This is why you can write `a + b` where `a`, `b` are `std::string`s; `std::string` has an `operator+` that produces concatenation. maya's DSL leans on this hard — `|` is taught a brand-new meaning.

An overload of a binary operator can be a method or a free function:

```
struct Vec2 {
    double x, y;

    // as a method (left-hand side is *this):
    Vec2 operator+(const Vec2& other) const {
        return {x + other.x, y + other.y};
    }
};

// as a free function:
Vec2 operator-(const Vec2& a, const Vec2& b) {
    return {a.x - b.x, a.y - b.y};
}

Vec2 a{1, 2}, b{3, 4};
Vec2 c = a + b;          // calls Vec2::operator+
Vec2 d = a - b;          // calls free operator-
```

Free-function form is preferred when the left operand could be of a type you do not own (e.g. `int + Vec2`).

3.17.1 Operators you can overload

Most of them: `+ - * / % & | ^ ~ ! < > <= >= == != && || << >> += -= ... [] () -> ->* , ++ --` (prefix and postfix), and conversion operators. You cannot invent new operators or change precedence or arity.

Four are particularly common:

- `operator()` — the call operator. Makes the type callable: `f(x, y)` where `f` is an object. Lambdas use it; `std::function` uses it.
- `operator[]` — subscript. `v[i]` on containers.
- `operator T()` — a *conversion operator*. Lets your type implicitly convert to `T`. This is how `StyTag<...>` becomes `Style` in the DSL.
- `operator|` — bitwise-or for built-in integers, but maya overloads it to mean "compose this style / modifier onto the thing on the left".

Example of `operator|` for a tiny style system:

```
struct Style { bool bold = false; bool italic = false; };
struct BoldTag {};
struct ItalicTag {};

constexpr Style operator|(Style s, BoldTag)   { s.bold = true;   return s; }
constexpr Style operator|(Style s, ItalicTag) { s.italic = true; return s; }

constexpr Style red = Style{} | BoldTag{} | ItalicTag{};
```

Left-to-right associativity of `|` makes the chain read naturally: `Style{}` is the start; each tag layers on a flag. maya's DSL is the same pattern, scaled up.

3.17.2 Conversion operators

A `operator T() const` lets your type act like a `T`:

```
struct Fahrenheit {
    double value;
    operator double() const { return value; }
};

Fahrenheit f{72.0};
double d = f;            // implicit conversion: d == 72.0
std::println("{}", f);   // also works
```

Mark conversions `explicit` unless you really want them implicit. Implicit conversions are silent and surprising; explicit ones force the caller to write `static_cast<double>(f)`.

The key exception: in maya's DSL, the `StyTag` types implicitly convert to `Style`, and that implicit conversion is the whole reason `node | Bold` works without writing `static_cast`.

3.18 Smart pointers

Raw pointers (`T*`) have ambiguous semantics: do they own the object? Are they allowed to be null? The standard library provides three **smart pointer** types that make the answer explicit.

3.18.1 std::unique_ptr<T>

A `std::unique_ptr<T>` is a single-owner pointer. It owns the `T`; when it goes out of scope, it deletes the `T`. It cannot be copied (otherwise two owners), only moved.

```
#include <memory>
std::unique_ptr<Buffer> p = std::make_unique<Buffer>(1024);

p->write("hi", 2);          // -> dereferences
(*p).flush();               // * dereferences

std::unique_ptr<Buffer> q = std::move(p);   // q owns it now
// p is now empty (nullptr)
// at end of scope, q's destructor deletes the Buffer
```

Use this as your default. If a function returns a `unique_ptr<T>`, the caller becomes the owner. If a function takes a `unique_ptr<T>` by value, the caller has handed ownership over.

3.18.2 std::shared_ptr<T>

A `std::shared_ptr<T>` is a reference-counted shared owner. Multiple `shared_ptr`s can point at the same object; the object is deleted when the last one goes out of scope.

```
std::shared_ptr<Cache> c = std::make_shared<Cache>();
std::shared_ptr<Cache> d = c;   // both own it
std::println("{}", c.use_count());   // 2
```

Use this only when you genuinely need shared ownership across threads or owners. The reference count is atomic and a real cost; for a single-owner relationship use `unique_ptr`.

3.18.3 `std::weak_ptr<T>`

A non-owning observer of a `shared_ptr`. Used to break reference cycles. You rarely write one in maya.

3.18.4 When to use raw `T*`

Not for ownership. Only as a non-owning *view* into something you know outlives the pointer: a parent pointer in a tree, an element of a stable array, a function parameter that may be null. For pure "read this thing" parameters, prefer `const T&`; for optional inputs, prefer `std::optional<T>`.

maya uses `unique_ptr` for heap-allocated children of variants where appropriate (`Element` children are stored in a `std::vector<Element>` rather than `vector<unique_ptr>` because `Element` is itself a variant with value semantics). The few raw pointers are non-owning parent links.

3.19 Iterators

The standard library's containers (`vector`, `map`, `set`, `string`, ...) are walked with **iterators**. An iterator is an object that acts like a pointer into a container: dereference to get the element, increment to advance to the next.

```
std::vector<int> v = {10, 20, 30, 40};

auto it  = v.begin();    // iterator to the first element
auto end = v.end();      // iterator one past the last

while (it != end) {
    std::println("{}", *it);   // dereference
    ++it;                      // advance
}
```

Three things to know:

- `begin()` returns an iterator to the first element.
- `end()` returns a sentinel *one past* the last element — you never dereference it; you compare against it.
- Iterators support `*`, `++`, and (where the container allows it) `-`, `+ n`, `- n`.

The pattern `for (auto it = c.begin(); it != c.end(); ++it)` is the classic STL walk. The range-`for` loop `for (auto& x : c)` expands to exactly this; modern code almost always prefers the range form for clarity.

3.19.1 Why iterators show up in maya errors

When something goes wrong with a container — you erase while iterating, the container reallocates and invalidates the iterator, you pass an iterator past the end — the error usually mentions "iterator". Common cases:

- `map.find(key)` returns an iterator. Compare to `map.end()` to check "not found". Dereferencing the end iterator is undefined behaviour.
- Pushing into a `std::vector` *invalidates* every existing iterator, pointer, and reference into it — because the buffer may have moved. Save indices, not iterators, across pushes.
- Erasing from a `std::vector` invalidates iterators from the erased element onward. Use the return value of `erase` (an iterator to the next valid element) to continue.

Iterators are also what makes algorithms generic: `std::sort(v.begin(), v.end())` works on any container that provides random-access iterators. maya uses standard algorithms throughout the layout and text engines.

3.20 `auto` and type deduction

Writing out the type of every variable is fine for `int` and `double`; it is exhausting for things like `std::vector<std::pair<std::string, Element>>::const_iterator`. C++11 introduced `auto`: "compiler, figure out the type from the initialiser".

```
auto x = 42;                  // int
auto pi = 3.14;               // double
auto name = std::string{"hi"};// std::string
auto v = std::vector<int>{1, 2, 3};
for (auto& item : v) ++item;  // item is int&
```

Three rules of thumb:

- Plain `auto` drops references and `const`. If you write `auto x = some_const_ref();` you get a fresh copy.
- `auto&` keeps references and `const`. Use this in range-`for` loops to avoid copying each element.
- `const auto&` is the most common form for read-only iteration: `for (const auto& cell : row) { ... }`.

In maya, `auto` is the default everywhere a type would be verbose. Spell types out only when the spelling is short or when it clarifies intent.

3.21 Move semantics

We saw that assignment copies. For a `std::vector<int>` with a million elements, copying is a million-element allocation plus a million-element copy. If you do not need the original after, the copy is pure waste.

Move semantics is the C++11 feature that lets you say "transfer the contents, do not copy". A move is usually a pointer swap: the new vector inherits the buffer; the old vector is left empty.

```cpp
std::vector<int> a = {1, 2, 3};
std::vector<int> b = std::move(a);   // O(1) --- b owns the buffer
// a is in a "valid but unspecified" state; do not read it.
```

Key points:

- `std::move(x)` does not actually move anything; it is a *cast* that says "treat `x` as a thing whose contents may be stolen". The move actually happens in the constructor or assignment operator that receives the rvalue.
- After moving from `x`, you must not assume `x` holds anything specific. You can still destroy it (safely) and you can still assign a fresh value to it.
- Move is an *optimisation*, not a semantic change. A correctly written program works whether `std::move` is present or not; `std::move` only changes how fast it runs.

The **rvalue reference** type `T&&` is how a function opts in to receiving a move:

```cpp
void take(std::string s);              // by value: caller can move into it
void take_ref(const std::string& s);   // by const ref: read-only
void take_rref(std::string&& s);       // rvalue ref: must be moved in

std::string name = "alice";
take(name);              // copies name into the parameter
take(std::move(name));   // moves name into the parameter
// after this, name is moved-from
```

In maya you see `std::move` most often in the runtime: `Model` flows through `update(Model, Msg)` without being copied, because `update` takes its `Model` by value and the runtime moves the current model into the call.

Pitfall

Do not `std::move` a local on the return statement (`return std::move(x);`). The compiler already moves return values automatically; writing `std::move` explicitly disables a separate optimisation called RVO (Return Value Optimisation) that is usually *better* than move.

3.22 Exceptions, and why maya prefers `expected`

An **exception** is C++'s built-in mechanism for signalling errors out of band: you `throw` a value, and the runtime unwinds the stack (destroying every local along the way) until it finds a matching `catch` handler.

```cpp
#include <stdexcept>

int sqrt_or_throw(int n) {
    if (n < 0) throw std::runtime_error("negative");
    return /* ... */;
}

try {
    int r = sqrt_or_throw(x);
    use(r);
} catch (const std::runtime_error& e) {
    std::println("failed: {}", e.what());
}
```

The machinery is real:

- `throw e` constructs and throws an exception value. Anything copyable or movable can be thrown; convention is to throw a class deriving from `std::exception`.
- Stack unwinding runs the destructor of every local between the `throw` and the `catch`. This is why RAII matters: your resources get released even on errors.
- `catch (const T&)` matches a thrown `T` or any derived type. The last `catch` can be `catch (...)` to catch everything (rarely useful except at the top of `main`).

Exception handling is cheap on the success path (modern implementations use zero-cost-when-not-thrown tables) and expensive when an exception is actually thrown.

3.22.1 Why maya avoids them for ordinary errors

Three reasons:

1. **Invisibility.** Looking at the signature `int parse(std::string_view s)`, you cannot tell whether it might throw. The caller has to know the convention or read the body. `std::expected<int, ParseError>` writes the failure mode into the signature.
2. **Closed errors.** Catching by base class means the caller cannot meaningfully switch on the failure mode without `dynamic_cast`-ing. `std::expected` with an `enum class Kind` error type gives a finite, switchable list.
3. **Cost on the wrong path.** Throwing is slow. A TUI that throws on every key press it doesn't recognise will feel sluggish.

maya still uses exceptions for truly exceptional things — out-of-memory, bugs in the runtime, panic conditions at top level. For ordinary, expected failures (parse failed, file missing, network timed out), it returns `std::expected`.

Idea

Throw on bugs, return on errors. Exceptions for "the world is broken"; `std::expected` for "this might not work, and that is fine".

3.23 Undefined behaviour: what it is and how to avoid it

A fact you need to know about C++: some operations are **undefined behaviour** (UB). The standard does not say what the program should do when you perform them; the compiler is allowed to do *anything*, including silently produce nonsense. This is jarring if you come from a safer language.

The most common forms of UB:

- Reading an uninitialised variable. `int x; std::println("", x);` is UB.
- Indexing out of bounds. `v[v.size()]` on a non-empty `std::vector` is UB. (`v.at(i)` is *not* UB; it throws.)

- Dereferencing a null or dangling pointer. `int* p = nullptr; *p;` is UB. `auto p = (int*)0xdeadbeef; *p;` is UB.
- Using a moved-from object as if it were valid. After `auto b = std::move(a);`, reading `a` (unless you have re-assigned to it) is implementation-defined or undefined depending on the type.
- Signed integer overflow. `int x = INT_MAX; ++x;` is UB.
- Data races: two threads concurrently access the same non-atomic variable, at least one of them writing.
- Returning a reference or pointer to a local variable, then using it after the function returns.

The surprise is that the compiler is *allowed* to assume UB never happens, and to optimise on that assumption. A program with UB can run fine in debug mode, fail in release mode, and crash on a different CPU. There is no easy "just inspect the value" fix; the fix is to not commit the UB in the first place.

3.23.1 How to avoid it

Three habits cover 90% of cases:

1. **Initialise everything.** Default values are free with brace init: `int x{};`, `Point p{};`. Never write `int x;` and read `x` before assignment.
2. **Use containers and views, not raw arrays.** `std::vector::operator[]` is technically UB on out-of-bounds *but* debug builds with sanitisers catch it; raw C arrays have neither bounds nor safety nets.
3. **Compile with sanitisers during development.** `-fsanitize=address,undefined` catches most of the above list at runtime with a clean diagnostic. Use this constantly while learning.

maya's test suite runs under both sanitisers in CI. Code reviews treat UB-trigger patterns as bugs even if they happen not to crash in the local build.

Pitfall

A program with UB is not a slow or buggy program; it is a program without a defined meaning. Treat UB as a hard error, not a warning.

3.24 Templates

A **template** is a recipe the compiler runs at compile time. You write the recipe once; the compiler bakes a fresh function or class for each set of arguments you give it.

3.24.1 Function templates

```
template <typename T>
T add(T a, T b) { return a + b; }

int    x = add(1, 2);          // T = int
double y = add(1.5, 2.5);      // T = double
std::string s = add<std::string>("foo", "bar");  // T = std::string
```

`typename T` introduces a type parameter named `T`. The function body uses `T` as if it were a type. The compiler *instantiates* the template once per distinct `T`: there are separate `add<int>`, `add<double>`, and `add<std::string>` functions in the final binary.

The template only compiles correctly if `T` actually supports `operator+`. Try `add(File, File)` and you will get a long error. Concepts (Section 3.27) fix this.

3.24.2 Class templates

```
template <typename T>
struct Box {
    T value;
    T& get() { return value; }
};

Box<int>    a{42};
Box<double> b{3.14};
Box<std::string> c{"hi"};
```

Each instantiation is a different type. `Box<int>` and `Box<double>` share no relationship — you cannot assign one to the other.

`std::vector<T>`, `std::optional<T>`, `std::unique_ptr<T>`, `std::variant<T, U, ...>` are all class templates.

3.24.3 Multiple template parameters

```
template <typename K, typename V>
struct Pair {
    K first;
    V second;
};

Pair<std::string, int> ages[] = {{"alice", 30}, {"bob", 25}};
```

3.24.4 Non-type template parameters (NTTPs)

The lesser-known but crucial flavour. A template parameter does not have to be a type — it can be a *value*, computed at compile time:

```
template <int N>
struct Array {
    int data[N];
    int size() const { return N; }
};

Array<8>  a;   // data is int[8]
Array<16> b;   // data is int[16]
// a and b are different types
```

`std::array<T, N>` works this way. The size is part of the type, so `std::array<int, 4>` and `std::array<int, 8>` are distinct types and the size is known at compile time.

The types of NTTPs were limited for a long time — integers, pointers, enums. Since C++20, NTTPs can be **structural class types**: any literal type with all-public members and no user-defined comparison oddities. This is the trick that powers maya's DSL.

3.24.5 NTTPs of structural type — the climax

We want to write `t<"Hello">` and have the string become part of the type. The straight path is to define a small "string of fixed length" type and use it as an NTTP:

```
template <std::size_t N>
struct Str {
    char data[N]{};

    consteval Str(const char (&s)[N]) {
        for (std::size_t i = 0; i < N; ++i) data[i] = s[i];
    }
};
```

Line by line:

- `template <std::size_t N>`: a class template parameterised by an integer.
- `char data[N]{}`: a fixed-size array of `N` characters, zero-initialised.
- `consteval Str(const char (&s)[N])`: a constructor that *must* run at compile time (`consteval`) and takes a *reference to a literal array of `N` chars*. The array-reference syntax `(&s)[N]` captures the length of the literal — the compiler picks `N` for you.
- The loop copies the string into `data`.

Now we can use `Str` as an NTTP:

```
template <Str S>
struct Text {
    static constexpr std::string_view text() {
        return {S.data};
    }
};

Text<"Hello"> a;     // S = Str{"Hello\\0"}, baked into the type
Text<"World"> b;     // distinct type
```

`Text<"Hello">` and `Text<"World">` are different types. The *bytes* of the string are part of the type identity. This is exactly how maya's `t<"...">` works, and it is the foundation of the whole compile-time DSL we tour in Chapter 5.

Idea

NTTPs let you encode *values* into the type system. maya encodes text, colours, paddings, borders, and style flags this way. Once a value is in a type, the compiler can check rules between values at compile time.

3.24.6 Templates and the <> syntax

When you see `Foo<X, Y>` in a C++ expression, the angle brackets are template-argument syntax, not less-than / greater-than. The ambiguity is one of the language's papercuts; the compiler usually figures it out from context, and where it cannot, you write `template` explicitly: `x.template foo<int>()`.

You will see `t<"Hi">`, `Fg<255, 100, 100>`, `pad<1>`, `border_<Round>` throughout `maya`. They are all template instantiations.

3.25 Parameter packs and fold expressions

A **variadic template** accepts any number of template arguments. The declaration looks like a pack of dots:

```
template <typename... Ts>
struct Tuple { /* ... */ };

Tuple<int, std::string, double> t;
```

`typename... Ts` introduces a **parameter pack** named `Ts`. Inside the template, `Ts...` expands the pack — likewise for function-parameter packs:

```
template <typename... Args>
void print_all(const Args&... args) {
    // args is a function-parameter pack
    // expanding (args)... yields a comma-separated list of arguments
}

print_all(1, "hi", 3.14);
```

The expansion syntax `pattern...` repeats the `pattern` once per element of the pack, comma-separated.

3.25.1 Fold expressions

Writing recursion-style overloads to walk a pack used to be the only way to consume it. C++17 added **fold expressions**: ask the compiler to fold the pack with a binary operator.

```
template <typename... Args>
auto sum(Args... args) {
    return (args + ...);     // unary right fold over +
}

sum(1, 2, 3, 4);              // 10 = ((1 + (2 + (3 + 4))))
```

Four forms exist (`(... op pack)`, `(pack op ...)`, `(init op ... op pack)`, `(pack op ... op init)`), but you usually just need one of:

```
(args + ...)              // sum
(args, ...)               // evaluate left-to-right, throw away
(os << ... << args)       // chained <<: equivalent to os << a << b << c
((std::println("{}", args)), ...);  // print each, in order
```

The last one — a fold over comma — is the idiomatic "do this once per pack element".

3.25.2 The `overload` helper, revisited

With packs in hand, the `overload` struct from earlier makes full sense:

```
template <typename... Fs>
struct overload : Fs... {
    using Fs::operator()...;
};
template <typename... Fs> overload(Fs...) → overload<Fs...>;
```

- `template <typename... Fs>`: `Fs` is a pack of types — the lambda types you pass in.
- `struct overload : Fs...`: inherit from every type in the pack. The pack expands into a comma-separated base list.
- `using Fs::operator()...`: bring each base's `operator()` into the combined class. This is a **using-declaration pack expansion**.
- The deduction guide `overload(Fs...) -> overload<Fs...>` lets you write `overload{lambda1, lambda2}` without spelling out the lambda types (which are unnameable anyway).

The result: `overload{f, g, h}` is a single object whose `operator()` resolves to whichever of `f`, `g`, or `h` matches the call. Combined with `std::visit`, it gives you pattern matching on variants.

3.26 constexpr, consteval, constinit

A **constant expression** is one the compiler can evaluate at translation time, before the program runs. C++ has three keywords governing this:

- `constexpr` on a function or variable: *may* be constant-evaluated; otherwise behaves like a normal function or variable.
- `consteval` on a function: *must* be constant-evaluated. Calling it at runtime is a compile error.
- `constinit` on a variable: must be initialised by a constant expression, but the variable itself is otherwise mutable. Used to forbid the "static initialisation order fiasco".

Example of `constexpr`:

```
constexpr int square(int n) { return n * n; }

constexpr int sixteen = square(4);   // evaluated at compile time
int runtime_n = 7;
int forty_nine = square(runtime_n);  // also legal, evaluated at runtime
```

Example of `consteval`:

```
consteval int square_only_ct(int n) { return n * n; }

constexpr int a = square_only_ct(4);  // ok
int runtime_n = 7;
// int b = square_only_ct(runtime_n); // error: must be constant-evaluated
```

maya's DSL is heavy on `constexpr` and `consteval`. The entire chain `t<"Hi"> | Bold | Fg<255, 80, 80>` produces a `constexpr` value — every `operator|` along the way is `constexpr`. Conversion to the runtime `Element` (which may allocate) happens only when you call `.build()`.

3.27 Concepts and requires

A template constrains its arguments implicitly: writing `a + b` in a template body means `T` must support `operator+`, but the constraint is invisible until you try to instantiate the template with a wrong type and get a wall of compiler errors.

C++20 **concepts** make those constraints explicit and named:

```
template <typename T>
concept Addable = requires(T a, T b) {
    { a + b } → std::convertible_to<T>;
};

template <Addable T>
T add(T a, T b) { return a + b; }

add(1, 2);                // ok, int is Addable
// add(File{...}, File{...});  // clean error: File does not satisfy Addable
```

The `requires` expression lists statements that must be valid for `T`. `{ a + b } -> std::convertible_to<T>` means "`a + b` must compile, and its result must convert to `T`". If a caller passes a `T` that does not satisfy `Addable`, the compiler points at the constraint, not at something seventeen layers deep inside the template body.

3.27.1 Maya's concepts

Maya defines several concepts in *core/concepts.hpp*. Three of the important ones:

```
// (paraphrased from maya source)

template <typename T>
concept Node = requires(const T& t) {
    { t.build() } → std::convertible_to<Element>;
};

template <typename P>
concept Program = requires(typename P::Model m, typename P::Msg msg) {
    typename P::Model;
    typename P::Msg;
    { P::init() };
    { P::update(m, msg) };
    { P::view(m) }       → std::convertible_to<Element>;
    { P::subscribe(m) };
};
```

`Node` says "has a `.build()` that returns something convertible to `Element`". Every DSL piece satisfies it. `Program` says "has the four Elm functions with the right signatures". `run<P>()` requires `Program<P>`; if your program is missing a function, the compiler tells you which one.

Idea

Concepts are documentation that the compiler checks. They turn "this template requires the type to do X" from a comment into a compile-time predicate.

3.28 std::variant and std::visit

A `std::variant<A, B, C>` is a value that holds *exactly one* of `A`, `B`, or `C`. It is the C++ closed sum type — the same construct functional languages call an algebraic data type.

```
#include <variant>
std::variant<int, std::string, double> v;
v = 42;          // holds an int
v = "hello";     // now holds a string
v = 3.14;        // now holds a double
```

You inspect a variant with `std::visit`, which dispatches to the overload matching the current alternative. The idiomatic helper is `overload`, a small struct that combines multiple lambdas into a single callable:

```
template <typename... Fs> struct overload : Fs... { using Fs::operator()...; };
template <typename... Fs> overload(Fs...) → overload<Fs...>;

std::visit(overload{
    [](int n)               { std::println("int  : {}", n); },
    [](const std::string& s){ std::println("str  : {}", s); },
    [](double d)            { std::println("real : {}", d); },
}, v);
```

The `overload` template uses two features:

- `Fs...` is a **parameter pack** — a variadic template, accepting any number of types.
- Inheriting from each one and using `using Fs::operator()...` brings every lambda's `operator()` into the combined class. `overload{f, g, h}` is then a single object with three call-operator overloads.

This is enough C++ sorcery to write off as a one-liner you copy. It is in *maya/core/overload.hpp*.

3.28.1 Maya's variants

Maya uses `std::variant` relentlessly:

- `Element` is `std::variant<BoxElement, TextElement, ElementList>`.
- `Event` is `std::variant<Key, Mouse, Resize, Paste, ...>`.
- `Cmd<Msg>` is a variant over every effect the runtime knows: `None`, `Quit`, `Batch`, `After`, `SetTitle`, `WriteClipboard`, `Task`, ...
- `Sub<Msg>` is a variant over event sources.

Whenever you see a `std::visit` in `maya`, it is the dispatch hub for one of these types.

Pitfall

If you add a new alternative to a variant and forget to handle it in a visitor, the visitor stops compiling. This is a feature: it makes refactors safe. Embrace it; do not paper over it with a generic fallback.

3.29 Modern syntax sugar you will see everywhere

A handful of small C++17 / 20 / 23 features change what idiomatic code *looks* like. They are not deep, but a beginner who has never seen them will stumble. We collect them here.

3.29.1 Structured bindings

Given a struct, tuple, or pair, you can destructure it into named variables in one line:

```
std::pair<int, std::string> p{42, "hi"};
auto [n, s] = p;          // n = 42, s = "hi"

std::map<int, std::string> m;
for (const auto& [key, value] : m) {
    std::println("{} → {}", key, value);
}

struct Point { double x, y; };
Point pt{3.0, 4.0};
auto [x, y] = pt;
```

The names are introduced as if by separate declarations. `auto&` and `const auto&` work too: `auto& [k, v] : m` gives references, no copies. Maya uses structured bindings in the runtime everywhere a function returns a `std::pair` — including `P::init()` and `P::update(m, msg)`.

3.29.2 Designated initialisers

For aggregates (structs with all-public fields), you can name fields in the initialiser:

```
struct Config {
    int  width  = 80;
    int  height = 24;
    bool mouse  = true;
};

Config a{};                              // all defaults
Config b{.width = 100};                  // override one
Config c{.width = 100, .height = 30};    // override two
```

The fields must appear in declaration order; you can skip fields (they get their defaults). This produces self-documenting call sites:

```
run<MyProgram>({.title = "counter", .alt_screen = true, .mouse = false});
```

Maya's `RunConfig` is exactly this shape; every example program passes a designated-initialiser literal.

3.29.3 Class template argument deduction (CTAD)

You normally write `std::vector<int> v{1, 2, 3}`. C++17 lets you drop the `<int>` when the arguments make it obvious:

```
std::vector v{1, 2, 3};                  // deduces vector<int>
std::pair  p{42, std::string{"hi"}};     // deduces pair<int, string>
std::array a{1, 2, 3, 4};                // deduces array<int, 4>
```

When you write your own class template, you can give it a **deduction guide** (we saw one for `overload`). For most user-defined classes the compiler deduces fine without one.

3.29.4 `auto` return types

A function's return type can be deduced from its return statements:

```
auto add(int a, int b) {
    return a + b;       // return type deduced to int
}

auto first(const std::vector<int>& v) → int {  // trailing return type
    return v.front();
}
```

The trailing-return-type syntax (`auto f(...) -> T`) is useful when the return type depends on the parameters in a way the leading position cannot reach.

For templates and lambdas, `auto` return is the default; the compiler infers from the body. For maya's public API, return types are usually written out for clarity.

3.29.5 `constexpr if`

Inside a template, `if constexpr (cond)` is a branch evaluated at compile time: the false arm is not even instantiated.

```
template <typename T>
void print(const T& x) {
    if constexpr (std::is_integral_v<T>) {
        std::println("int: {}", x);
    } else if constexpr (std::is_floating_point_v<T>) {
        std::println("float: {}", x);
    } else {
        std::println("other: {}", x);
    }
}
```

With plain `if`, both arms would need to compile for every `T` (which would fail when `T` is not, say, formattable). With `if constexpr`, only the live arm is instantiated.

maya's DSL uses `if constexpr` extensively to specialise behaviour by node kind without writing separate overloads.

3.29.6 `nullptr`, not `NULL`

C++ 0 (= `NULL`) is an integer literal in disguise; it does not type-check well. C++11 added `nullptr`, a value of type `std::nullptr_t` that converts to any pointer type but not to `int`. Always write `nullptr`.

3.30 `std::expected<T, E>`

C++23's `std::expected<T, E>` is a sum type for fallible operations: either a success of type `T` or an error of type `E`. It is maya's preferred alternative to throwing exceptions for "failure is expected, just rare" cases.

```cpp
#include <expected>
#include <charconv>

std::expected<int, std::string> parse_int(std::string_view s) {
    int n;
    auto [end, ec] = std::from_chars(s.data(), s.data() + s.size(), n);
    if (ec != std::errc{}) return std::unexpected("not an int");
    return n;
}

if (auto r = parse_int("42")) {
    std::println("value: {}", *r);
} else {
    std::println("error: {}", r.error());
}
```

The API at a glance:

- `(bool)r` — true if success.
- `*r` or `r.value()` — the success value.
- `r.error()` — the error value.
- `std::unexpected(e)` — factory for the error case.

In maya the error type is usually a small struct:

```cpp
enum class HttpErrorKind { Tls, Timeout, Refused, Status, Cancel };
struct HttpError {
    HttpErrorKind kind;
    int           http_status;
    std::string   detail;
};
using HttpResult = std::expected<Response, HttpError>;
```

The caller switches on `kind` without parsing strings — a proper closed sum of failure modes. This is what we call a "refined" error type.

3.31 Lambdas and capture

A **lambda** is an anonymous function object. Under the hood, the compiler synthesises a nameless class with an `operator()` that runs the lambda body.

```cpp
auto add = [](int a, int b) { return a + b; };
int n = add(3, 4);    // 7
```

The square brackets at the front are the **capture list**: they list names from the surrounding scope the lambda should remember.

```cpp
int x = 10;

auto by_value = [x](int n) { return x + n; };   // copy of x
auto by_ref   = [&x](int n){ return x + n; };   // reference to x

x = 99;
by_value(0);   // 10  --- the lambda has the old copy
by_ref(0);     // 99  --- the lambda sees the new x
```

Shorthand captures:

```
[=]        // capture every used name by value
[&]        // capture every used name by reference
[=, &out]  // mostly by value, but out by reference
[&, n]     // mostly by reference, but n by value
```

Default captures (`[=]` and `[&]`) are convenient but dangerous in long-lived lambdas; you can easily capture a reference to a local that has gone out of scope. Prefer explicit capture lists.

3.31.1 Init-captures and moving into a lambda

C++14 added **init-captures**: `[name = expression]` introduces a new name local to the lambda with any initialiser. This is how you capture a move-only type into a lambda:

```
auto buf = std::make_unique<Buffer>();
auto cb  = [buf = std::move(buf)] {
    buf→flush();
};
// outer buf is moved-from; only the lambda owns the buffer.
```

3.31.2 Storing lambdas: `std::function` vs. `std::move_only_function`

A lambda's type is unique to it; you cannot write the type by hand. To store a lambda in a class member or container, you need a type-erased wrapper:

- `std::function<R(Args...)>` (C++11) holds any callable matching the signature, but the callable must be *copyable*. That excludes lambdas capturing move-only state (`std::unique_ptr`, sockets, file handles).
- `std::move_only_function<R(Args...)>` (C++23) requires only *movable*. maya's signal callbacks and `Cmd<Msg>::Task` use this; effect descriptions routinely carry move-only state.

3.32 Standard-library bits you will see

The standard library is large. A non-exhaustive list of names that appear in maya and what they are:

- `std::string_view` — a non-owning view of a contiguous `char` sequence. Two pointers under the hood. Cheap to pass; does not own the bytes; the underlying data must outlive the view.
- `std::span<T>` — the same idea for any contiguous range of `T`. `std::span<const Cell>` is how the renderer receives a row.
- `std::array<T, N>` — a fixed-size array; size is a template argument. Same memory layout as `T[N]` but with proper value semantics and methods.
- `std::vector<T>` — the dynamic-array workhorse. Heap-allocated buffer that grows.
- `std::optional<T>` — either a `T` or nothing. Use when "absent" is a real state but you do not need a richer error.

- `std::format` / `std::print` / `std::println` — C++20/23 formatted output, like Python's f-strings.
- `std::source_location` — a value carrying the file, line, and function name where it was constructed. maya uses it for stable cache identifiers.
- `std::chrono` — duration and time-point types. `std::chrono::milliseconds(50)` reads cleanly.
- `std::atomic<T>` — lock-free operations on a value shared across threads.
- `std::jthread` — a thread that joins on destruction (no leaked threads on early return). C++20.
- `std::ranges` / `std::views` — lazy, composable algorithms over sequences (`xs | views::transform(...) | views::filter(...)`).

Learn the views as you meet them; the ranges library is large and you do not need it all at once.

3.33 A complete worked example: counting words

Let us put the whole chapter together in a small but real program. It reads lines from standard input until EOF, counts how many times each word appears, and prints the top ten words sorted by count. It uses values, references, `string_view`, `string`, maps, sorting, lambdas, structured bindings, range-`for`, brace-init, `auto`, and a tiny RAII guard — the whole starter kit.

```
#include <print>
#include <string>
#include <string_view>
#include <unordered_map>
#include <vector>
#include <algorithm>
#include <iostream>
#include <chrono>

// RAII guard: prints how long its scope took.
struct Timer {
    using clock = std::chrono::steady_clock;
    clock::time_point start{clock::now()};
    ~Timer() {
        auto ms = std::chrono::duration_cast<std::chrono::milliseconds>(
                      clock::now() - start).count();
        std::println(stderr, "[took {} ms]", ms);
    }
};

// Split a line into whitespace-separated word views.
// Returns views into the input line --- the caller must keep it alive.
static std::vector<std::string_view> split(std::string_view line) {
    std::vector<std::string_view> out;
    std::size_t i = 0;
    while (i < line.size()) {
        while (i < line.size() && std::isspace((unsigned char)line[i])) ++i;
        std::size_t start = i;
        while (i < line.size() && !std::isspace((unsigned char)line[i])) ++i;
        if (i > start) out.push_back(line.substr(start, i - start));
    }
    return out;
```

```
}

int main() {
    Timer t{};                      // RAII: time the whole run
    std::unordered_map<std::string, int> counts;

    std::string line;
    while (std::getline(std::cin, line)) {
        // split returns string_views into `line`; we copy each into
        // a std::string key because the map must own its keys.
        for (auto word : split(line)) {
            counts[std::string{word}] += 1;
        }
    }

    // Move pairs into a vector for sorting.
    std::vector<std::pair<std::string, int>> pairs(counts.begin(), counts.end());
    std::sort(pairs.begin(), pairs.end(),
              [](const auto& a, const auto& b) { return a.second > b.second; });

    std::size_t n = std::min<std::size_t>(10, pairs.size());
    for (std::size_t i = 0; i < n; ++i) {
        const auto& [word, count] = pairs[i];
        std::println("{:6}  {}", count, word);
    }
}
```

Walk through:

- `Timer` is a RAII guard. Constructing one records the start time; the destructor (at scope exit) prints the elapsed. `Timer t{};` at the top of `main` times the whole program.
- `split` takes the line by `std::string_view` — no copy — and returns a `std::vector<std::string_view>` of pieces. The views are valid only as long as `line` is.
- The map key is `std::string`; the value is `int`. `counts[key] += 1` either inserts a new entry with value 1 or increments an existing one.
- `std::string{word}` converts a `string_view` to an owned `std::string` — the explicit brace init makes it obvious we are allocating.
- The `auto word :` in the range-`for` loop is actually `std::string_view word :`; the compiler deduces it.
- `std::sort` takes two iterators and a comparator. The comparator is a lambda taking two pairs and returning `true` if the first should come before the second. We sort descending by count.
- `std::min<std::size_t>(10, pairs.size())` clamps to ten while keeping types matched.
- `const auto& [word, count] = pairs[i]` is a structured binding into the pair — no copy of the strings.
- `std::println("{:6} {}", count, word)` formats the count in a six-wide column then the word.
- When `main` returns, `t`'s destructor runs and prints the elapsed time.

Build and run:

```
$ g++-15 -std=c++26 -O2 -Wall wordcount.cpp -o wordcount
$ < some-book.txt ./wordcount
  1234  the
   987  of
   654  and
   ...
[took 42 ms]
```

If you can read every line of that program with confidence, you can read maya. The next two sections cement the connection.

3.34 Reading real maya code

With the toolkit in hand, let us read a fragment of actual maya. This is from the style header; it declares the tag types behind `Bold`, `Fg<R, G, B>`, and friends.

```
template <CTStyle V>
struct StyTag {
    constexpr operator Style() const noexcept { return V.runtime(); }
};

inline constexpr StyTag<CTStyle::bold()>   Bold{};
inline constexpr StyTag<CTStyle::italic()> Italic{};

template <std::uint8_t R, std::uint8_t G, std::uint8_t B>
inline constexpr StyTag<CTStyle::fg(R, G, B)> Fg{};
```

Line by line:

- `template <CTStyle V>`: `StyTag` is a class template parameterised by an NTTP `V` of type `CTStyle`. (`CTStyle` is a structural class type — another struct defined elsewhere — carrying a compile-time description of a style: which attributes are on, the colours, and so on.)
- `struct StyTag { ... }`: the type has no data members. It is a *tag* type — empty at runtime, distinct at compile time. Two `StyTag<X>`'s with different `X` are different types.
- `constexpr operator Style() const noexcept`: an implicit conversion from `StyTag<V>` to the runtime `Style` type. `constexpr`, so it can run at compile time; `noexcept` promises it does not throw. `const` on the trailing position says the conversion does not modify the object.
- `return V.runtime();`: ask the compile-time style for its runtime form. `V` is the NTTP, available inside the class.
- `inline constexpr StyTag<CTStyle::bold()> Bold{};`: a `constexpr` variable of type `StyTag<CTStyle::` initialised with default braces. `inline` on a variable in a header lets it appear in multiple translation units without violating ODR. `Bold` is the value you write in user code.
- `Fg<R, G, B>`: a *variable template* — a template that yields a variable, not a function or class. The expression `Fg<255, 80, 80>` instantiates the template with those three integers, producing a unique `StyTag<...>` value.

If you can read that fragment now and feel the pieces in place — NTTP, class template, `constexpr`, `operator T()`, variable template, the implicit conversion that lets `Bold` appear in places that want a `Style` — you can read the rest of maya's headers. From here on, the rest of the book is mostly about what each file *does*, not how its syntax decodes.

3.34.1 A second worked fragment: `Cmd<Msg>`

For practice, here is a paraphrased shape of maya's `Cmd<Msg>` from *core/cmd.hpp*. `Cmd<Msg>` is the type of effect descriptions the runtime knows how to perform.

```
template <typename Msg>
class Cmd {
public:
    struct None {};
    struct Quit {};
    struct After {
        std::chrono::milliseconds delay;
        Msg                       msg;
    };
    struct SetTitle      { std::string title; };
    struct WriteClipboard{ std::string content; };
    struct Batch         { std::vector<Cmd> cmds; };
    struct Task {
        std::move_only_function<void(std::function<void(Msg)>)> run;
    };

    using Variant = std::variant<None, Quit, After, SetTitle,
                                 WriteClipboard, Batch, Task /*, ...*/>;

    Cmd() : v_(None{}) {}
    Cmd(Variant v) : v_(std::move(v)) {}

    [[nodiscard]] static Cmd none() { return Cmd{}; }
    [[nodiscard]] static Cmd quit() { return Cmd{Quit{}}; }
    [[nodiscard]] static Cmd set_title(std::string s) {
        return Cmd{SetTitle{std::move(s)}};
    }
    // ... and so on for every effect

    template <typename Visitor>
    auto visit(Visitor&& vis) const {
        return std::visit(std::forward<Visitor>(vis), v_);
    }

private:
    Variant v_;
};
```

Line by line:

- `template <typename Msg>`: `Cmd` is a class template parameterised by the program's message type, so each program has its own `Cmd` type carrying its own `Msg`s.
- `struct None {}; struct Quit {}; ...`: each effect is a tiny inner struct. Empty structs (`None`, `Quit`) have no data; others carry the parameters of the effect (`After` has a delay and a message; `SetTitle` has a title string).
- `struct Batch { std::vector<Cmd> cmds; }`: a `Batch` contains *other* `Cmd`s, so the runtime can interpret a list of effects atomically. Note this is self-referential — `Cmd` contains a vector of `Cmd` — which works because `Batch`'s storage is in a `std::vector` (heap-allocated, no compile-time recursion).
- `struct Task { std::move_only_function<...> run; }`: a Task carries a callable that the runtime will invoke on a background thread, passing in a callback the task can use to deliver messages back. We met `std::move_only_function` in Section 3.31; we use it here because the callable often captures move-only state (channels, file handles, etc.).

- `using Variant = std::variant<None, Quit, ...>`: the closed sum of every effect kind. This is a type alias inside the class; it gives the variant a stable name.
- `Cmd() : v_(None{}) {}`: default constructor sets the variant to `None`, meaning "no effect". A default `Cmd` is harmless.
- Static factories `none()`, `quit()`, `set_title(...)`: convenience builders. Each returns a `Cmd` holding the right variant alternative. They are `[[nodiscard]]` so you cannot accidentally drop one.
- `template <typename Visitor> auto visit(Visitor&& vis) const`: the public interface for inspecting the effect. Forwards to `std::visit` on the inner variant. `Visitor&&` is a *forwarding reference*; combined with `std::forward`, it preserves whether the caller passed an lvalue or rvalue. You can read this idiom as "perfectly forward the visitor into `std::visit`".
- `Variant v_`: the actual storage. A single member variant; one effect at a time.

The runtime that consumes a `Cmd<Msg>` calls `visit` with an `overload` matching every alternative. If a new effect is added, the runtime visit is the one place that has to change — user code that builds a `Cmd` via the factories does not even recompile (well, of course it does, but it does not need to be edited).

If you understood that fragment, you have just read the core type of the runtime layer. Chapter 14 is the rest of the story: which effects exist, what each does, and how the runtime interprets them.

3.35 Reading compiler errors

C++ compiler errors are notoriously long. A single misplaced semicolon can produce a paragraph of diagnostics; a template error can produce screens of them. Here is a survival guide.

3.35.1 Read the first error first

The rest are almost always consequences of the first. Fix the first and recompile; many of the others usually evaporate.

3.35.2 Look for "required from" or "in instantiation of"

Template errors print a long "stack" of

```
foo.cpp:10:5:   in instantiation of 'X<int>' requested here
bar.hpp:42:7:   in instantiation of 'Y<X<int>>' requested here
baz.hpp:99:3:   required from here
qux.hpp:12:1:   error: no type named 'value_type' in 'X<int>'
```

The *deepest* error is the actual reason; the lines above are the path the compiler took to get there. Start at the bottom; work up.

3.35.3 Concepts errors are tractable

If the failing template is constrained by a concept, the error names the concept and the unmet requirement directly. This is one of the killers reasons to use concepts. Compare:

```
# without concepts:
error: no match for 'operator+' (operand types are 'File' and 'File')
... 50 lines of template instantiation context ...

# with concepts:
error: 'add(File, File)' does not satisfy concept 'Addable'
  note: the required expression '(a + b)' is invalid
```

When a maya API takes a constrained template parameter, the error you get if you violate the constraint is a single, named, readable line.

3.35.4 Common shapes you will see

- "`no matching function for call to ...`" — you called a function with the wrong argument types. The compiler lists every candidate and says why each was rejected.
- "`undefined reference to ...`" — a *linker* error, not a compiler error. You declared a function but did not define it (or did not link the file that does).
- "`expected ';' before ...`" — a missing semicolon somewhere *earlier*, often at the end of a struct declaration.
- "`use of deleted function ...`" — you tried to copy something that is move-only (e.g. a `unique_ptr`). Add `std::move`.
- "`conversion from X to Y is ambiguous`" — two different paths lead to the conversion. Disambiguate with an explicit `static_cast`.

Pitfall

Compiler errors involving `std::tuple`, `std::variant`, or `std::function` can run to thousands of characters. Pipe to `less` or use an editor with line-wrap. The relevant signal is still the first error.

Recap

The C++ maya uses, in one paragraph: templates with type and non-type parameters; `constexpr`/`consteval` for compile- time evaluation; concepts and `requires` for named constraints; `std::variant` + `std::visit` + `overload` for closed sum types; `std::expected` for structured errors; lambdas with explicit capture and `std::move_only_function` for storage; move semantics to avoid copies; RAII to make resources self-managing. Together they support one discipline: encode invariants in the type system, make impossible states unrepresentable, defer side effects to the edge.

Workshop

The exercises here are calibrated for someone who has never used the feature. If a step takes you fifteen minutes and three Google searches, that is the right speed.

1. **RAII guard.** Write a class `Timer` whose constructor records `std::chrono::steady_clock::now()` and whose destructor prints the elapsed time. Use it at the top of a function; verify the message appears on return.

2. **Templates and NTTPs.** Implement a `FixedArray<T, N>` class template with a member `T data[N]`, a `size()` method returning `N`, and `operator[]`. Instantiate `FixedArray<int, 4>` and `FixedArray<int, 8>`; convince yourself they are different types by trying to assign one to the other.

3. **NTTP of structural type.** Type out the `Str<N>` template from this chapter. Define a class template `Tag<Str S>` parameterised by it. Instantiate `Tag<"hello">` and `Tag<"world">` and call a method that returns the string.

4. **Concept.** Write a concept `Drawable` requiring a `void draw() const` method. Use it on a template function `render(const T& t)`. Make two structs satisfying it, one not, and observe the error.

5. **Variant.** Define `using Shape = std::variant<Circle, Square, Triangle>` with three trivial structs (each with one field). Write a `double area(const Shape&)` using `std::visit` and the `overload` helper.

6. **Expected.** Write `std::expected<int, std::string> safe_div(int a, int b)`. Call it with `(10, 2)` and `(10, 0)`; print each result.

7. **Lambda with move capture.** Build a `std::vector<int>`, capture it into a lambda by move, and return the lambda from a function. Call the returned lambda; confirm the vector outlives the function it came from.

Part II

Architecture

The Elm loop and algebraic effects

maya organises a whole application around a single idea borrowed from the Elm language: every program is a pure function from the current state and an incoming event to the next state. Anything that has to *happen* in the outside world — a timer firing, a key being read, a clipboard being written, an HTTP request — is returned as a *description of work to do*, not performed inside the function. This chapter explains the idea, why it is worth the discipline, and how maya encodes it.

4.1 Where state usually lives

The most common way to write a UI program is to keep state in mutable fields scattered across whatever objects feel relevant, and to mutate them from inside event handlers:

```
class Counter {
    int count = 0;
public:
    void on_plus()  { ++count; redraw(); }
    void on_minus() { --count; redraw(); }
};
```

This works for tiny programs. It breaks down for the kind of program where maya is interesting — programs with several concurrent sources of events (keys, mouse, timers, async tasks), where you want to undo, replay, snapshot, or test the logic in isolation. The state is everywhere; the events come from everywhere; there is no single place to put a breakpoint.

4.2 The Elm shape

The Elm Architecture says: pick one type to be your state (the **Model**), pick one closed sum type to be your events (the **Msg**), and write three functions:

- `init() -> Model` — the starting state.
- `update(Model, Msg) -> Model` — given old state and an event, return new state.
- `view(const Model&) -> Element` — given state, produce the UI to display.

Everything outside those three functions is the *runtime*: it reads keys, calls `update`, calls `view`, paints the result, loops.

Here is the world's smallest example, the counter from Chapter 1 but written in this style:

```
struct Counter {
    struct Model { int count = 0; };
    struct Inc {}; struct Dec {}; struct Quit {};
    using Msg = std::variant<Inc, Dec, Quit>;

    static Model init() { return {}; }
```

```
    static Model update(Model m, Msg msg) {
        return std::visit(overload{
            [&](Inc)  → Model { return {m.count + 1}; },
            [&](Dec)  → Model { return {m.count - 1}; },
            [&](Quit) → Model { return m; }
        }, msg);
    }

    static Element view(const Model& m) {
        return v(text(m.count) | Bold,
                 t<"[+/-] q quit"> | Dim) | pad<1> | border_<Round>;
    }
};
```

Read it twice. Notice:

- `Model` is a plain value. No pointers, no inheritance.
- `Msg` is a closed sum: *exactly* `Inc`, `Dec`, or `Quit`. If you add a fourth event tomorrow, every `std::visit` call in the program complains until you handle it.
- `update` is pure: same model + same message ⇒ same result. You can call it from a test with hand-rolled `Model`s and `Msg`s and check equality.
- `view` is pure too: same model ⇒ same UI.

Idea

A pure `update` function is the smallest unit of testable behaviour in a maya program. There is no mock terminal, no fake event loop, no setup code — just two values in, one value out.

4.3 But programs need to do things

Real programs have to perform side effects. Reading a file. Setting the terminal title. Spawning a subprocess. Quitting. If `update` is pure, it can't do any of those.

The Elm answer is brilliant: have `update` *return* the side effects as values. They are descriptions of work, not work. The runtime, which has permission to do impure things, interprets them.

This is the technique known as **algebraic effects** or *effects as data*. In Haskell it is the `IO` monad. In Rust it is libraries like `tokio::Command`. In `maya` it is `Cmd<Msg>`, which is a `std::variant` of all the effects the runtime knows how to interpret:

```
template <typename Msg>
class Cmd {
public:
    struct None {};
    struct Quit {};
    struct Batch { std::vector<Cmd> cmds; };
    struct After { std::chrono::milliseconds delay; Msg msg; };
    struct SetTitle      { std::string title; };
    struct WriteClipboard { std::string content; };
    struct Task         { std::function<void(std::function<void(Msg)>)> run; };
    struct IsolatedTask { std::function<void(std::function<void(Msg)>)> run; };
    // ...
};
```

`update` now returns a `std::pair<Model, Cmd<Msg»`:

```
static std::pair<Model, Cmd<Msg>> update(Model m, Msg msg) {
    return std::visit(overload{
        [&](Inc)  → std::pair<Model, Cmd<Msg>> {
            return {{m.count + 1}, Cmd<Msg>{}};
        },
        [&](Save) → std::pair<Model, Cmd<Msg>> {
            return {m, Cmd<Msg>::write_clipboard(std::to_string(m.count))};
        },
        [&](Quit) → std::pair<Model, Cmd<Msg>> {
            return {m, Cmd<Msg>::quit()};
        }
    }, msg);
}
```

`update` still does no I/O. It *describes* I/O. The runtime sees `WriteClipboard`, calls the OS clipboard API, and the next call to `update` sees the world in a new state.

4.4 Subscriptions: the other half

`Cmd` describes effects the program *starts*. There is also the symmetric notion: events the program *receives*. Keystrokes, mouse moves, timers, async results — these are sources of `Msg`s. maya models them with `Sub<Msg>`:

```
template <typename Msg>
class Sub {
public:
    struct Keys     { std::function<std::optional<Msg>(Key)>   f; };
    struct Mouse    { std::function<std::optional<Msg>(Mouse)> f; };
    struct Tick     { std::chrono::milliseconds period; std::function<Msg()> f; };
    // ...
};
```

`subscribe(const Model&) -> Sub<Msg>` is the fourth program function. Like the others, it is pure: given a model, declare which event sources are active right now. The runtime samples the subscriptions, reads from those sources, and feeds the `Msg`s back into `update`.

A complete Elm-style maya program is therefore a struct with four static functions:

```
init        →                    (Model, Cmd<Msg>)
update      (Model, Msg) →       (Model, Cmd<Msg>)
view        (const Model&) →     Element
subscribe   (const Model&) →     Sub<Msg>
```

This four-tuple is the `Program` concept:

```
template <typename P>
concept Program = requires(typename P::Model m, typename P::Msg msg) {
    typename P::Model;
    typename P::Msg;
    { P::init() }           → std::convertible_to<std::pair<typename P::Model, Cmd<
    typename P::Msg>>>;
    { P::update(m, msg) }   → std::convertible_to<std::pair<typename P::Model, Cmd<
    typename P::Msg>>>;
    { P::view(m) }          → std::convertible_to<Element>;
    { P::subscribe(m) }     → std::convertible_to<Sub<typename P::Msg>>;
};
```

`run<P>()` requires `Program<P>`; if your program is missing a function or has a wrong signature, the compiler tells you which one and why.

4.5 Why this is worth the effort

A pure-functional core has properties that an imperative one doesn't:

- **Testability without mocks.** You write `ASSERT(update({5}, Inc{}).first == Model{6})`. No terminal, no event loop, no fixtures.
- **Determinism.** Given a recorded sequence of `Msg`s, you can reproduce any rendered frame. maya leverages this for debugging.
- **Time travel.** Storing a list of past models is storing a list of plain values — snapshot, replay, undo.
- **Local reasoning.** A bug in `update` cannot corrupt the renderer; a bug in the renderer cannot mutate the model. The interfaces between layers are values, not method calls.

The cost is some up-front structure. The four-function skeleton feels heavy for a 30-line program; maya provides simpler entry points (`run(event_fn, render_fn)`) for those cases. For anything that will live longer than an afternoon, the Elm shape is the right shape.

4.6 The runtime loop, sketched

The `run<P>()` runtime is one loop. Stripped of error handling and the inline-mode branch, it looks like this:

```
template <Program P>
void run(RunConfig cfg) {
    auto [model, cmd0] = P::init();
    Renderer r{cfg};
    interpret(cmd0);                          // perform initial effects

    while (!quit_requested()) {
        auto subs = P::subscribe(model);       // declare event sources
        Msg msg   = wait_for(subs);            // block until something arrives

        auto [next_model, next_cmd] = P::update(std::move(model), msg);
        model = std::move(next_model);
        interpret(next_cmd);                   // perform effects from update

        r.paint(P::view(model));               // re-render
    }
}
```

That is the entire shape. Everything else in maya's architecture chapter is an embellishment: the canvas double-buffer, the SIMD diff, the inline-mode renderer, the BG thread pool that interprets `Task` commands. All in service of this loop.

Theory

Algebraic effects is a programming-language idea: separate *describing* what should happen from *deciding how*. The function returns a value of an effect algebra; an interpreter chooses how to run it. The same description can be run for real, run in a test (with a mock interpreter), or replayed from a log. Elm's `Cmd` and `Sub` are a small, practical algebra of effects.

4.7 From single-function to Program: a worked refactor

Here is the same program in both styles. First, the quick-tool form (useful for one-off scripts):

```
Signal<int> n{0};
run({.title = "counter"},
    [&](const Event& ev) {
        if (key(ev, '+')) n.update([](int& x){ ++x; });
        if (key(ev, '-')) n.update([](int& x){ --x; });
        return !key(ev, 'q');
    },
    [&] {
        return v(text("Count: " + std::to_string(n.get())) | Bold,
                 t<"[+/-] q quit"> | Dim) | pad<1>;
    });
```

This is fine. But the state is captured by reference inside a lambda, the event handler returns a `bool`, and there is no separation between business logic and presentation.

The Program version:

```
struct Counter {
    struct Model { int count = 0; };
    struct Inc {}; struct Dec {}; struct Quit {};
    using Msg = std::variant<Inc, Dec, Quit>;

    static auto init() → std::pair<Model, Cmd<Msg>> {
        return {{}, Cmd<Msg>::none()};
    }
    static auto update(Model m, Msg msg) → std::pair<Model, Cmd<Msg>> {
        return std::visit(overload{
            [&](Inc)  { return std::pair{Model{m.count + 1}, Cmd<Msg>{}}; },
            [&](Dec)  { return std::pair{Model{m.count - 1}, Cmd<Msg>{}}; },
            [](Quit)  { return std::pair{Model{}, Cmd<Msg>::quit()}; }
        }, msg);
    }
    static Element view(const Model& m) {
        return v(text("Count: " + std::to_string(m.count)) | Bold,
                 t<"[+/-] q quit"> | Dim) | pad<1>;
    }
    static Sub<Msg> subscribe(const Model&) {
        return key_map<Msg>({{'+', Inc{}}, {'-', Dec{}}, {'q', Quit{}}});
    }
};
int main() { run<Counter>({.title = "counter"}); }
```

It is a few more lines but the parts are now nameable. Tests look like:

```
TEST_CASE("incrementing the counter") {
    auto [m, _] = Counter::update({.count = 5}, Counter::Inc{});
    CHECK(m.count == 6);
}
```

No terminal, no event loop, no render. That is the entire point.

Recap

A `maya` application is a `Program`: a `Model` type, a `Msg` sum, and four pure functions `init`/`update`/ `view`/`subscribe`. Effects (`Cmd`) and event sources (`Sub`) are described as data; the runtime interprets them. The result is a program whose business logic is trivially testable.

Workshop

1. Add a `Reset` message to the counter that sets `count` back to zero. Add a `r` keybinding.
2. Add a `Tick` subscription that increments the counter once per second.
3. Add a `Save` message that writes the current count to the clipboard via `Cmd<Msg>::write_clipboard(...)`.
4. Write a unit test (just a `static_assert` or a one-line `main`) that pumps three `Inc`s through `update` and checks the final count.

The compile-time DSL

maya's most distinctive feature is its DSL — a tiny embedded language inside C++ for describing UI trees. The DSL is unusual because it lives entirely at the type level: an expression like

```
t<"Hello"> | Bold | Fg<100, 180, 255>
```

is a *single value* whose type encodes every choice you made (*the string*, *bold*, *the colour*). This chapter shows how that works, why the compiler refuses certain expressions, and how to escape into the runtime when you need to.

5.1 The shape

The DSL has six kinds of node:

`TextNode<Str, CTStyle>`	compile-time text
`RuntimeTextNode<S>`	runtime text (from a string or number)
`BoxNode<Dir, Cfg, Cs...>`	vertical or horizontal container
`DynNode<F>`	lambda escape hatch
`MapNode<R, P>`	project a range to elements
`SpacerNode`	a flexible gap

Every node satisfies the `Node` concept:

```
template <typename T>
concept Node = requires(const T& n) {
    { n.build() } → std::convertible_to<Element>;
};
```

A `Node` is anything with a `.build()` that yields a runtime `Element`. That is the only contract; the rest is how nodes combine.

5.2 `t<"Hello">` — string in a type

The simplest node is the compile-time text node:

```
template <Str S, CTStyle Sty = CTStyle{}>
struct TextNode {
    static constexpr Str  str   = S;
    static constexpr CTStyle style = Sty;

    Element build() const {
        return TextElement{std::string{S.view()}, Sty.runtime()};
    }
    operator Element() const { return build(); }
};

template <Str S>
inline constexpr TextNode<S> t{};
```

The template parameters are: an `Str` (the string contents, an NTTP from Chapter 3) and a `CTStyle` (a structural type holding the compile-time style). `t<"Hello">` is the variable template: `TextNode<"Hello", CTStyle{}>{}`.

Two facts to internalise:

- `t<"a">` and `t<"b">` have *different types*. Their string contents live in the type system, not in a field.
- `TextNode` has no nonstatic data members. It is a zero-size type. Two instances of `TextNode<"a">` are indistinguishable; the compiler treats them as the same value.

5.3 Pipes are operator overloads

The DSL's pipe (`|`) is just `operator|`. `maya` defines overloads that match `(Node, StyTag)` and return a new node with the style merged in:

```
template <Str S, CTStyle Sty, CTStyle V>
constexpr auto operator|(TextNode<S, Sty>, StyTag<V>) {
    return TextNode<S, Sty.merge(V)>{};   // new type, same instance shape
}
```

Read the return type: the merged style is computed by `Sty.merge(V)`, which is `constexpr`, so the result is a brand-new `TextNode<S, Sty'>`. Apply two style tags and you get yet another type. The pipeline is sequential type rewriting at compile time.

Idea

Because the result of each pipe is a new type, the compiler can enforce *type-state* rules: a method is only available on a node that has reached a particular state.

This is the trick behind the border colour example. The pipe overload for `BColTag` (a border colour tag) has a `requires` clause:

```
template <FlexDirection Dir, BoxCfg Cfg, typename... Cs,
          uint8_t R, uint8_t G, uint8_t B>
    requires (Cfg.has_border)
constexpr auto operator|(BoxNode<Dir, Cfg, Cs...> n, BColTag<R, G, B>) {
    /* return a new BoxNode with border colour set */
}
```

If you call `v(...) | bcol<60, 65, 80>` without first applying `border_<Round>`, `Cfg.has_border` is `false`, the `requires` clause fails, no overload matches, and the compiler prints an error. It is the same mechanism a Rust crate would use with typestate traits.

5.4 `v()` and `h()` — variadic containers

A box takes any number of children and a direction:

```
template <FlexDirection Dir, BoxCfg Cfg, typename... Cs>
struct BoxNode {
    std::tuple<Cs...> children;

    Element build() const {
        BoxBuilder b{Dir, Cfg};
```

```
        std::apply([&](const auto&... cs) {
            (b.add(cs.build()), ...);   // fold-expression over children
        }, children);
        return b.finalize();
    }
};

template <typename... Cs>
constexpr auto v(Cs... cs) {
    return BoxNode<FlexDirection::Column, BoxCfg{}, Cs...>{
        std::tuple{std::move(cs)...}
    };
}

template <typename... Cs>
constexpr auto h(Cs... cs) {
    return BoxNode<FlexDirection::Row, BoxCfg{}, Cs...>{
        std::tuple{std::move(cs)...}
    };
}
```

Two C++ features worth pausing on:

- **Variadic templates** (`typename... Cs`): you can declare "zero or more types". `Cs...` is a *parameter pack*; you expand it with `...`.
- **Fold expressions** (`(b.add(cs.build()), ...)`): short for "apply this expression to every element of the pack". The comma is the *fold operator* — evaluate each, discard intermediate results.

The result: `v(t<"a">, t<"b">, t<"c">)` is a single `BoxNode<Column, BoxCfg{}, TextNode<"a">, TextNode<"b">, TextNode<"c"»`, holding a `tuple` of three zero-size text nodes. Total runtime cost: zero bytes (the tuple of empty types is empty).

5.5 BoxCfg — the type-state

`BoxCfg` is a `constexpr` struct that records every layout decision applied to a box:

```
struct BoxCfg {
    int pad_top    = 0, pad_right = 0, pad_bottom = 0, pad_left = 0;
    int gap        = 0;
    int grow_num   = 0;
    bool has_border       = false;
    BorderStyle border    = BorderStyle::None;
    bool has_border_color = false;
    Color border_color{};
    Align align_items    = Align::Auto;
    Justify justify      = Justify::Auto;
    // ... etc.

    constexpr BoxCfg with_padding(int t, int r, int b, int l) const {
        BoxCfg c = *this;
        c.pad_top = t; c.pad_right = r; c.pad_bottom = b; c.pad_left = l;
        return c;
    }
    constexpr BoxCfg with_border(BorderStyle bs) const {
        BoxCfg c = *this; c.has_border = true; c.border = bs; return c;
    }
```

```
    // ...
};
```

Every layout pipe (`pad<1>`, `gap_<1>`, `border_<Round>`, `grow_<2>`) returns a new `BoxNode` whose `BoxCfg` template parameter has been replaced via the corresponding `with_*` method. The type carries the full configuration; the final `.build()` reads the type and produces the runtime `BoxElement`.

5.6 Static, dynamic, and the bridge

A pure-compile-time tree is fine for a banner but useless for a real app: the count, the file list, the current time are all known only at runtime. maya provides three bridge nodes.

5.6.1 `text(...)` — runtime strings

```
text("hello")               // wraps a string_view
text(std::to_string(n))     // wraps a string
text(42)                    // wraps an int, formatted at build time
text(3.14)                  // wraps a double
```

`text` returns a `RuntimeTextNode<S>` — a node whose content is a runtime value but which still supports the DSL pipes. Its `build()` formats the value and produces a `TextElement`.

5.6.2 `dyn([]() ... )` — arbitrary lambdas

```
dyn([&] {
    return loading ? text("Loading...") | Dim
                   : text("Ready")      | Bold | Fg<80, 220, 120>;
})
```

`dyn` wraps a lambda that returns anything convertible to `Element`. `build()` just calls the lambda. This is the universal escape hatch: when you cannot express something in the DSL itself, write a lambda.

5.6.3 `map(range, projection)` — list comprehension

```
std::vector<File> files = ...;

auto list = v(
    t<"Files:"> | Bold,
    map(files, [](const File& f) {
        return h(text(f.name) | Bold, space, text(f.size) | Dim);
    })
);
```

`map` produces a `MapNode<R, P>`. At `build()` time it walks the range, applies the projection, and produces an `ElementList` that is spliced into the parent's children.

5.7 Reading a real pipeline

Put it all together. Here is a typical maya panel:

```
auto panel = v(
    t<"Dashboard"> | Bold | Fg<100, 180, 255>,
    sep,
    dyn([&] { return text("Active users: " + std::to_string(state.users)); }),
    map(state.items, [](const auto& it) {
        return h(text(it.name) | Bold, space, text(it.value) | Dim);
    }),
    space,
    t<"[q] quit"> | Dim
) | border_<Round> | bcol<60, 65, 80> | pad<1, 2> | gap_<1>;
```

What is the type of `panel`? Approximately:

```
BoxNode<
    FlexDirection::Column,
    BoxCfg{ /* round border, colour, pad 1/2, gap 1 */ },
    TextNode<"Dashboard", { bold, fg=(100,180,255) }>,
    SepNode,
    DynNode<lambda1>,
    MapNode<vector<Item>, lambda2>,
    SpacerNode,
    TextNode<"[q] quit", { dim }>
>
```

The compiler holds every choice you made — the strings, the styles, the border, the padding — in this one large type. `panel.build()` then walks it once, recursively, and produces an `Element` tree that the layout engine and renderer take from there.

5.8 Why compile-time?

You could write this DSL with runtime polymorphism (each node a `shared_ptr<Node>`, each pipe a method that mutates fields). It would work. But:

- **Allocations vanish.** A purely compile-time tree of zero-size types has no heap traffic.
- **Errors are at compile time.** Border colour without a border, negative padding (template parameter constraint `N >= 0`), wrong type of child — all rejected before the program runs.
- **The compiler can fold.** A `constexpr` tree can be assigned to a `constexpr` variable and rendered at startup with no parsing.

The trade-off: error messages can be long. The first time you see a 500-line template error, breathe. The bottom of the message — the `required from here` line and the constraint that failed — is usually the actionable bit.

5.9 Where the runtime takes over

`.build()` is the boundary. Above it: types, NTTPs, fold expressions, the world the compiler sees. Below it: `Element`, a runtime value the layout and renderer understand. The next chapter goes through that runtime side.

Recap

maya's DSL is a set of node types that compose with `|` into larger node types. Strings, styles, borders, paddings, and tree structure all live as template parameters. Type-state constraints make invalid trees uncompilable. Runtime content enters through `text`, `dyn`, and `map`. `.build()` converts the compile-time tree into a runtime `Element`.

Workshop

1. Write a panel that shows your name in bold, an underlined tagline, and a divided list of three hobbies. Use only the compile-time pipes.
2. Add a runtime counter (an `int` variable) with `dyn`. Increment it in a loop with `maya::live(...)` (covered in Chapter 14) and watch the panel refresh.
3. Try `bcol<60, 65, 80>` on a box without a border. Read the compiler error. Find the line that says `constraints not satisfied` and the `Cfg.has_border` that triggered it.

Elements, style, colour, border

`.build()` converts the compile-time tree from Chapter 5 into an `Element`. That `Element` is the input to the layout engine, the renderer, and every widget. This chapter dissects `Element` and the value types it depends on.

6.1 `Element`: a tagged union

`Element` is a `std::variant` of three alternatives:

```
using Element = std::variant<BoxElement, TextElement, ElementList>;
```

- `TextElement` — a piece of styled text, possibly with a wrap policy.
- `BoxElement` — a container with flex layout, padding, border, gap, etc., and a vector of child elements.
- `ElementList` — a flat sequence of elements that should be inlined into the parent's children (used by `map`).

Three alternatives, a closed set. Every algorithm in maya that walks an element tree (layout, render, hit-testing, search) ends up in a `std::visit` on this variant. Add a fourth alternative tomorrow and the compiler tells you all the places to update.

6.1.1 `TextElement`

```
struct TextElement {
    std::string text;        // the bytes (UTF-8)
    Style       style;       // colours + attributes
    TextWrap    wrap = TextWrap::Wrap;
    HyperlinkId link{};      // optional OSC-8 link
};
```

A leaf in the tree. The string is owned (so the source data can be a temporary). Styling lives in a `Style` struct (next section). `TextWrap` is one of `Wrap`, `NoWrap`, `Truncate`, `Ellipsize`.

6.1.2 `BoxElement`

```
struct BoxElement {
    layout::FlexStyle    style;     // direction, padding, gap, ...
    std::optional<Border> border;
    std::vector<Element>  children;
    std::optional<std::string> title;  // border title
    // ... a few more renderer hints
};
```

The composite. `style` carries every layout property as integer cell counts; `border` is set when the DSL chain included `border_<...>`. `title` is set when a border title was piped on.

6.1.3 ElementList

```
struct ElementList { std::vector<Element> items; };
```

A pseudo-element. The renderer treats it as "these children belong to my parent." It is the runtime equivalent of a parameter pack expansion.

6.2 Style

`Style` is a small POD struct describing the appearance of a run of text:

```
struct Style {
    Color     fg{};
    Color     bg{};
    Attribute attrs{};   // bold | italic | underline | dim | ...

    constexpr Style with_bold()      const noexcept;
    constexpr Style with_italic()    const noexcept;
    constexpr Style with_fg(Color c) const noexcept;
    constexpr Style with_bg(Color c) const noexcept;
    constexpr Style merge(Style other) const noexcept;
    // ...
};
```

Two flavours exist:

- `Style` (runtime), used in `TextElement`.
- `CTStyle` (compile-time), used as an NTTP in `TextNode`. It is structurally identical but lives in template parameters and is constructed with `constexpr` factory functions (`CTStyle::bold()`, `CTStyle::fg(R, G, B)`, etc.).

The DSL pipes operate on `CTStyle`; `.build()` calls `CTStyle.runtime()` to project back to a `Style`. The distinction matters because NTTPs require structural types whose fields are compared by value, while runtime `Style` can have fancier constructors.

6.2.1 Attribute bitset

Attributes (bold, dim, italic, underline, strikethrough, inverse, blink) are a bitset:

```
enum class Attribute : std::uint8_t {
    None       = 0,
    Bold       = 1 << 0,
    Dim        = 1 << 1,
    Italic     = 1 << 2,
    Underline  = 1 << 3,
    Strike     = 1 << 4,
    Inverse    = 1 << 5,
    Blink      = 1 << 6,
};

constexpr Attribute operator|(Attribute a, Attribute b) noexcept;
constexpr bool      has(Attribute set, Attribute bit) noexcept;
```

Combining attributes is bitwise OR. Checking is bitwise AND. The renderer translates each bit to its SGR parameter (1 for bold, 2 for dim, 3 for italic, etc.) when serialising.

6.3 Color

A `Color` is a 24-bit RGB triple plus a tag for "default" (unspecified, inherit from terminal):

```
struct Color {
    std::uint8_t r = 0, g = 0, b = 0;
    bool         is_default = true;   // means "use terminal default"
};

constexpr Color rgb(uint8_t r, uint8_t g, uint8_t b) noexcept {
    return {r, g, b, false};
}
```

maya only deals in 24-bit truecolour internally. The 256-colour and 16-colour palettes were popular when terminals had limited support; modern terminals all do truecolour. If you really need 256-colour output (because you are targeting a niche emulator), the writer can quantise on the way out.

6.3.1 Themes

A **theme** is a 24-slot palette of colours — one for primary, secondary, success, warning, error, plus background tiers and text tiers. maya's built-in themes (`dark`, `light`, `terminal`) are `constexpr` aggregates; you can derive your own and pass it in `RunConfig`.

```
struct Theme {
    Color bg_primary;
    Color bg_secondary;
    Color text_primary;
    Color text_secondary;
    Color accent;
    Color success;
    Color warning;
    Color error;
    // ... 24 slots total
};
```

The `Ctx` parameter to a render function (in the quick-tool `run(...)`) carries the active theme. Widgets read it instead of hard-coding colours — so the same widget tree looks right under dark and light themes.

6.4 Border

A border is described by its style (single, double, round, thick, classic, arrow), its colour (or default), and which sides are drawn:

```
enum class BorderStyle : std::uint8_t {
    None, Single, Double, Round, Bold, Classic, Arrow,
};

struct BorderSides {
    bool top = true, right = true, bottom = true, left = true;

    static constexpr BorderSides all()        { return {true, true, true, true}; }
    static constexpr BorderSides none()       { return {false,false,false,false}; }
    static constexpr BorderSides horizontal() { return {true,false,true,false}; }
    static constexpr BorderSides vertical()   { return {false,true,false,true}; }
};
```

```

struct Border {
    BorderStyle style = BorderStyle::Single;
    Color       colour{};
    BorderSides sides = BorderSides::all();
};
```

Each `BorderStyle` maps to a set of eight glyphs (corners + edges) that the renderer paints. The mapping is a `constexpr std::array` in *maya/style/border.hpp*.

6.4.1 `Sep` and `VSep`

The DSL's `sep` and `vsep` nodes are syntactic sugar for a box with a one-side-only border — horizontal or vertical. The same machinery; the renderer treats them identically.

6.5 `BoxBuilder`: runtime construction

The DSL `.build()` fans out into a `BoxBuilder` which assembles a `BoxElement`:

```
class BoxBuilder {
public:
    BoxBuilder(FlexDirection dir, BoxCfg cfg)
      : dir_(dir), cfg_(cfg) {}

    void add(Element child) { children_.push_back(std::move(child)); }
    void add(ElementList list) {
        children_.insert(children_.end(),
                         std::make_move_iterator(list.items.begin()),
                         std::make_move_iterator(list.items.end()));
    }

    Element finalize() &&;
private:
    FlexDirection         dir_;
    BoxCfg                cfg_;
    std::vector<Element>  children_;
};
```

Two overloads of `add`: one for ordinary children, one for an `ElementList` which is spliced. `finalize()` translates `BoxCfg` into a `layout::FlexStyle`, attaches a `Border` if requested, wraps it in a `BoxElement`, and returns the variant.

6.6 Walking an element tree

Every algorithm that processes an element tree is a recursive `std::visit`. Here is the shape of a pretty-printer:

```
void print(const Element& e, int depth = 0) {
    std::string indent(depth * 2, ' ');
    std::visit(overload{
        [&](const TextElement& t) {
            std::println("{}TEXT  : \"{}\"", indent, t.text);
        },
        [&](const BoxElement& b) {
            std::println("{}BOX   : dir={}, {} children",
```

```
                        indent, int(b.style.direction), b.children.size());
            for (const auto& c : b.children) print(c, depth + 1);
        },
        [&](const ElementList& l) {
            for (const auto& c : l.items) print(c, depth);
        }
    }, e);
}
```

This is the template every renderer-side pass follows: visit, branch on alternative, recurse on children.

6.7 Hyperlinks

A small but neat feature: text elements can carry a hyperlink. The renderer emits an OSC-8 sequence around the run:

```
ESC ] 8 ; ; URL ESC \  text  ESC ] 8 ; ; ESC \
```

Modern emulators turn that into a clickable link. maya interns the URL into a `HyperlinkId` so the per-cell representation stays compact (more on that in Chapter 8).

6.8 Why so many small structs?

A reasonable question: why isn't `Element` a class with virtual methods? maya's answer: virtual dispatch precludes a packed cell representation, prevents `constexpr` construction, and makes serialization harder. The variant + visitor pattern gives you polymorphism without the overhead — `std::visit` compiles to a jump table or a chain of branches, both faster than a virtual call, and the data layout stays cache-friendly.

Recap

The runtime tree is `Element = variant<BoxElement, TextElement, ElementList>`. Style, Color, and Border are small POD structs; attributes are a bitset; themes are 24-slot palettes. Every algorithm in maya walks the tree with `std::visit`. `BoxBuilder` bridges the compile-time DSL to the runtime tree.

Workshop

1. Open *maya/include/maya/element/element.hpp* and find the `Element` alias. Note that it is exactly the variant described in this chapter.
2. Write a function `int count_text_nodes(const Element&)` using `std::visit`. Test it on a panel from Chapter 5.
3. Construct an `Element` by hand — not via the DSL — using `BoxBuilder`. Render it with `print(...)`.

Flexbox in integers

Given an element tree, maya must decide where every text run, every box, every border goes — in cells, not pixels. The algorithm it uses is a tailored re-implementation of CSS Flexbox. This chapter explains flexbox from scratch (no web background assumed) and then walks maya's implementation.

7.1 The problem layout solves

You have a rectangular area — the terminal — and a tree of elements each with constraints (*this one wants to be 20 cells wide; this row should pack its children with a 1-cell gap; this panel must grow to fill leftover space*). Layout is the function that assigns concrete `(x, y, w, h)` rectangles to each node such that all the constraints are satisfied.

There are many possible algorithms (table layout, constraint solving, grid layout). Flexbox is the most appropriate for terminal UIs: predictable, single-pass, no surprises, no solver.

7.2 Flexbox in one paragraph

Flexbox arranges children along a **main axis** (row or column). Children may *grow* to fill leftover space and *shrink* when oversized; they may *wrap* onto new lines when they don't fit; they may be *aligned* along the cross axis and *justified* along the main axis. Each container picks an axis (row or column); each child carries a few numbers (basis, grow, shrink, gap, alignment override).

7.3 Vocabulary

A worked example will be clearest. Take a horizontal row of three children inside a 30-cell-wide box:

```
h(
    panel_a,    // wants 10 cells
    panel_b,    // grow=1
    panel_c     // wants 8 cells
) | gap_<1>
```

maya reasons about it like this:

1. **Sum the basis sizes.** A=10, B=0 (no fixed size, grows), C=8. Plus 2 gap cells. Total committed: 20.
2. **Leftover space.** 30 - 20 = 10 cells.
3. **Distribute by grow.** Only B has `grow > 0`, so B receives all 10 cells. New size: B = 10.
4. **Place.** A at column 0, gap, B at column 11, gap, C at column 22.

The cross axis (vertical, in this example) is handled by `Align`: children stretch to the parent height by default, but they can be `Start`, `Center`, `End`, or `Stretch`.

7.4 The flex properties

A node in maya's layout engine carries the following style:

```
struct FlexStyle {
    FlexDirection direction = FlexDirection::Column;
    FlexWrap      wrap      = FlexWrap::NoWrap;

    Align   align_items   = Align::Auto;
    Align   align_self    = Align::Auto;
    Justify justify       = Justify::Auto;

    int     gap           = 0;

    int     pad_top    = 0, pad_right = 0,
            pad_bottom = 0, pad_left  = 0;
    int     mar_top    = 0, mar_right = 0,
            mar_bottom = 0, mar_left  = 0;

    int     width       = -1;   // -1 = auto
    int     height      = -1;
    int     min_width   = 0, max_width  = INT_MAX;
    int     min_height  = 0, max_height = INT_MAX;

    int     basis       = -1;   // preferred main-axis size, -1 = auto
    float   grow        = 0.0f;
    float   shrink      = 1.0f;

    Display display     = Display::Flex;
    Overflow overflow   = Overflow::Visible;
    bool    has_border  = false;
};
```

These mirror CSS flexbox almost exactly. The differences:

- Every quantity is an *integer* number of cells. No `em`, no `px`, no fractional pixels. Sub-cell positions don't exist in a terminal.
- `display: Flex` and `display: None` are the only options. There is no block, no inline.
- Percentages are limited; most sizes are explicit integers.

7.5 The data structure

A layout tree is a flat `std::vector<LayoutNode>` indexed by `int`. Each node has a list of child indices.

```
struct LayoutNode {
    FlexStyle           style;
    std::vector<int>    children;     // indices into the same vector
    layout::Rect        computed;     // output: x, y, w, h
};
```

Why a flat vector and not a pointer tree? Three reasons:

- **Cache locality.** Walking siblings is a stride of `sizeof(LayoutNode)`; the prefetcher loves it.
- **No allocation in the hot path.** A `vector<int>` for children is cheaper than a `vector<unique_ptr<LayoutNode»`.
- **Easier reasoning.** Indices cannot dangle; an index is either in range or out of range. Pointer trees create aliasing puzzles.

Idea

"Make it an index, not a pointer" is a recurring optimisation in maya. The same idea drives the style pool (a 16-bit ID into a table) and the hyperlink interning (a 32-bit ID).

7.6 The algorithm, step by step

`layout::compute(nodes, root, available_width)` runs in three phases per box:

1. **Measure.** Walk children recursively and compute their intrinsic main-axis size (the basis if specified, else the content's measured size).
2. **Distribute.** Add up basis sizes plus gaps. Compare to the parent's main-axis size. If there is leftover space and any child has `grow > 0`, distribute the leftover proportionally; if there is overflow and any child has `shrink > 0`, shrink proportionally.
3. **Place.** Compute final `(x, y, w, h)` for each child given the resolved main-axis sizes and the cross-axis alignment.

Wrapping (when `wrap = Wrap`) re-enters the algorithm per line. Justify and align use simple integer division: `SpaceBetween` puts the leftover between children, `SpaceAround` puts it on both sides, `Center` halves it.

7.6.1 Padding, border, margin

Three rectangles concentric:

margin	cells *outside* the box (sibling spacing)
border	one cell thick if present (drawn glyphs)
padding	cells *inside* the border (content inset)
content	where children are laid out

The available width for children is `w - left_pad - right_pad - border_l - border_r`. The available height is the same on the vertical. maya's renderer paints the border in the one-cell ring around the padded content.

7.7 Worked example: a status panel

```
auto panel = v(
    t<"Status"> | Bold,
    sep,
    h(t<"CPU:"> | Dim, space, t<"42%"> | Bold) | gap_<1>,
    h(t<"Mem:"> | Dim, space, t<"8.2G"> | Bold) | gap_<1>
) | border_<Round> | pad<1, 2> | gap_<1>;
```

Suppose the parent box gives us 24 cells wide. The layout proceeds:

1. Outer box: border (1 cell each side) + padding (2 on each side) leaves 24 - 2 - 4 = **18 cells** of inner width.
2. Inner column has four children: title (1 row), separator (1 row), CPU line (1 row), Mem line (1 row), with gap 1 between them. Total height 4 + 3 = 7 rows of content. Plus the border pads (1 + 1) and padding (1 + 1) makes the outer box 11 rows.
3. Inside each `h(...)` line, the children are: a static label, a `space` (grow=1), and a value text. Total fixed width: maybe 4 + 3 + gap 1 = 8 cells. Leftover 10 goes entirely to the spacer. The value text is pushed to column 8 + 10 = 18, i.e. the right edge.

The renderer then walks the resulting rectangles and writes glyphs.

7.8 When layout disagrees with you

Two situations confuse newcomers:

- **Nothing happens when I set `grow`.** The parent must have a fixed (or grow-controlled) size for the leftover space to be meaningful. A `grow=1` child of a parent that sizes itself to its content has nothing to grow into.
- **Text wraps inside a column unexpectedly.** A `TextElement`'s width is its content. If you put a long string inside a column, it tries to take its natural width; the parent box constrains it to its own width, which causes wrap. Set `TextWrap::Truncate` or `Ellipsize` if that is wrong.

7.9 Where the code lives

maya/include/maya/layout/yoga.hpp and *maya/src/layout/yoga.cpp*. Despite the name, this is not a binding to Facebook's Yoga library — it is an independent implementation that follows the same flexbox spec. "Yoga" here is a nickname for the algorithm. The implementation is around 800 lines and has no external dependencies.

Recap

Layout decides where things go. maya uses an integer-cell flexbox solver over a flat `vector<LayoutNode>` addressed by index. Three phases: measure, distribute, place. The same model handles rows, columns, wrapping, alignment, justification, and nested boxes.

Workshop

1. Write a row that displays your name on the left and the date on the right, with the space between them growing to fill the terminal width.
2. Build a 3-column dashboard: three boxes, each with `grow=1`, separated by 1-cell gaps. Resize your terminal — watch the boxes split the new width.
3. Read *maya/src/layout/yoga.cpp*. Identify the three phases (measure, distribute, place). Note where the available width is subdivided.

Part III

Rendering

The canvas and packed cells

After layout decides where things go, the renderer needs a place to paint them. That place is the **canvas**: a 2-D grid of cells representing the terminal screen. maya's canvas is unusual in two ways: each cell is a 64-bit packed integer (not a struct), and the canvas is double-buffered (front and back) to support the SIMD diff in the next chapter.

8.1 What a cell is

A terminal cell holds:

- A Unicode code point (up to 21 bits).
- A foreground colour, a background colour, and a set of text attributes.
- Optionally, a hyperlink (an OSC-8 URL).
- A width marker: 1 for normal characters, 2 for wide CJK and emoji, 0 for combining marks and the trailing half of a wide cell.

A naive struct would look like:

```
struct Cell {
    char32_t cp;        // 4 bytes
    Color    fg;        // 4 bytes
    Color    bg;        // 4 bytes
    uint8_t  attrs;     // 1 byte
    uint8_t  width;     // 1 byte
    uint32_t link;      // 4 bytes
};  // ~18 bytes, padded to 24
```

Two problems. First, that is a lot of memory for an 80x24 grid: 1920 cells × 24 bytes = 46 KiB, and we need two grids (double-buffered) = 92 KiB. Second, and more importantly, comparing two cells means comparing several fields, which is several instructions and several branches.

8.2 Packing

maya packs every cell into a single 64-bit integer:

```
//  63                                                          0
// +---------+-------+----------+-----+---------------------------+
// | width:2 | tag:2 | hyper:14 | sty:16 |     codepoint:30        |
// +---------+-------+----------+--------+-------------------------+
```

The exact bit layout is in *maya/include/maya/render/canvas.hpp*; the field names and widths above are illustrative. What matters is the technique:

- **Code point** occupies the low bits (Unicode is only 21 bits, so 30 is generous).

- **Style ID** is a 16-bit index into a `StylePool`. The pool interns full `Style` structs (fg, bg, attrs); the canvas just stores the ID.
- **Hyperlink ID** likewise indexes a hyperlink pool.
- **Width marker** and a small tag fill the top.

Two consequences. First, the canvas shrinks to one quarter of the naive representation. Second, and crucially: **cell equality is `uint64_t` equality**. One machine instruction. No branches. This is the lever `maya` uses to diff thousands of cells per microsecond in the next chapter.

Idea

Whenever you find yourself comparing two structs in a hot loop, ask whether you can pack them into a single integer. SIMD becomes possible the moment your unit of comparison fits in a register lane.

8.3 The style pool

The 16-bit style ID is what makes packing work. There are typically only a few dozen distinct styles on screen at any time (text colour, background, bold, dim, etc.) — nowhere near the 65536 the ID supports. The pool is a simple table of `Style` structs plus a hash map from `Style` to ID:

```
class StylePool {
public:
    StyleId intern(Style s) {
        auto [it, inserted] = lookup_.try_emplace(s, table_.size());
        if (inserted) {
            table_.push_back(s);
            cache_sgr_.push_back(build_sgr_string(s));
        }
        return it->second;
    }

    const Style&        style(StyleId id) const { return table_[id]; }
    const std::string&  sgr  (StyleId id) const { return cache_sgr_[id]; }

private:
    std::vector<Style>      table_;
    std::vector<std::string> cache_sgr_;     // pre-built SGR per style
    std::unordered_map<Style, StyleId> lookup_;
};
```

Two clever bits:

- **Pre-cached SGR.** The first time a `Style` is interned, the pool builds the ANSI byte string for it (`ESC[1;38;2;100;180;255m` or whatever the style demands). During the render walk, switching style is one `memcpy` of the cached SGR — no formatting, no field comparisons.
- **Stable IDs across frames.** The pool persists across frames. Frame 2's cells use the same IDs as Frame 1's for the same styles, so cell-level equality (which depends on the ID) still works.

8.4 The canvas data structure

A canvas is two of these grids:

```
class Canvas {
    int width_  = 0;
    int height_ = 0;

    AlignedBuffer front_;   // what's currently on the terminal
    AlignedBuffer back_;    // what we're drawing next

    StylePool    styles_;
    HyperlinkPool links_;

    Rect         damage_;   // dirty region for the diff
    std::vector<Rect> clip_stack_;  // overflow:hidden support
public:
    void clear();
    void set(int x, int y, char32_t cp, StyleId sty);
    void draw_text(int x, int y, std::string_view utf8, StyleId sty);
    void flip();   // swap front and back after diff
    // ...
};
```

`front_` is what the user is looking at. `back_` is what the next frame should look like. Each frame:

1. Clear `back_` (zero-fill, AVX-512 `vmovdqu64` when available).
2. Walk the element tree; for each visible cell, call `back_.set(x, y, cp, sty)`.
3. Diff `front_` against `back_`; emit ANSI for the changed cells.
4. Swap roles — `back_` becomes `front_`.

This is the classic *double-buffered* pattern.

8.4.1 Why 64-byte alignment?

The `AlignedBuffer` class allocates cells aligned to 64-byte boundaries (a full CPU cache line). The diff loops in the next chapter load 256 bits at a time (AVX2) or 512 bits (AVX-512); unaligned loads cost cycles when they cross a cache line. Aligning to 64 bytes guarantees the SIMD loads never split a line.

8.5 Drawing text

The text-drawing path looks innocuous but does Unicode work:

```
void Canvas::draw_text(int x, int y, std::string_view utf8, StyleId sty) {
    int cx = x;
    for (char32_t cp : utf8_decode(utf8)) {
        int w = unicode_width(cp);
        if (w == 0) {
            // combining mark - attach to previous cell (skipped for brevity)
        } else if (w == 1) {
            set(cx, y, cp, sty);
```

```
            cx += 1;
        } else if (w == 2) {
            set(cx,     y, cp, sty, /*width=*/2);
            set(cx + 1, y, 0,  sty, /*width=*/0);  // trailing half
            cx += 2;
        }
    }
}
```

The wide-character handling is what keeps CJK and emoji from corrupting the layout. The lookup table at *maya/include/maya/text/unicode_width_table.hpp* is generated from the Unicode standard's `EastAsianWidth.txt` and `emoji-data.txt` by the script *maya/scripts/gen_unicode_width.py*.

8.6 The damage rectangle

To avoid diffing the entire canvas every frame, maya tracks the bounding box of any cell that has been written this frame:

```
void Canvas::set(int x, int y, char32_t cp, StyleId sty) {
    back_[y * width_ + x] = pack(cp, sty);
    damage_ = damage_ | Rect{x, y, 1, 1};   // union the rectangle
}
```

The diff in the next chapter visits only the cells inside `damage_`. If most of the screen is unchanged (the typical case for steady-state UIs), the diff is fast in proportion to what moved, not in proportion to the screen size.

8.6.1 The clip stack

A box with `overflow: Hidden` (a scrollable list, a modal) needs its rendering clipped to its rectangle. maya pushes the clip rectangle onto a stack and intersects every `set()` with the current top:

```
void Canvas::push_clip(Rect r) { clip_stack_.push_back(intersect(top(), r)); }
void Canvas::pop_clip()        { clip_stack_.pop_back(); }
```

Children that try to draw outside the clip simply have their writes silently dropped.

8.7 Resize

When `SIGWINCH` fires, the runtime queries the new size and calls `canvas.resize(w, h)`. The buffers reallocate (preserving old contents is unnecessary — the next frame redraws everything), the damage rect is set to the full new screen, and the next paint emits ANSI for the whole grid.

8.7.1 Shrinking after a wide window

A neat detail: if the user grows the terminal to 200 cells wide, runs the app, then shrinks back to 80 cells, the AlignedBuffer would otherwise stay at the wide capacity forever. maya ships `shrink_to_fit_if_oversized` that releases capacity when at most half of it is in use, to avoid permanent over-allocation but also to avoid thrashing the allocator on tiny oscillations.

8.8 Putting it together

A frame, end to end, in pseudocode:

```
void Renderer::frame(const Element& root) {
    canvas.clear_back();
    layout::compute(layout_tree, /*root=*/0, canvas.width());
    paint_visit(canvas, layout_tree, /*node=*/0);   // fills back_

    std::string ansi;
    diff(canvas.front(), canvas.back(),
         canvas.damage(), canvas.styles(), ansi);   // SIMD-accelerated
    writer.write(ansi);

    canvas.flip();
}
```

The chapter has set up everything but the diff. That is next.

Recap

The canvas is a double-buffered grid of packed 64-bit cells. Packing makes cell equality a single integer comparison; the style pool interns full `Style` structs into 16-bit IDs. A damage rectangle limits the next chapter's diff to the changed region. Wide characters and clip regions are handled in the same data structure.

Workshop

1. Open *maya/include/maya/render/canvas.hpp* and find the `pack()` / `unpack()` helpers. Note the bit layout.

2. Run an example with wide characters (*examples/messenger.cpp*). Notice how alignment stays correct when the text contains emoji.

3. Estimate, on the back of an envelope, the byte cost of an 80x24 canvas (1920 cells), then double it for the two buffers, then add typical pool sizes. You should land near 32 KiB — well under L1 cache.

SIMD frame diffing

Now that we have two grids of packed cells — the front buffer (what the terminal is showing) and the back buffer (what we want it to show) — we have to compute, quickly, where they differ and emit the minimal sequence of ANSI bytes that transforms one into the other. This chapter is the most performance-sensitive corner of maya, and it is where SIMD earns its keep.

9.1 Why SIMD?

A modern terminal frame is something like 200 columns by 50 rows — 10,000 cells. At 60 frames per second that is 600,000 cells per second, and almost all of them are unchanged between consecutive frames. The naive cell-by-cell comparison is:

```
for (int i = 0; i < width * height; ++i) {
    if (front[i] != back[i]) {
        emit_change(i, back[i]);
    }
}
```

One `cmp` instruction, one branch, one increment, per cell. The branches are unpredictable (changes are sparse and clustered), so the CPU's branch predictor mispredicts often. Cache loads dominate.

SIMD (Single Instruction Multiple Data) lets you compare multiple cells per instruction. A single AVX2 instruction operates on a 256-bit register, which is *four* 64-bit cells. AVX-512 doubles that to eight. NEON, on ARM, gives you two 64-bit lanes per instruction.

Theory

SIMD instructions live on dedicated execution units alongside the scalar ALU. They don't run faster — they run *wider*. The throughput win comes from doing more work per instruction without adding more branches.

9.2 The algorithm

maya's diff loop has a beautifully simple shape:

1. Iterate row by row within the damage rectangle.
2. Within a row, use SIMD to skip runs of equal cells four (AVX2) or eight (AVX-512) at a time.
3. When a difference is found, find the first changed cell inside the SIMD batch using a bit-scan.
4. Emit the cursor move, the style change (if any), the character bytes.
5. Continue scanning for the run of *changed* cells until they become equal again; then resume bulk-skip.

The hot inner loop is:

```
[[nodiscard]] static std::size_t find_first_diff(
    const uint64_t* a, const uint64_t* b, std::size_t count) noexcept
{
    std::size_t i = 0;
    for (; i + 4 <= count; i += 4) {
        __m256i va  = _mm256_loadu_si256((const __m256i*)(a + i));
        __m256i vb  = _mm256_loadu_si256((const __m256i*)(b + i));
        __m256i cmp = _mm256_cmpeq_epi64(va, vb);
        unsigned mask = _mm256_movemask_pd(_mm256_castsi256_pd(cmp));
        if (mask != 0xFu) {
            return i + std::countr_zero((unsigned)(~mask & 0xFu));
        }
    }
    for (; i < count; ++i) if (a[i] != b[i]) return i;   // scalar tail
    return count;
}
```

Read it piece by piece.

9.2.1 Loading four cells

`_mm256_loadu_si256` loads 256 bits (four 64-bit cells) from memory into a register. The `u` suffix means "unaligned" — it tolerates any alignment, with a small performance cost compared to the aligned variant. maya's buffer is 64-byte aligned, so in practice we always hit the fast path.

9.2.2 Comparing them

`_mm256_cmpeq_epi64` performs four 64-bit integer equality checks in parallel. The result is a register where each lane is `0xFFFFFFFFFFFFFFFF` (all ones) if the lanes were equal, `0` otherwise.

9.2.3 Reducing to a mask

`_mm256_movemask_pd` extracts the top bit of each 64-bit lane into a 4-bit integer. (We cast to `pd` (packed double) because `movemask` on 64-bit lanes uses the floating-point variant of the instruction — same idea, different mnemonic.) Now `mask` is a 4-bit number where bit k is set if lane k was equal.

9.2.4 Detecting any change

If all four cells were equal, `mask == 0xF`. If even one differs, `mask != 0xF`, and we have to find which.

9.2.5 Finding the first changed lane

`~mask & 0xF` flips the bits so that "not equal" is now 1. `std::countr_zero` (count trailing zeros) returns the index of the lowest set bit. That is the first changed cell within the batch.

9.2.6 The scalar tail

The SIMD loop steps by 4. If `count` is not a multiple of 4, the last few cells are checked one at a time. This tail is essential — forgetting it is the classic SIMD bug.

9.3 Architecture dispatch

Different CPUs support different SIMD widths. maya provides implementations for:

- **AVX-512.** 8 cells per iteration. Modern Intel server chips, recent Xeon, some Ryzen.
- **AVX2.** 4 cells. Haswell (2013) and newer Intel, Zen 1+ AMD — basically every desktop and laptop in current use.
- **SSE2.** 2 cells. Universal x86-64 baseline.
- **NEON.** 2 cells per iteration. ARM (Apple Silicon, Cortex-A series, AArch64 servers).
- **Scalar.** 1 cell. The portable fallback.

The dispatch is done at compile time via the `maya::simd::Ops` template, specialised per ISA tag:

```
struct Avx2 {};
struct Sse2 {};
struct Neon {};
struct Scalar {};

template <typename ISA>
struct Ops {
    static std::size_t find_first_diff(const uint64_t*, const uint64_t*,
                                       std::size_t) noexcept;
    static void streaming_fill(uint64_t*, uint64_t, std::size_t) noexcept;
    // ...
};

template <> struct Ops<Avx2> { /* AVX2 implementation */ };
template <> struct Ops<Sse2> { /* SSE2 implementation */ };
// ...
```

The renderer picks the highest-supported ISA detected at build time (via `#if defined(__AVX2__)` etc.) and uses it everywhere. At runtime, maya also offers an optional dispatch that selects the best ISA *on the current CPU* via `cpuid` probing — so a binary compiled with AVX2 baseline can still light up AVX-512 paths if the user runs it on a chip that has them.

Idea

The portable-with-fallback pattern: write the operation once for each SIMD width, expose the same interface, pick the best at build or runtime. The architecture-specific code is small and centralised; the rest of the codebase just calls `Ops<ISA>::find_first_diff`.

9.4 The other SIMD operations

`find_first_diff` is the showpiece, but maya's SIMD module has a few more:

- `skip_equal(a, b, start, end)` — bulk-skip equal cells inside a row. Same shape as `find_first_diff` but starts at an offset.
- `streaming_fill(dst, value, count)` — fill a region with a constant cell (e.g. for clearing). On AVX-512 uses non-temporal stores that bypass the cache — useful for clears, because the data won't be read again before being overwritten.
- `bulk_eq(a, b, count)` — whole-row equality, returns a bool. Used to check whether an entire row is unchanged, in which case the row is skipped in one go.

9.5 Putting it in the renderer

The main render loop walks each row of the damage rectangle:

```
for (int y = damage.top; y < damage.bottom; ++y) {
    const uint64_t* front_row = front + y * width;
    const uint64_t* back_row  = back  + y * width;

    std::size_t x = damage.left;
    while (x < damage.right) {
        // Skip equal cells with SIMD.
        x = Ops<ISA>::skip_equal(front_row, back_row, x, damage.right);
        if (x >= damage.right) break;

        // Emit ANSI for the run of changed cells starting at x.
        std::size_t run_end = find_run_end(front_row, back_row, x, damage.right);
        emit_run(out, x, y, back_row + x, run_end - x);
        x = run_end;
    }
}
```

The `emit_run` call is where ANSI bytes are written: cursor positioning, style change (consulting the StylePool's pre-cached SGR), UTF-8 encoding of the code points. Chapter 10 covers that side.

9.6 What about diff algorithms (Myers, etc.)?

In version-control land, "diff" usually means a clever longest-common-subsequence algorithm. maya's diff is not that. It is line-by-line (or rather cell-by-cell) literal comparison. The reason: terminal rendering does not benefit from finding "moved" content — moving a row in the terminal still requires writing the new bytes, because the terminal has no concept of moving a row. The wall-clock-fastest diff is the one that finds the changed cells in the most cache-friendly order, which is exactly what packed cells plus SIMD give us.

9.7 Validation

A subtle bug in any of this would be hard to spot — the screen might show a smear of stale glyphs or a corrupted style for a frame. maya validates the diff in two ways:

- Unit tests in *maya/tests/test_diff.cpp* compare SIMD output to a known-correct scalar implementation on thousands of random inputs.

- In debug builds, the canvas can be configured to maintain a "witness chain" — a running hash of the bytes emitted, which is later replayed in a virtual terminal to confirm that the final state matches the back buffer. We will see this in Chapter 11.

Recap

The frame differ is maya's performance heart. It compares the front and back buffers via SIMD — four cells per AVX2 instruction, eight per AVX-512 — bulk-skips equal runs, and emits ANSI only for the cells that changed. The pattern (compare, movemask, ctz, scalar tail) is the canonical SIMD idiom for "find first X in a stream of fixed-size records."

Workshop

1. Build and run *examples/fps.cpp*. It does a 3D raycaster in the terminal. The renderer pushes a fresh frame every ∼16 ms. Watch your CPU usage — it should be low.

2. Read *maya/include/maya/platform/simd/avx2.hpp*. Find the loop that uses `_mm256_cmpeq_epi64`. Identify each of the five steps in this chapter.

3. Write the same `find_first_diff` in scalar code and benchmark both on a million 64-bit integers (random, occasionally differing). On a recent x86, the SIMD path should be 3-5x faster.

ANSI serialization and the writer

The diff in the previous chapter tells us *which cells changed and what they should be*. This chapter is about the last mile: turning that information into the exact byte stream the terminal expects, and getting those bytes out to the wire as cheaply as possible.

10.1 What the renderer must emit

For every run of changed cells, the renderer must produce:

1. A **cursor positioning** sequence to move to the start of the run.
2. A **style change** (SGR) sequence if the new style differs from the cursor's current style.
3. The **UTF-8 bytes** of the cells' code points.
4. Optionally, **hyperlink** OSC-8 wrap if a link is associated with the cells.

For a typical animation frame at, say, 1% changed cells on an 80x24 grid, that is around 20 runs, each maybe 4 cells long, each requiring a CUP (8-10 bytes), maybe an SGR (15-30 bytes for full truecolour), and the UTF-8 bytes. Total: about a kilobyte per frame. That kilobyte must be assembled and written in well under one frame (∼16 ms) to keep up with 60 FPS.

10.2 Zero-allocation building blocks

The old version of maya's renderer built a `std::vector<RenderOp>` and then serialised it — which allocated for every entry. The current version writes ANSI bytes directly into one growing `std::string`. Every per-cell allocation was eliminated.

The lowest layer is *maya/include/maya/terminal/ansi.hpp*, a collection of free functions that append ANSI sequences to a caller-supplied string:

```
MAYA_FORCEINLINE void write_cursor_up(std::string& out, int n);
MAYA_FORCEINLINE void write_cursor_down(std::string& out, int n);
MAYA_FORCEINLINE void write_move_to(std::string& out, int col, int row);
MAYA_FORCEINLINE void write_erase_lines(std::string& out, int n);
```

Each one is a few `out += ...` calls and a single `std::to_chars` for the integer parameter. `std::to_chars` is C++17's allocation-free, locale-free integer formatter; it writes into a stack buffer that we then `append` onto `out`.

10.2.1 A worked function

```
namespace detail {
    MAYA_FORCEINLINE void append_int(std::string& out, int n) {
        char buf[12];                              // 2^31 fits in 11 chars + sign
```

```
        auto [end, _] = std::to_chars(buf, buf + sizeof(buf), n);
        out.append(buf, end);
    }
}

MAYA_FORCEINLINE void write_move_to(std::string& out, int col, int row) {
    out += "\x1b[";
    detail::append_int(out, row);
    out += ';';
    detail::append_int(out, col);
    out += 'H';
}
```

No heap allocation. No `std::format`. `out += "\x1b["` appends a `string_view` which is two characters; the integers add 1–3 bytes; the totals are usually 8-10 bytes per CUP. `MAYA_FORCEINLINE` is a portable wrapper for `__attribute__((always_inline))` (GCC/Clang) and `__forceinline` (MSVC) — a hint that this should be inlined even at low optimisation levels.

10.2.2 Per-cell positioning, faster

The diff loop's hot path bypasses even `std::to_chars` for the common case of row and column numbers under 1000, using a hand-written ASCII digit splitter:

```
MAYA_FORCEINLINE char* write_uint_pos(char* p, unsigned n) noexcept {
    if (n < 10) {
        *p++ = char('0' + n);
    } else if (n < 100) {
        *p++ = char('0' + n / 10);
        *p++ = char('0' + n % 10);
    } else if (n < 1000) {
        *p++ = char('0' + n / 100);
        *p++ = char('0' + (n / 10) % 10);
        *p++ = char('0' + n % 10);
    } else {
        // fallback: std::to_chars
    }
    return p;
}
```

Inlined into the diff loop, this compiles to a tiny chain of conditional moves and stores — a few cycles per integer. Premature optimisation? Not in a loop that runs once per changed cell.

10.3 UTF-8 encoding

For every changed cell we need to encode its 32-bit code point into 1–4 UTF-8 bytes. maya's encoder is in `render/diff.hpp`:

```
MAYA_FORCEINLINE void encode_utf8(char32_t cp, std::string& out) {
    if (cp < 0x80) [[likely]] {                    // ASCII fast path
        out += char(cp);
        return;
    }
    char buf[4];
    int len;
    if (cp < 0x800) {                              // 2-byte
        buf[0] = char(0xC0 | (cp >> 6));
        buf[1] = char(0x80 | (cp & 0x3F));
        len = 2;
```

```
    } else if (cp < 0x10000) [[likely]] {            // 3-byte (BMP)
        buf[0] = char(0xE0 |  (cp >> 12));
        buf[1] = char(0x80 | ((cp >> 6) & 0x3F));
        buf[2] = char(0x80 |  (cp        & 0x3F));
        len = 3;
    } else {                                         // 4-byte (supplementary)
        buf[0] = char(0xF0 |  (cp >> 18));
        buf[1] = char(0x80 | ((cp >> 12) & 0x3F));
        buf[2] = char(0x80 | ((cp >>  6) & 0x3F));
        buf[3] = char(0x80 |  (cp        & 0x3F));
        len = 4;
    }
    out.append(buf, len);
}
```

The `[[likely]]` attribute tells the compiler to lay out the branches with the ASCII and BMP cases on the hot path; the rare 4-byte case takes the cold branch.

10.4 The style cache

A style change emits an SGR sequence whose exact bytes depend on the `Style` struct: `ESC[0;1;3;38;2;100;180;` for "reset, bold, italic, fg=(100,180,255), bg=(20,25,35)". Building that string takes a dozen `to_chars` calls and several conditional branches.

Doing this once per changed cell would dominate runtime. maya does it *once per distinct style, ever* — when the style is interned into the `StylePool` — and caches the bytes:

```
StyleId StylePool::intern(Style s) {
    auto [it, inserted] = lookup_.try_emplace(s, table_.size());
    if (inserted) {
        table_.push_back(s);
        cache_sgr_.push_back(build_sgr_string(s));   // do it once
    }
    return it->second;
}
```

The diff loop's style change is then a single `memcpy`:

```
if (new_style_id != current_style_id) {
    out += pool.sgr(new_style_id);
    current_style_id = new_style_id;
}
```

No formatting. No branches. The bytes are already laid out.

10.5 The terminal writer

After the renderer assembles its kilobyte of ANSI, the runtime needs to put those bytes on the terminal file descriptor. The writer (*maya/include/maya/terminal/writer.hpp*) wraps that:

```
class Writer {
public:
    explicit Writer(int fd) : fd_(fd) {}
    void write(std::string_view bytes);
    void flush();
private:
    int         fd_;
    std::string buffer_;
```

```
};
```

Three details worth noting:

- **Buffered.** Multiple `write()` calls accumulate into `buffer_`; one `::write(fd, ...)` system call sends them all. System calls are expensive; one big write beats many small writes.
- **Partial-write loop.** A blocking `::write` may return having sent fewer bytes than requested. The writer loops until everything is out (or an error pins it).
- **Single-frame atomicity.** The whole frame is one contiguous string. Even if the terminal is slow, the user never sees a half-rendered frame — the writer hands the whole buffer to the kernel in one go.

10.5.1 Synchronised output mode

A subtle problem: even with one `write`, the terminal can paint the bytes incrementally, producing a brief flash of the half-applied frame (a phenomenon called *tearing*). Modern emulators support "synchronised output mode" — a pair of DCS escape sequences that tell the emulator "hold off painting until I say I am done." maya wraps each frame in these:

```
ESC P = 1 s ESC \    // begin synchronised update
... frame bytes ...
ESC P = 2 s ESC \    // end synchronised update; paint atomically
```

Emulators that do not understand the sequence ignore it; those that do paint the frame as one unit. Result: zero tearing on supporting terminals, no regression on others.

10.6 Putting it all together

Here is the simplified inner loop of the renderer's `emit_run`:

```
void emit_run(std::string& out, int x, int y,
              const uint64_t* cells, std::size_t count,
              const StylePool& pool,
              StyleId& current_style)
{
    // 1. Cursor positioning (1-based in ANSI; +1 to both).
    out += "\x1b[";
    char buf[16];
    char* p = buf;
    p = write_uint_pos(p, y + 1);
    *p++ = ';';
    p = write_uint_pos(p, x + 1);
    *p++ = 'H';
    out.append(buf, p - buf);

    // 2. Cells.
    for (std::size_t i = 0; i < count; ++i) {
        auto [cp, sty, width] = unpack(cells[i]);
        if (sty != current_style) {
            out += pool.sgr(sty);
            current_style = sty;
        }
        if (width == 0) continue;       // trailing half of a wide cell
        encode_utf8(cp, out);
```

```
    }
}
```

Notice the discipline: every operation is either a `string_view` append, a `memcpy`, or a few stack-buffer writes followed by an append. There are no allocations inside the loop.

10.7 Cost, measured

On a modern x86 laptop, with AVX2 and a 200x50 terminal, maya's render loop — including layout, paint, diff, and serialise — typically takes *30-80 microseconds* per frame in steady state. At 60 FPS the per-frame budget is 16,666 microseconds; we are using under 1%. The remaining headroom is the terminal emulator's business.

Recap

The renderer turns the diff into ANSI bytes via a chain of allocation-free helpers: stack-buffered integer formatting, inline-friendly UTF-8 encoding, and a per-style pre-cached SGR string. The writer batches the frame into one `::write()` and optionally wraps it in synchronised-output mode for tear-free rendering. The hot path is dominated by `memcpy`; the allocation count is zero.

Workshop

1. Read *maya/include/maya/terminal/ansi.hpp*. Identify every `constexpr string_view` constant and match it to an ANSI sequence from Chapter 2.
2. In a scratch program, construct a `Canvas`, write some text into it with `draw_text`, and dump the diff bytes against an empty front buffer. Inspect the bytes by hand.
3. Toggle the synchronised-output sequences off in a debug fork of `maya` and run an animation example. On a fast terminal you will see frames tear; on a slow one (try `ssh localhost` into the same machine) the effect is more obvious.

Inline mode and the witness chain

Most TUI libraries take over the entire terminal: they switch to the **alternate screen**, paint, restore the original screen on exit. maya supports that — it is the `Fullscreen` rendering mode. But maya also supports something rarer and far more delicate: **inline mode**, where the UI lives *at the bottom of your normal scrollback*, redrawing in place but never destroying what is above it.

Inline mode is what makes agentty feel different from a "classic" TUI: when you type `agentty` in your shell, the chat UI appears below your prompt without taking over the terminal, and when it exits, your shell is exactly where you left it — with the conversation now permanent scrollback above the cursor. That behaviour is, in principle, one wrong escape sequence away from permanently corrupting the user's terminal history.

This chapter is a careful walk through how maya guarantees it *doesn't*. We will look at the failure modes that real bugs in the renderer produced (with citations), the four-part theorem maya commits to (**Scrollback Integrity**), and the five layers of type-system machinery that close each leak in the theorem into a compile error.

11.1 The two modes, briefly

```
enum class Mode {
    Inline,      // raw mode, no alt screen, scrollback preserved
    Fullscreen,  // alt screen buffer, double-buffered cell diff
};
```

Fullscreen mode writes `ESC[?1049h` on startup ("enter alt screen"), paints absolute-positioned cells, writes `ESC[?1049l` on exit ("leave alt screen"). The user's scrollback is bytes the renderer never touches. Easy.

Inline mode stays on the normal screen. The cursor starts wherever the shell left it; the UI occupies some number of rows *below the cursor*; when the program quits, those rows remain (or are erased and replaced). On every frame, the renderer must re-paint the same region without ever writing a byte that affects the rows above. The shell's history, your previous commands, the output of `ls` you ran two minutes ago — all immutable.

The naive question: what could go wrong? Two answers, both with real bug reports in the repository.

11.2 Failure mode 1: ghosting from over-tall paint

The first real bug, documented in *maya/docs/internals/inline-autogrow.md*, came from trying to optimise the renderer. The legacy code pattern was: paint into the existing canvas, measure how tall the content turned out, resize and paint again if it overflowed. Two paint passes when content was big; the first one always discarded.

The optimisation seemed obvious: ask the layout engine to compute the natural height *first*

(with an effectively infinite height constraint), resize, then paint once.

```
constexpr int kBigH = 1 << 20;        // "infinite"
build_layout_tree(root, layout_nodes, theme);
layout::compute(layout_nodes, root_idx, w, kBigH);
const int natural_h = layout_nodes[root_idx].computed.size.height.raw();
if (canvas.height() < natural_h + 8) canvas.resize(w, natural_h + 8);
paint_element(root, canvas, pool, layout_nodes, root_idx, 0, 0);
```

One paint, correctly-sized canvas, no out-of-bounds window for the component cache to capture from. Everything looks fine.

It ghosted. Interactive sessions started showing rows duplicated and stacked into scrollback.

11.2.1 Why

The layout engine's `height` parameter is the *available height* for the root container. That value flows down to children as their containing block. A child with `flex-grow > 0` or `align-items: stretch` distributes a share of that available height. Under `kBigH`, "remaining height" is pathologically large: elastic children claim huge vertical extents that they would never have claimed under a realistic constraint.

So the painted frame was a 50-row layout on a 24-row terminal.

11.2.2 The downstream catastrophe

After paint, `InlineFrameState::prev_rows` records the painted height (50). The next frame calls `compose_inline_frame`, which positions the cursor relative to where it left off:

```
const int cursor_row_start = prev_rows - 1;            // 49
const int delta = first_changed - cursor_row_start;    // e.g. -45
if (delta < 0) {
    int up = std::min(-delta, prev_on_screen - 1);     // capped at 23
    if (up > 0) ansi::write_cursor_up(out, up);
    out += '\r';
}
```

`prev_on_screen = min(prev_rows, term_h) = 24`. The `prev_on_screen - 1` cap is essential — without it, the cursor would walk *up into scrollback-committed history*, overwriting the user's earlier output. With it, when we genuinely need to move 45 rows up to reach `first_changed`, we move only 23. The cursor lands at the wrong physical line.

The per-row emit loop then proceeds as if the cursor were at `first_changed`:

```
for (int y = first_changed; y <= last_row_to_visit; ++y) {
    if (y > first_changed) out += "\r\n";
    emit_row_diff(y, ...);
}
```

Every row gets written 22 lines too far down. Old content above stays visible; new content overwrites whatever was previously at those positions. When the content next shrinks back to a normal size, the cycle repeats with a different misalignment — and rows that have already been pushed past the bottom into native scrollback are gone, permanently.

11.2.3 The fix

Keep the two-compute pattern, but skip the wasted first paint:

```
build_layout_tree(root, layout_nodes, theme);
layout::compute(layout_nodes, root_idx, w, canvas.height());   // realistic budget
const int needed_h = layout_nodes[root_idx].computed.size.height.raw();
if (canvas.height() < needed_h + 8) {
    canvas.resize(w, needed_h + 8);
    layout::compute(layout_nodes, root_idx, w, canvas.height());  // recompute
}
paint_element(root, canvas, pool, layout_nodes, root_idx, 0, 0);  // single paint
```

The recompute is what produces the painted layout, with the resized height as the constraint — byte-for-byte equivalent to legacy's second `render_tree` call, but with the first paint elided.

The doc concludes with the **invariant for future readers**:

> *The layout that gets painted must come from a `layout::compute` call whose height parameter is at least as large as the canvas about to receive the paint.*

That sentence is, in the larger sense, an unenforced human discipline. A future contributor could "simplify" the code right back into the broken shape, and CI would not catch it. We will return to this kind of fragility in a moment — it is why the witness chain exists.

11.3 Failure mode 2: `commit_prefix` over-claim

The second bug class, documented in *scrollback-corruption-audit.md*, is more insidious. It begins with a feature: a long-running chat UI that streams turns should eventually *commit* old turns to scrollback, so that the live frame stays small and fresh content keeps painting at the bottom.

The Cmd that does this is `Cmd::commit_scrollback(int rows)`. The handler, in the legacy implementation, simply decremented `prev_rows` and `memmove`-d `prev_cells` up:

```
// src/render/serialize.cpp (legacy)
prev_rows -= rows;
// (no bytes written, no escape emitted, no verification)
```

The implicit contract: "`rows` is the count of rows that the terminal *already scrolled out of the viewport on its own*, because we emitted more `\r\n`s than the terminal could fit."

11.3.1 Why a guessed row count corrupts everything

agentty's `maybe_virtualize` called

```
return Cmd<Msg>::commit_scrollback(kSliceChunk * kRowsPerDroppedMessageLower);
```

The arithmetic was a *guess* at the rendered height of dropped messages, not a count of physical viewport scrolls. Concrete scenario from the audit:

- 50-row terminal, 8 turns at ~3 rows each: `prev_rows` $\approx$ 24.
- All 24 rows are on screen; zero rows have scrolled.
- `commit_inline_prefix(8)` runs: `prev_rows` drops from 24 to 16; `prev_cells` is shifted up by 8.
- Physical viewport: unchanged. The renderer's model and reality now disagree by 8 rows.

Next compose:

- `cursor_row_start = 16 - 1 = 15`.
- Physical cursor: actually at row 23.
- Every emit lands 8 rows below where the renderer thinks.
- Later, when `\r\n` finally scrolls the viewport, the rows that scroll into native scrollback are **the stale ghost rows** — the renderer's incorrect model, frozen into the user's permanent history.

That is the corruption. It is not recoverable: bytes already committed to native scrollback are out of the renderer's reach.

11.3.2 The structural fix

Two paths, both shipped:

1. **Clamp inside the handler.** The Cmd handler now enforces `min(rows, max(0, prev_rows - term_h))`: a request to commit more rows than have actually overflowed the viewport becomes a no-op rather than a corruption.
2. **Prefer the typed alternative.** `Cmd::commit_scrollback_overflow` takes no row count; it asks the renderer to commit precisely the rows that have provably overflowed (`prev_rows - term_h`). The caller cannot get the number wrong because there is no number.

Path 1 makes the bug class harder to trigger; path 2 makes it impossible. The library now prefers (2).

11.4 The theorem: Scrollback Integrity

After enough of these one-off fixes, the maya authors did something unusual: they wrote down the *theorem* the renderer should satisfy and then engineered the type system so violations become compile errors.

Theory

Scrollback Integrity. For any well-typed maya program P, for any sequence of host events $e_1 \dots e_n$, the bytes the terminal emulator commits to its native scrollback are *exactly* the bytes `serialize` would produce for the overflow-portion of past frames, in order. No row is ever targeted by an escape sequence emitted by maya once it has overflowed the viewport.

The theorem decomposes into four sub-claims, each closed by a different layer of the chain:

- **T0 (canvas-cache integrity).** The `last_content_col(y)` and `max_content_row()` metadata a diff trusts is a true function of the cell buffer.
- **T1 (no-ghost).** No cursor-positioning escape targets a row above the viewport top.

- **T2 (no-corruption).** No cell-paint escape disagrees with `paint(view(model))` at the corresponding canvas coordinate.
- **T3 (monotone scrollback).** Once a row leaves the viewport via `\r\n` overflow, no subsequent render targets it.

T0 closes the "stale tail" ghost class (header shrinks; old tail not cleared). T1 closes the "cursor walks into history" class. T2 closes the "wire and shadow disagree" class. T3 closes the "commit_prefix over-claim" class above.

Together: the bytes in emulator scrollback at any moment are the union of *truths committed so far* and *truth currently on screen*, both byte-identical to what the canvas claims. The emulator's scrollback is what the model says it is. End of theorem.

11.4.1 The honest hypotheses

A theorem with no hypotheses is suspicious. maya's claim holds *only* when four assumptions hold:

- **A1 — Terminal conformance.** The emulator honours ECMA-48 cursor control, DEC 2026 synchronised output if advertised, EL, ED, DECAWM. A buggy emulator that misinterprets `CSI A` is out of scope.
- **A2 — Kernel write fidelity.** `write(2)`'s return value accurately reflects bytes delivered. POSIX guarantee; treat as axiom.
- **A3 — Exclusive fd ownership.** maya's `Writer` is the only entity in the process writing to fd 1. Host code that calls `std::cout` or `fwrite(stdout)` outside `Cmd::print` or `Runtime::with_external_io` bypasses the entire chain. Cannot be enforced by the type system because C++ does not let us seal `stdout`.
- **A4 — No silent memory corruption.** Between `verify_shadow` returning a witness and `compose_inline_frame_v2` consuming it, the `prev_cells` buffer is not mutated by use-after-free, an overflow in adjacent code, or a cosmic ray. An FNV-1a re-hash inside compose catches mutation with probability $1 - 2^{-64}$.

11.5 The witness chain: five layers

Each sub-claim is closed by a distinct piece of type machinery. None of them depends on a comment, a convention, or a code review catching it. The compiler is the proof assistant.

11.5.1 Layer 0 — `CanvasWitness` (T0)

Header: *maya/include/maya/render/canvas_witness.hpp*.

A `Canvas` caches metadata: per-row last-non-blank column (`last_content_col_[y]`) and overall max-content row (`max_y_`). The diff loop reads these to skip blank tails — an enormous optimisation for typical UIs that are mostly empty columns to the right of text.

The risk: if the cache lies (claims a longer tail than the cells actually contain), the diff can re-emit cells that should be blank, producing the classic "stacked header, stale tail" ghost row.

`CanvasWitness` is move-only, has a private constructor, and is friended only to `verify_canvas`. The verifier re-derives the cache from the cells in a slow $O(W \cdot H)$ scan and returns `nullopt` if they disagree. On match it records FNV-1a hashes of both the cell buffer and the `(last_col_, max_y_)` tuple and stamps the witness with them.

```
class CanvasWitness {
    friend std::optional<CanvasWitness> verify_canvas(const Canvas&);
    Canvas* canvas_;
    uint64_t cells_hash_;
    uint64_t cache_hash_;
    CanvasWitness(Canvas* c, uint64_t ch, uint64_t kh)  // private
      : canvas_(c), cells_hash_(ch), cache_hash_(kh) {}
public:
    CanvasWitness(const CanvasWitness&) = delete;
    CanvasWitness(CanvasWitness&&) noexcept = default;
    uint64_t cells_hash() const noexcept { return cells_hash_; }
    uint64_t cache_hash() const noexcept { return cache_hash_; }
};
```

The `diff()` function that consumes the witness re-hashes the moved-in canvas and `std::aborts` on mismatch. The renderer is single-threaded; a mismatch between two consecutive calls signals memory corruption, and there is no recovery path the renderer would trust.

The host's only recovery if `verify_canvas` returns `nullopt` is `clear_row()` plus full repaint — never "emit anyway." The escape hatch does not exist by construction.

11.5.2 Layer 1 — `WireRow` and `Viewport` (T1)

Header: *maya/include/maya/render/wire_row.hpp*.

Cursor coordinates are saturating. A `WireRow` is a wrapper around an `int` whose constructor clamps:

```
class WireRow {
    friend class Viewport;
    int row_;
    WireRow(int row, int top, int bottom)
      : row_(std::clamp(row, top, bottom - 1)) {}
public:
    int raw() const noexcept { return row_; }
};
```

The only public producers:

- `Viewport::for_fresh_frame(term_h)` — a fresh `Viewport` with the live region $[0, \texttt{term\_h})$.
- `Viewport::for_redraw(term_h, wire_cursor_rows)` — the redraw variant, used after a frame has already started.
- `Viewport::row(int y)` — the sole producer that takes an arbitrary absolute row index; clamps via `std::clamp`.
- `WireRow::up(n)` / `down(n)` — arithmetic that saturates inside the same $[top, bottom)$.

`emit_move(out, from, to)` is the sole emitter of cursor-up / cursor-down escapes, and its parameters are typed `WireRow`. There is no public `WireRow(int)` constructor, no `data()`, no escape hatch.

Proof of T1. The set of cursor targets any `emit_move` can produce is precisely the set of `WireRow` values; the set of `WireRow` values is precisely `Viewport::[top, bottom)`. Targeting a row above the viewport top is, by construction, unrepresentable.

This is the same flavour of compile-time proof as agentty's permission policy from earlier: encode the rule as a type, make violations uncompilable. The `kBigH` ghosting story from earlier in this chapter could not happen against this layer — even if the renderer miscomputed the cursor delta, the resulting `WireRow` would have saturated at the viewport top and no escape would have aimed above it.

11.5.3 Layer 2 — `ContentRows` (closes one wrong-source class)

Header: *maya/include/maya/render/serialize.hpp*.

`ContentRows` is an `int`-wrapper with a private constructor. The sole producer is `content_rows(canvas)`, which calls `content_height(canvas)` on the canvas about to be diffed and binds the result to that canvas's pointer.

```
class ContentRows {
    friend ContentRows content_rows(const Canvas&);
    int rows_;
    const Canvas* src_;
    ContentRows(int r, const Canvas* s) : rows_(r), src_(s) {}
public:
    int raw()              const noexcept { return rows_; }
    const Canvas* source() const noexcept { return src_; }
};
```

`compose_inline_frame_v2` consumes `ContentRows`. The historical bug class — computing the row count from the wrong source (the layout's height, a hard-coded constant, a stale `prev_rows` from a different canvas, an agentty caller's guess of how many rows are about to scroll) — becomes *uncompilable*. There is exactly one way to construct a `ContentRows`, and it forces the value to come from the canvas the renderer is about to paint from.

11.5.4 Layer 3 — `ShadowWitness` (T2)

Header: *maya/include/maya/render/serialize.hpp*.

The diff inside `compose_inline_frame_v2` reads `state.prev_cells` — the *shadow buffer*, a mirror of what the renderer believes is currently on the terminal — and emits ANSI for the cells that differ from the new canvas. If `prev_cells` disagrees with reality (use-after-free, an unrelated bug overwriting it, an out-of-bounds write from another subsystem), the diff emits the wrong bytes.

`ShadowWitness` is the move-only token that proves `prev_cells` matches the recorded shadow hash. The sole producer is `verify_shadow(state)`, which re-hashes `state.prev_cells` and returns `nullopt` on disagreement:

```
class ShadowWitness {
    friend std::optional<ShadowWitness> verify_shadow(InlineFrameState&);
    InlineFrameState* state_;
    uint64_t hash_at_issue_;
    ShadowWitness(InlineFrameState* s, uint64_t h)         // private
      : state_(s), hash_at_issue_(h) {}
public:
    ShadowWitness(const ShadowWitness&) = delete;
    ShadowWitness(ShadowWitness&&) noexcept = default;
```

```
    uint64_t hash_at_issue() const noexcept { return hash_at_issue_; }
};
```

`compose_inline_frame_v2` consumes a `ShadowWitness&&`. Inside, it re-hashes the moved-in state and `std::aborts` on mismatch. The window between `verify_shadow` returning a witness and compose re-checking is small — both calls happen on the same thread, in the same function — so under hypothesis A4 (no silent memory corruption) the cells the diff reads are the cells the terminal is showing, and the emitted bytes are the byte-for-byte difference.

11.5.5 Layer 4 — `FrameBytes` (atomic state advance)

Header: *maya/include/maya/render/frame_bytes.hpp*.

`compose_inline_frame_v2` returns a `FrameBytes` — the bytes the frame would emit, plus the `InlineFrameState` the caller should hold *if the bytes ship*. It is move-only. The only consumers:

- `commit_to(writer) && -> CommitOutcome` — a `std::variant` of `commit::Synced{state}` (full delivery) or `commit::Stale{state, reason}` (hard I/O error).
- `abandon() && -> commit::Stale{state, ok()}` — explicit drop.

Because the return type is a `std::variant` and structured bindings on `variant` are ill-formed, every call site must `std::visit` both arms. **There is no success-only code path.**

```
auto bytes = std::move(synced).render(canvas, content_rows(canvas),
                                      term_h, pool, *std::move(wit));
auto outcome = std::move(bytes).commit_to(*writer);
return std::visit(overload{
    [](commit::Synced&& s) { return RenderResult{std::move(s.state)}; },
    [](commit::Stale&&  s) { return RenderResult{std::move(s.state),
                                                 s.reason}; }
}, std::move(outcome));
```

The historical bug class — "state advanced; `write` returned partial; next compose diffed against a state the wire doesn't show" — becomes a compile error, because the only way to obtain a `Synced` state is the return value of a successful `commit_to`, and the only way to obtain a `Stale` state is from either `commit_to`'s I/O-error arm or `abandon()`. State advance and byte delivery are fused into one operation whose outcome is a value the caller must destructure.

11.5.6 Layer 5 — `InlineFrame<Tag>` (linear chain)

Header: *maya/include/maya/render/inline_frame.hpp*.

The apex. `InlineFrame<Tag>` is a phantom-tagged type-state. Five tags:

Tag	Meaning
`Empty`	no frame yet
`Fresh`	first frame just rendered
`Synced`	wire and shadow agree (this is the steady state)
`Stale`	wire and shadow may disagree (after an I/O error)
`Sealed`	program is exiting; no further render

Each tag is a distinct class (template specialisation); each is move-only; each has a private constructor friended only to its predecessors in the transition graph:

From	Op	To
Empty	seed()	Fresh
Fresh	render(...)	Synced or Stale
Synced	verify()	optional<ShadowWitness>
Synced	render(..., wit&&)	Synced or Stale
Synced	demote_to_stale()	Stale
Synced	commit(marker)	Synced (scrollback advance)
Stale	render(...)	Synced or Stale
any	finalize(out)	Sealed

Compile-time consequences:

- `InlineFrame<Synced>::render` requires `ShadowWitness&&`. You cannot render a Synced frame without first calling `verify()`.
- `InlineFrame<Sealed>` has no `render` method — rendering after `finalize` is a compile error.
- `InlineFrame<Empty>::render` does not exist — rendering before `seed` is a compile error.
- All five specialisations are non-copyable — holding two parallel views of the state is a compile error.

The non-copyability is enforced by `static_assert`:

```
static_assert(!std::is_copy_constructible_v<InlineFrame<Empty>>);
static_assert(!std::is_copy_constructible_v<InlineFrame<Fresh>>);
static_assert(!std::is_copy_constructible_v<InlineFrame<Synced>>);
static_assert(!std::is_copy_constructible_v<InlineFrame<Stale>>);
static_assert(!std::is_copy_constructible_v<InlineFrame<Sealed>>);
static_assert(!std::is_copy_constructible_v<ShadowWitness>);
static_assert(!std::is_copy_constructible_v<FrameBytes>);
```

If a future refactor accidentally adds `= default` after removing the move-only `deletes`, these `static_asserts` fire and the build breaks. The discipline is checked by the compiler on every commit.

11.5.7 Scrollback commit, formalised (T3)

The Cmd that committed the legacy bug — "commit N rows to scrollback" — has been completely re-typed. The new flow:

```
auto marker = synced.scrollback_marker(rows);   // rows clamped to [0, prev_rows]
auto next   = std::move(synced).commit(std::move(marker));
```

`ScrollbackMarker` is move-only and consumed exactly once. The returned `InlineFrame<Synced>` has `prev_rows` strictly less than or equal to the prior. The set of row coordinates any future cursor move can target is bounded above by `prev_rows_at_emit + W` where `W` is the wire-cursor row count from the most recent successful emit.

T3 follows: once a row leaves the viewport via overflow, no future emit can target it, because no `Viewport` produced from any later state contains it, and `WireRow` closes the cursor inside the viewport.

11.5.8 What the chain does *not* claim

Honest scope:

- It does not prevent bugs *inside* the diff algorithm. If a per-row emit miscalculates a span or wide-char snap fails on a new emoji range, the wrong bytes go out for the right state. Type-state cannot witness algorithmic correctness; property-based fuzzing of the diff is the right complement.
- It does not prevent the terminal emulator from dropping bytes (A1).
- It does not prevent host code from writing to fd 1 directly (A3).
- It does not prevent silent memory corruption (A4); the FNV-1a re-checks are the probabilistic defense.

These are not handwaves. Each is a hypothesis named in the proof.

11.6 Where the code lives

A quick map for when you want to read the source:

File	Role
maya/include/maya/render/canvas_witness.hpp	Layer 0 — T0
maya/include/maya/render/wire_row.hpp	Layer 1 — T1
maya/include/maya/render/serialize.hpp	Layers 2–3 — ContentRows, ShadowWitness
maya/include/maya/render/frame_bytes.hpp	Layer 4 — atomic state advance
maya/include/maya/render/inline_frame.hpp	Layer 5 — linear chain
maya/src/render/serialize.cpp	`compose_inline_frame_v2`
maya/src/render/inline_frame.cpp	runtime transitions
maya/docs/internals/witness-chain.md	this chapter's source material
maya/docs/internals/scrollback-corruption-audit.md	every prior bug, with citations
maya/docs/internals/inline-autogrow.md	the ghosting trap, in detail
maya/tests/test_wire_row.cpp	10 tests + saturation proofs
maya/tests/test_inline_frame.cpp	5 happy paths + `static_asserts`

11.7 Why all this matters

It is reasonable to ask: this is a lot of machinery for what is, in the end, "don't write the wrong escape sequence." Why?

The answer is the same one you find in every chapter of this book. maya is a library; libraries are used by people who did not write them; the bugs that matter most are the ones the user never triggers in dev because their terminal happens to be 80x24 and their chat fits in a viewport. They appear in production — on a 50-line terminal, after a long session, after a virtualising `Cmd::commit_scrollback`, in a way that destroys the user's permanent terminal history.

The fixes you'd write to those bugs as one-off patches eventually diverge from the discipline they enforce. A new contributor will read the patched code, not the patch, and re-introduce the bug. The witness chain makes the discipline a *property of the type system*. The compiler enforces it on every build, in every contributor's editor, without anyone having to remember.

If you take one thing from this chapter, take that: **discipline encoded in types survives refactoring; discipline written in prose does not**.

Recap

Inline mode is rendering into the user's normal scrollback without ever destroying what is above the UI. Two historical bug classes — *ghosting from over-tall paint* and *over-claimed scrollback commit* — showed how brittle the convention-only approach was. maya's answer is the **witness chain**: a four-part theorem (*Scrollback Integrity*) decomposed into T0 (canvas cache truth), T1 (no cursor escape above viewport), T2 (no cell paint disagreeing with the model), T3 (monotone scrollback), each closed by a move-only typed token (`CanvasWitness`, `WireRow`, `ShadowWitness`, `FrameBytes`) and ultimately by the `InlineFrame<Tag>` linear chain. Every prior bug class is now a compile error.

Workshop

1. Read *maya/include/maya/render/wire_row.hpp*. Find the private constructor, the `std::clamp` call, and the list of friends. Try to construct a `WireRow` outside `Viewport`; convince yourself the compiler refuses.

2. Read *maya/docs/internals/witness-chain.md*. Identify which paragraph of the theorem corresponds to T0, T1, T2, T3. Match each to its layer in the chain.

3. Read *maya/docs/internals/scrollback-corruption-audit.md*, Finding #1 (`commit_prefix` overclaim). Match the failure mode in the audit to the layer in the chain that now prevents it.

4. Look at *maya/tests/test_inline_frame.cpp*. Identify the `static_assert`s that pin non-copyability. Imagine deleting one of the `deleted` copy constructors; predict which `static_assert` fires.

Part IV

Reactivity, runtime, and widgets

Signals, computeds, effects

The Elm runtime from Chapter 4 is one way to organise state and updates. maya ships a second, complementary mechanism: **fine-grained reactivity** via `Signal<T>`, `Computed<T>`, and `Effect`. Inspired by SolidJS (and Leptos in Rust), it lets you write code that *looks* like direct mutation but automatically tracks dependencies and runs only the precise computations that depend on whatever changed.

This chapter explains the model, its data structures, and when to reach for it over a `Program`.

12.1 The problem reactivity solves

In an Elm-style program, after every `Msg`, you call `view` on the entire `Model` and re-render the whole UI. The diff machinery makes this affordable, but the *view function itself* still touches every field — even fields that did not change.

In a small app this is fine. In a 10,000-line dashboard with 200 panels, you would like a different property: when only one field changes, only the views that read that field should re-evaluate.

That is what fine-grained reactivity gives you. It is not better than Elm; it is a different point on the trade-off curve.

12.2 The three primitives

12.2.1 `Signal<T>`: a value with subscribers

A `Signal<T>` holds a value and remembers who is reading from it:

```
Signal<int> count{0};

int    n = count.get();        // read the current value
count.set(5);                  // write a new one
count.update([](int& x){ ++x; }); // mutate in place
```

Underneath, a `Signal` is a `ReactiveNode` (the base type in *maya/include/maya/core/signal.hpp*) with a value field, a list of `subscribers`, and a version counter.

12.2.2 `Computed<T>`: a value derived from signals

A `Computed<T>` is a function that uses other signals:

```
Signal<int>   count{0};
Computed<int> doubled = computed([&] { return count.get() * 2; });

count.set(5);
int x = doubled.get();   // 10
```

Two ideas at play:

- When the lambda runs, every `get()` it calls is tracked as a *dependency*. maya's reactive runtime keeps a thread-local "current scope" that signals push themselves onto.
- When any dependency changes, the computed is marked *dirty*. The next `get()` on the computed re-runs the lambda; the get is cheap and idempotent.

12.2.3 `Effect`: run a side effect when signals change

An `Effect` is a lambda that runs once on construction and re-runs whenever its dependencies change:

```
Signal<int> count{0};
Effect e = effect([&] {
    std::println("count is now {}", count.get());
});

count.set(1);   // prints "count is now 1"
count.set(2);   // prints "count is now 2"
```

Effects are how reactive state reaches the outside world. The render function in maya's `run(event_fn, render_fn)` quick-tool API is effectively wrapped in an effect: the render runs once at startup; subsequent renders happen automatically when any signal read inside the render changes.

12.3 How dependency tracking works

Reactivity, when you understand it, is almost embarrassingly simple.

```
namespace detail {
struct ReactiveNode {
    std::vector<ReactiveNode*> subscribers;
    std::vector<ReactiveNode*> dependencies;
    bool pending = false;       // O(1) de-dup flag
    uint64_t version = 0;
    virtual void mark_dirty() = 0;
    virtual void evaluate() = 0;
};

// Thread-local: the node currently being evaluated.
inline thread_local ReactiveNode* current = nullptr;
}
```

The dance:

1. When a computed or effect runs its lambda, it sets `current = this` for the duration.
2. Inside the lambda, every `Signal<T>::get()` checks `current`. If it is non-null, the signal adds `current` to its `subscribers`, and the current node adds the signal to its `dependencies`.
3. When `Signal<T>::set()` runs, it walks `subscribers` and calls `mark_dirty()` on each.
4. A dirty computed re-evaluates on next `get()`; a dirty effect schedules a re-run.

The graph is bipartite-ish: signals are sources, effects are sinks, computeds are both. Nothing is automatic but the bookkeeping; user code just reads and writes signals as if they were variables.

12.3.1 Batching

If you write `count.set(5)` then `count.set(6)` in rapid succession, you do not want effects to fire twice. `maya` provides `Batch`:

```
{
    Batch batch;
    count.set(5);
    count.set(6);
    name.set("new");
}   // batch destructor flushes all dirty effects, exactly once each
```

Inside a batch, signal writes accumulate dirty flags on subscribers but do not actually re-run them. When the batch is destroyed, every dirty subscriber runs once. The `pending` bool on `ReactiveNode` is the deduplication flag — it ensures an effect that depends on three batched signals fires once, not three times.

12.4 When to reach for reactivity

A rule of thumb: reactivity is most useful when the UI has many **independent** sub-views over a large state. A clock that ticks every second alongside a chat scroll that updates on every message alongside a status bar that watches a network connection — all three can be effects in their own right, each touching only the signals they care about.

A pure `Program` is more useful when the state is small enough to re-render in full every frame, when you want to record-and-replay, or when you want to test the update function in isolation.

You can mix the two. `agentty` runs Elm at the top level (`Model` + `Msg` + `update`) and uses signals internally for ephemeral UI state (cursor position, scroll offset, animation phase).

12.5 A worked example

A weather panel that re-renders only the temperature line when the temperature changes:

```
Signal<double>    temperature{20.5};
Signal<int>       humidity{60};
Signal<std::string> location{"Tokyo"};

auto panel = v(
    t<"Weather"> | Bold,
    dyn([&] { return text("Location: "    + location.get()); }),
    dyn([&] { return text("Temperature: " + std::to_string(temperature.get())); }),
    dyn([&] { return text("Humidity: "    + std::to_string(humidity.get()) + "%"); })
);
```

The `dyn(...)` lambdas each read exactly one signal. Under `maya`'s reactive runtime, only the changed line is recomputed when its signal changes. The diff machinery from Chapter 9 then notices that only a few cells differ and emits a few bytes of ANSI. The total cost of changing the temperature is on the order of *microseconds*.

12.6 Lifetime and ownership

A natural question: who owns the `ReactiveNode` graph? maya's nodes are heap-allocated and reference-counted, but the references are scoped:

- `Signal<T>` is a value handle. Copy it freely; the underlying node is shared.
- `Computed<T>` likewise.
- `Effect` is RAII. When the `Effect` object goes out of scope, the effect is unregistered from its dependencies and will not fire again.

The thread-local current scope means signals are *not* thread-safe across threads. The doc-level rule: one reactive graph per thread. Cross-thread updates go through `Cmd<Msg>::task` (a `Program` mechanism) or through a thread-safe queue you flush on the UI thread.

12.7 Comparison with React, Solid, Leptos

If you have used reactive UI libraries:

- maya's model is closest to **Solid**'s. Signals are granular; tracking is automatic; there is no virtual DOM.
- Unlike **React**, there is no component re-render cycle. A signal change wakes only the effects that read it.
- Unlike **Leptos**, the host language is C++ so there is no macro DSL — signals are plain objects, the `|` pipes are the visual difference.

Recap

maya ships a signal/computed/effect reactive runtime in *core/signal.hpp*. Signals hold values; computeds derive; effects run side effects. Dependency tracking is automatic via a thread-local current-scope pointer. Batching deduplicates notifications. Use reactivity for fine-grained sub-view updates; use `Program` for testable top-level logic. The two compose.

Workshop

1. Write a clock effect that prints the current time every second. Drive it with a `Signal<std::string>` you update from a background thread (use `Cmd::Task` for the safe way).
2. Build a `Computed<int>` that depends on two signals. Mutate them inside a `Batch` and verify the computed re-evaluates exactly once.
3. Read *maya/include/maya/core/signal.hpp*. Find the `thread_local` current-scope pointer. Trace how `Signal::get()` consults it.

Events, focus, input parsing

A TUI consumes events: keystrokes, mouse clicks, paste, resize, focus changes. This chapter dissects how maya parses these from the ragged byte stream the terminal delivers, and how it routes them to the right handler.

13.1 What the terminal sends

In raw mode (Chapter 2) every keystroke arrives as one or more bytes on `stdin`. The mapping is:

Key	Bytes
a	0x61
Ctrl-A	0x01
Esc	0x1B
Alt-a	0x1B 0x61
Up	0x1B [A
Down	0x1B [B
F1	0x1B O P
Shift-Tab	0x1B [Z
Mouse click (SGR)	0x1B [< button ; col ; row M
Paste	0x1B [200 ... 0x1B [201

The parser has to:

1. Recognise prefixes incrementally (a single `0x1B` might be the start of a longer sequence, or it might be a lone Escape keypress).
2. Time out: if no bytes follow `0x1B` within a few milliseconds, treat it as a lone Esc.
3. Disambiguate identical prefixes (some terminals send `ESC[OP` for F1, others `ESC[[A`).
4. Handle bracketed paste — treat the whole `200 ... 201` block as one event, not a stream of fake keystrokes.

The parser lives in *maya/src/terminal/input.cpp*. It is a state machine driven by incoming bytes; we will not reproduce it here, but the shape of what it produces is the interesting bit.

13.2 The `Event` variant

What the parser *outputs* is a variant:

```
struct Key {
    KeyCode  code;          // Char, Up, Down, F1, ...
    char32_t ch = 0;        // for KeyCode::Char
    Mods     mods{};        // bitset: Ctrl, Alt, Shift
```

```
};

struct Mouse {
    int  x = 0, y = 0;
    MouseButton button;     // Left, Middle, Right, ScrollUp, ScrollDown, None
    MouseAction action;     // Press, Release, Move, Drag
    Mods         mods{};
};

struct Resize { int width = 0, height = 0; };
struct Paste  { std::string text; };
struct Focus  { bool gained = true; };

using Event = std::variant<Key, Mouse, Resize, Paste, Focus>;
```

Every event maya delivers is one of these. The renderer never sees raw bytes; the user code never sees raw bytes. The parser is the choke point.

13.2.1 Why types, not strings?

Some libraries deliver *key chord strings* (`"Ctrl+Up"`). maya delivers structured values. Two reasons:

- Pattern matching is exhaustive. `std::visit` on `Event` forces you to handle every alternative.
- No string parsing in user code. To check "user pressed Ctrl-Up," you read the `Key` struct's fields, not a string.

13.3 Event helpers

For the common pattern of "did the user press this key," maya provides predicate helpers:

```
bool key(const Event& e, char32_t ch);                  // any key press of ch
bool ctrl(const Event& e, char32_t ch);                 // ctrl-ch
bool alt(const Event& e, char32_t ch);                  // alt-ch
bool key(const Event& e, KeyCode code);                 // arrows, F-keys, ...
bool mouse_clicked(const Event& e, MouseButton btn);    // any click
bool mouse_clicked_in(const Event& e, Rect r, MouseButton b);

template <typename F>
void on(const Event& e, char32_t ch, F&& fn);            // syntactic sugar
```

Usage:

```
if (key(ev, 'q'))                 return false;           // quit
if (key(ev, KeyCode::Up))         scroll_up();
if (ctrl(ev, 'c'))                cancel();
if (mouse_clicked(ev, MouseButton::Left)) select_under_cursor();
```

13.4 From event to Msg — `key_map`

A `Program`'s `subscribe` function returns a `Sub<Msg>`. The most common subscription is "map keys to messages," and maya ships a builder for it:

```
template <typename Msg>
Sub<Msg> key_map(std::initializer_list<std::pair<char32_t, Msg>> bindings);
```

In use:

```
static Sub<Msg> subscribe(const Model&) {
    return key_map<Msg>({
        {'+',  Inc{}},
        {'-',  Dec{}},
        {'q',  Quit{}},
    });
}
```

The runtime, given this `Sub`, knows that when the user presses `+` it should construct an `Inc{}` `Msg` and feed it through `update`. Compose more elaborate subscriptions by combining `key_map`, `mouse_map`, `tick` (timers), and `batch`.

13.5 Focus

When the user clicks into a different terminal window or tab, the emulator may send a focus event (`ESC [ I` for focus-gained, `ESC [ O` for focus-lost) if focus reporting is enabled. The parser emits a `Focus{gained: true|false}` event; `maya` programs typically use it to pause animations or dim the UI when unfocused.

Inside the app, `maya` has a focus manager (*maya/include/maya/core/focus.hpp*) that tracks which widget has logical focus, so Tab/Shift-Tab can move focus between input fields. The widget catalogue in Chapter 15 expects this manager to exist.

13.6 Resize

When `SIGWINCH` fires, the runtime resamples the terminal size with `ioctl(TIOCGWINSZ)` and emits a `Resize{w, h}` event. The renderer recomputes layout against the new size and repaints. Most user code does not need to handle resize explicitly; the layout engine reflows automatically.

If you do want to react (for example, to re-load a configuration based on the new aspect ratio), pattern-match on `Resize` in your event handler.

13.7 Paste

Modern terminals support **bracketed paste**: when the user pastes a block of text, the terminal wraps the pasted bytes in `ESC[200~` ... `ESC[201~`. `maya`'s parser collects all bytes between these markers into one `Paste{text}` event. This is essential because pasted text often contains literal newlines and tabs that would otherwise look like Enter and Tab keypresses — which would, for example, submit a chat message after every pasted newline.

To enable bracketed paste, write `ESC[?2004h` on startup (`maya` does this automatically). To disable, write `ESC[?2004l`.

13.8 The polling loop

How does maya actually wait for events? On POSIX, the runtime uses `poll(2)` over three file descriptors:

- `stdin` — for key/mouse/paste bytes.
- A `signalfd`-or-equivalent — for `SIGWINCH`.
- A `wake_fd` — a pipe used to interrupt the poll when a background task (a `Cmd::Task`) dispatches a message.

The wake_fd is a classic Unix trick: when a background thread wants to feed a `Msg` into the event loop, it pushes the message onto a thread-safe queue and writes one byte to the pipe. The poll wakes up, drains the pipe, drains the queue, and processes the message. This is how maya's pure-functional `update` cooperates with real concurrency.

13.9 Windows

Windows has no `poll(2)` for keyboard input. The Windows implementation uses `ReadConsoleInputW` on a dedicated reader thread, posting events onto a shared queue that the main thread polls. The interface — the `Event` variant — is the same; the plumbing differs.

Recap

The terminal sends raw bytes; maya's parser turns them into a typed `Event = variant<Key, Mouse, Resize, Paste, Focus>`. Helpers (`key`, `ctrl`, `on`) make pattern-matching ergonomic. The polling loop ties stdin, signals, and a wake-fd together so that background tasks can interrupt the main loop. Bracketed paste prevents pasted text from being misread as keystrokes.

Workshop

1. Write a tiny program that prints every `Event` it receives, then run it and try unusual keys (F-keys, function + modifier combos, mouse, paste).
2. Read the first 200 lines of *maya/src/terminal/input.cpp*. Identify the state machine's states (waiting for ESC, in CSI, in OSC, ...).
3. Build a small program that uses mouse events to drag a coloured square around the screen. The squares' position lives in a `Signal<Point>`.

Cmd, Sub, and the runtime

Chapter 4 introduced `Cmd<Msg>` as the type of "effects as data." This chapter goes deeper: every alternative of `Cmd`, every alternative of `Sub`, how the runtime interprets them, and how background work plays with the pure `update` function.

14.1 The full `Cmd<Msg>` variant

```
template <typename Msg>
class Cmd {
public:
    struct None        {};
    struct Quit        {};
    struct Batch       { std::vector<Cmd> cmds; };
    struct After       { std::chrono::milliseconds delay; Msg msg; };
    struct SetTitle    { std::string title; };
    struct WriteClipboard { std::string content; };
    struct Task        { std::function<void(std::function<void(Msg)>)> run; };
    struct IsolatedTask { std::function<void(std::function<void(Msg)>)> run; };
    struct CommitRows  { int rows; };
    // ... and a few more
};
```

Each alternative is a small struct. Smart constructors give the ergonomic API:

```
Cmd<Msg> cmd1 = Cmd<Msg>::quit();
Cmd<Msg> cmd2 = Cmd<Msg>::after(std::chrono::seconds(1), Tick{});
Cmd<Msg> cmd3 = Cmd<Msg>::write_clipboard("hello");
Cmd<Msg> cmd4 = Cmd<Msg>::batch({cmd1, cmd2, cmd3});
```

Let us tour each one.

14.1.1 `None` — the identity

The do-nothing effect. Returned by `update` when there is no side effect to perform. Equivalent to `Cmd<Msg>{}` (default construction).

14.1.2 `Quit` — exit the program

The runtime, on seeing `Quit`, breaks out of its main loop, restores cooked mode, exits alt-screen, and returns from `run<P>()`. `main` then returns.

14.1.3 `Batch` — multiple effects

```
Cmd<Msg> save_and_quit = Cmd<Msg>::batch({
    Cmd<Msg>::write_clipboard(model.text),
    Cmd<Msg>::quit(),
});
```

Batched commands are interpreted in order. The runtime walks the vector and dispatches each one. There is no concurrency guarantee *within* a batch — `Task` commands inside a batch may finish in any order.

14.1.4 `After` — one-shot timer

```
return {model, Cmd<Msg>::after(std::chrono::seconds(3), Hide{})};
```

The runtime schedules a one-shot timer; when it fires, it dispatches the `Hide{}` `Msg` into `update` as if it had come from the user. Useful for toasts (auto-hide after N seconds), debouncing, retry backoff.

14.1.5 `SetTitle` — set the terminal title

Writes `ESC]0;`*title*`ESC\` to set the terminal's window title. Most emulators honour it.

14.1.6 `WriteClipboard` — OSC-52 clipboard

Writes `ESC]52;c;`*base64*`ESC\`, the OSC-52 "set system clipboard" sequence. Modern emulators (kitty, iTerm2, WezTerm) honour it, even over SSH. maya provides a portable path that falls back to xclip / pbcopy / clip.exe on terminals that do not.

14.1.7 `Task` — async work on a shared pool

The escape hatch for *arbitrary* side effects:

```
return {model, Cmd<Msg>::task([](auto dispatch) {
    auto result = http_get("https://example.com/data");
    dispatch(Loaded{std::move(result)});
})};
```

The lambda receives a `dispatch` callback. It runs on maya's shared, elastic-but-bounded background thread pool. When it calls `dispatch(msg)`, the runtime feeds `msg` into the next `update` call as if it had come from any other source.

Use `Task` for short tasks: HTTP calls, file reads, tiny shell-outs. The pool is bounded; if you saturate it, subsequent tasks queue.

14.1.8 `IsolatedTask` — async work on a dedicated thread

Same shape as `Task`, but each `IsolatedTask` gets its own detached `std::thread`. Use this for work that can wedge indefinitely on a blocking syscall: slow filesystem mounts, network freezes, sandboxed subprocesses that may hang. A wedged isolated task leaks one thread; a wedged `Task` would consume a slot in the shared pool permanently. The trade is some thread-construction overhead per call (~100-300 microseconds).

14.1.9 `CommitRows` — inline-mode scrollback commit

As described in Chapter 11: tells the inline renderer to mark the top `n` rows of the current frame as committed to scrollback. The next frame starts below them.

14.2 The Cmd functor

If you build composite UIs, each sub-component has its own `Msg` type. To embed a child component's `Cmd<ChildMsg>` into a parent's `Cmd<ParentMsg>`, `Cmd` provides a `map` (the categorical-functor `fmap`):

```
Cmd<ChildMsg> child_cmd = child_update(...);
Cmd<ParentMsg> parent_cmd = child_cmd.map([](ChildMsg cm) → ParentMsg {
    return ParentMsg::FromChild{std::move(cm)};
});
```

`map` walks every alternative; for those that carry a `Msg` (`After`, `Task`, `IsolatedTask`), it wraps the inner `Msg` with the provided function. The result is a new `Cmd` the parent runtime can interpret.

This is exactly Elm's `Cmd.map`; it is what makes the architecture compose to many nested components without leaking their message types upward.

14.3 The full Sub<Msg> variant

Subscriptions describe event sources you want active right now:

```
template <typename Msg>
class Sub {
public:
    struct Keys     { std::function<std::optional<Msg>(Key)>   f; };
    struct Mouse    { std::function<std::optional<Msg>(Mouse)> f; };
    struct Resize   { std::function<std::optional<Msg>(Size)>  f; };
    struct Paste    { std::function<std::optional<Msg>(std::string)> f; };
    struct Focus    { std::function<std::optional<Msg>(bool)>  f; };
    struct Tick     { std::chrono::milliseconds period; std::function<Msg()> f; };
    struct Batch    { std::vector<Sub> subs; };
    // ...
};
```

Each subscription is a function that takes a typed event and returns a `Msg` (or `nullopt` to ignore). The runtime samples them on every iteration.

Builders:

```
auto subs = Sub<Msg>::batch({
    key_map<Msg>({{'q', Quit{}}, {'r', Refresh{}}}),
    Sub<Msg>::tick(std::chrono::seconds(1), []{ return Tick{}; }),
    Sub<Msg>::mouse([](Mouse m) → std::optional<Msg> {
        if (m.action == MouseAction::Press)
            return Click{m.x, m.y};
        return std::nullopt;
    }),
});
```

`subscribe` returns this whole tree on every frame, so it can *depend on the model*: a help-modal-open model returns different keybindings than a help-modal-closed one. The runtime re-samples each frame, so changing the subscription set is just returning a different `Sub` from `subscribe`.

14.4 The runtime loop, expanded

Now we can sketch the runtime in full:

```cpp
template <Program P>
void run(RunConfig cfg) {
    auto [model, cmd0] = P::init();
    Terminal term{cfg.mode};                  // RAII raw mode + alt-screen
    Writer   writer{term.fd()};
    Renderer renderer{cfg};
    EventSource events{term.fd()};
    BgPool      bg;

    auto interpret = [&](Cmd<typename P::Msg> cmd) {
        std::visit(overload{
            [&](Cmd<Msg>::None) {},
            [&](Cmd<Msg>::Quit) { quit_requested = true; },
            [&](Cmd<Msg>::Batch& b) {
                for (auto& c : b.cmds) interpret(std::move(c));
            },
            [&](Cmd<Msg>::After& a) {
                bg.timer(a.delay, [a]{ events.dispatch(a.msg); });
            },
            [&](Cmd<Msg>::SetTitle& s) {
                writer.write(set_title(s.title));
            },
            [&](Cmd<Msg>::Task& t) {
                bg.run([t](auto dispatch) { t.run(dispatch); });
            },
            // ... more arms ...
        }, cmd);
    };

    interpret(std::move(cmd0));

    while (!quit_requested) {
        renderer.paint(P::view(model), writer);

        auto subs = P::subscribe(model);
        Event ev = events.wait();             // poll(2) + wake_fd

        auto msg = match_event(ev, subs);     // produce a Msg via subs
        if (!msg) continue;                    // event ignored

        auto [next_model, next_cmd] = P::update(std::move(model), *msg);
        model = std::move(next_model);
        interpret(std::move(next_cmd));
    }
}
```

That is the entire shape. Roughly 70 lines of real code; the maya version is longer because it handles error recovery, inline-mode redraw, signal-driven dirty checking, and Windows.

14.5 Other entry points

`run<P>()` is the canonical entry point. maya provides three others for common short-form cases:

- `run(event_fn, render_fn)` — the quick-tool form shown in Chapter 1. Internally constructs a trivial `Program` that closes over signals.
- `live(render_fn)` — animated output, no event loop. Useful for progress bars and live monitors. Calls `render_fn(dt)` every frame.

- `canvas_run(setup, event_fn, paint_fn)` — imperative Canvas access. Useful for games, demos, and visualisations where the abstraction of layout is in the way.
- `print(element)` — one-shot, no event loop. Useful for command-line tools that just want a styled banner.

All four lead to the same underlying terminal driver and rendering pipeline. The difference is in how state is managed.

14.6 Testing a Program without a terminal

Because `Program` is just four pure functions, you can drive it without ever opening a terminal:

```
TEST_CASE("counter goes up on Inc and stays put on Quit") {
    auto [m0, _0] = Counter::init();
    REQUIRE(m0.count == 0);

    auto [m1, _1] = Counter::update(m0, Counter::Inc{});
    REQUIRE(m1.count == 1);

    auto [m2, _2] = Counter::update(m1, Counter::Quit{});
    REQUIRE(m2.count == 1);   // Quit shouldn't increment
}
```

You can also assert on the `Cmd` that came back:

```
auto [_, cmd] = Counter::update(m, Counter::Quit{});
REQUIRE(std::holds_alternative<Cmd<Msg>::Quit>(cmd.inner));
```

This is the testing superpower of effects-as-data: you don't need a terminal, you don't need a clipboard, you don't need a network — you just match on the value the function returned.

Recap

`Cmd<Msg>` is the algebra of effects the runtime knows how to perform. `Sub<Msg>` is the algebra of event sources the runtime samples. Both are sum types with smart constructors and a functor-style `map`. The runtime is a tiny loop: paint, wait for an event, dispatch through the subs to get a `Msg`, call `update`, interpret the returned `Cmd`, repeat.

Workshop

1. Write a toast: a `Msg` that shows a message, scheduled by `Cmd::after` to dispatch a `HideToast` after 3 seconds.
2. Add a `Cmd::task` that performs a slow operation (`std::this_thread::sleep_for(2s)`) and dispatches a `Loaded Msg`. Notice the UI stays responsive.
3. Read *maya/include/maya/core/cmd.hpp*. Find the `map` method on `Cmd`. Trace how it walks the variant and rewraps the inner `Msg`.

The widget catalogue

The chapters so far described the engine. This chapter is a tour of the cabin: the 69 widgets maya ships under *maya/include/maya/widget/*. We will not document every one — that is what *docs/13-widgets.md* and *docs/11-api-reference.md* are for. Instead, we group them, explain the common interface, and build a real composite UI to show how they cooperate.

15.1 The widget contract

A maya widget is, by convention, a struct with:

- A `Config` struct of inputs (state to render).
- A free function `build(Config) -> Element` or a method `operator Element() const`.
- Optionally, a free `handle(Event&, Config&) -> std::optional<Msg>` for interactive widgets.
- Optionally, sub-widget-specific helper functions.

The widget never owns model state. It transforms a `Config` into an `Element`. The host owns the state, threads it through the widget on each render, and feeds events back. This is the same discipline as `Program::view` but applied per-widget.

15.2 Categories

The widgets fall into five rough groups.

15.2.1 Data display

- `LineChart` — a sparkline or full chart over time.
- `BarChart` — horizontal or vertical bars.
- `Gauge` — a circular percentage indicator.
- `Sparkline` — a one-line trend indicator.
- `Heatmap` — 2-D coloured grid for matrix data.
- `FlameChart` — profiling-style stacked bars.
- `Waterfall` — streaming spectrum or timing view.
- `TokenStream` — the LLM “tokens arriving live” view.

- `ContextWindow` — token budget bar.
- `GitGraph` — commit tree.

These take ranges (`std::span<double>`, `std::vector<Sample>`) and produce a rectangular widget. Many use the canvas API (Chapter 8) directly for sub-cell graphics (braille characters, half-blocks).

15.2.2 Input

- `Input` — single-line text field.
- `Textarea` — multi-line editor with word wrap.
- `Select` — single-choice dropdown.
- `Slider` — numeric range picker.
- `Checkbox` — boolean toggle.
- `Radio` — mutually-exclusive choices.
- `Button` — clickable rectangle.
- `Menu` — vertical list of choices.
- `CommandPalette` — a Ctrl-K style fuzzy launcher.

These widgets follow the *controlled* pattern: their state lives outside, in your model, and they receive it via `Config`. On a relevant key, they return a `Msg` (or, in the quick-tool API, mutate a signal).

15.2.3 Layout helpers

- `Table` — rows and columns with sort indicators.
- `Tabs` — horizontal tab strip + content area.
- `Tree` — collapsible hierarchical view.
- `Scrollable` — viewport over a too-tall child.
- `Modal` — a floating dialog with backdrop.
- `Popup` — a small contextual overlay.
- `Toast` — a transient banner.
- `Disclosure` — expand/collapse panel.
- `Divider` — horizontal or vertical line.
- `Breadcrumb` — path navigation crumbs.

15.2.4 Display

- `Markdown` — a real (small) markdown renderer.
- `InlineDiff` — single-line +/- diff view.
- `DiffView` — multi-line diff with syntax highlight.
- `Badge` — coloured pill (status, tag).
- `Spinner` — animated activity indicator.
- `ProgressBar` — discrete or continuous.
- `Calendar` — month view.
- `Canvas` — imperative drawing surface.
- `Image` — iTerm2 / Kitty graphics protocol.
- `LogViewer` — scrolling log tail.

15.2.5 Agent UI (the agentty family)

Several widgets are designed for agentic chat UIs:

- `ToolCall`, `BashTool`, `ReadTool`, `EditTool`, `WriteTool`, `FetchTool` — a purpose-built card per tool kind.
- `Message`, `ThinkingBlock`, `StreamingCursor`.
- `ActivityBar`, `PermissionPrompt`, `ModelBadge`, `CostTracker`.
- `GitStatus`, `FileChanges`, `SystemBanner`, `ErrorBlock`, `TurnDivider`, `ConversationView`, `PlanView`, `ApiUsage`.

These are not part of any TUI orthodoxy — they were built because agentty needed them. Their existence shows what the engine can sustain. Their source is a worked tour of using the layout engine, signals, and canvas together.

15.3 Anatomy of a widget: `Input`

Let us look at one widget in enough depth that you can read the others. `Input` (file *widget/input.hpp*) is a single-line text field.

15.3.1 Configuration

```
struct InputConfig {
    std::string value;                  // current text
    int         cursor_pos = 0;          // character index
    std::string placeholder;             // dimmed text when empty
    int         max_length = 0;          // 0 = no limit
    bool        password = false;        // show as bullets
    Style       text_style{};            // optional override
    int         width = -1;              // -1 = grow
};
```

State out, view in.

15.3.2 Building the element

```
Element input(const InputConfig& cfg);

inline Element input(const InputConfig& cfg) {
    auto display = cfg.value.empty()
        ? text(cfg.placeholder) | Dim
        : text(cfg.password ? std::string(cfg.value.size(), '*') : cfg.value);

    // The cursor block: a highlighted cell at cursor_pos.
    // (Real implementation handles word boundaries, scroll, etc.)
    return h(display);
}
```

The real implementation is fancier — it scrolls horizontally when the value exceeds the box width, it tracks the cursor as a separate overlay, it respects bracketed paste. But the shape is: take `Config`, return `Element`.

15.3.3 Handling input

```
struct InputEvent {
    enum class Kind { Edited, Submitted, Cancelled };
    Kind        kind;
    std::string new_value;
    int         new_cursor;
};

std::optional<InputEvent> handle_input(const Event& ev, InputConfig& cfg);
```

The host calls `handle_input(ev, my_cfg)`. If the event is relevant, the widget mutates `cfg` and returns an `InputEvent` the host can translate into its own `Msg`. If irrelevant, returns `nullopt`.

This pattern — `handle(event, config) -> optional<event_out>` — repeats across every interactive widget.

15.4 Composing widgets

A real screen is many widgets:

```
auto chat_screen(const Model& m) → Element {
    return v(
        h(
            // sidebar
            v(
                t<"Threads"> | Bold,
                map(m.threads, [](const auto& t) {
                    return text(t.title);
                })
            ) | width(20) | border_<Single>,

            // main conversation
            v(
                map(m.messages, [](const auto& msg) {
                    return message(MessageConfig{
```

```
                        .role  = msg.role,
                        .body  = msg.body,
                        .time  = msg.time
                    });
                })
            ) | grow_<1> | scrollable
        ) | grow_<1>,

        // composer at the bottom
        input(InputConfig{
            .value       = m.draft,
            .cursor_pos  = m.draft_cursor,
            .placeholder = "Message...",
        }) | border_<Round> | pad<0, 1>
    );
}
```

Each widget is a small function; the screen is composition. The host's `update` routes events to whichever widget should receive them, mutates the relevant config, returns the new model.

15.5 Where the polish lives

If you skim widget source files, you will notice they are mostly *boring*. Most of the implementation is layout, theming, and small DSL chains. The interesting bits are concentrated in a few widgets that exercise the canvas API (charts, sparklines, heatmaps), the markdown renderer (a full incremental parser), and the `Tool*` family (which model rich agent UIs).

The lesson: maya is engineered so that adding a widget is a short job. Picking a widget's name, defining a `Config`, writing a `build` function. The library you call once you have read this book.

Recap

maya's widgets are small structs: a `Config` of inputs, a `build` function returning `Element`, an optional `handle` for events. They never own model state. The catalogue spans data display, input, layout, display, and an agent-UI family inherited from `agentty`. The same pattern — a function from value to element — scales from `Badge` to `ConversationView`.

Workshop

1. Pick a widget you've never used. Read its header. Build a standalone example that drives it.
2. Implement your own widget: a `Stopwatch` that takes a duration and renders `MM:SS.mmm`. Use the canvas API for a thick-stroked clock face.
3. Read *maya/include/maya/widget/markdown.hpp*. Understand how it streams markdown incrementally without re-parsing the whole document on every keystroke.

Building, packaging, shipping

This last chapter is the practical mile: how to compile maya, how to depend on it from your own project, how to ship a binary that runs anywhere, and what to do when something goes wrong. It assumes you have read the architecture chapters and want to put a real program on a real machine.

16.1 Compiler requirements

maya requires C++26. As of this writing the meaningful toolchains are:

Compiler	Minimum	Notes
GCC	15	recommended on Linux and macOS
Clang	18	needs libstdc++ 15 or libc++ trunk
MSVC	14.40	C++23 + `/std:c++latest`
AppleClang	—	not yet C++26-capable; use Homebrew GCC

On Apple Silicon this means `brew install gcc` then explicitly configure with `cmake -DCMAKE_CXX_COMPILER=g`
The AppleClang shipped with Xcode is not ready for C++26 yet.

16.2 Building maya

The standard incantation:

```
git clone --recursive https://github.com/1ay1/agentty
cd agentty/maya
cmake -B build
cmake --build build -j$(nproc)
```

A few useful CMake options:

- `-DMAYA_BUILD_EXAMPLES=ON` — build the 26 example programs. On by default if maya is the top-level project; off when it is a sub-build.
- `-DMAYA_BUILD_TESTS=ON` — build the test suite. Run with `ctest -test-dir build`.
- `-DCMAKE_BUILD_TYPE=Release` — enable `-O3 -DNDEBUG`. maya also turns on link-time optimisation (LTO) automatically in Release.
- `-DCMAKE_INTERPROCEDURAL_OPTIMIZATION=OFF` — disable LTO if your linker is short on memory.

A clean Release build of maya plus all examples takes about a minute on a modern laptop.

16.3 Linking maya into your own program

The supported path is to install maya and `find_package` it from your project's CMake.

16.3.1 Installing

```
cmake -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j
cmake --install build --prefix /usr/local
```

This installs headers under `$prefix/include/maya/` and the static library under `$prefix/lib/`, plus a CMake config package under `$prefix/lib/cmake/maya/`.

16.3.2 Depending on it

In your project's `CMakeLists.txt`:

```
cmake_minimum_required(VERSION 3.28)
project(my_app LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 26)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

find_package(maya 0.1 REQUIRED)

add_executable(my_app main.cpp)
target_link_libraries(my_app PRIVATE maya::maya)
```

Then in `main.cpp`:

```
#include <maya/maya.hpp>
#include <maya/widget/markdown.hpp>     // each widget header opt-in

int main() {
    maya::print(maya::dsl::t<"Hello from your app">.build());
}
```

16.3.3 Vendoring via `FetchContent`

If you prefer to vendor maya alongside your sources:

```
include(FetchContent)
FetchContent_Declare(maya
    GIT_REPOSITORY https://github.com/1ay1/agentty.git
    GIT_TAG        master
    SOURCE_SUBDIR  maya
)
set(MAYA_BUILD_EXAMPLES OFF CACHE BOOL "" FORCE)
set(MAYA_BUILD_TESTS    OFF CACHE BOOL "" FORCE)
FetchContent_MakeAvailable(maya)

target_link_libraries(my_app PRIVATE maya::maya)
```

This downloads, builds, and links maya as part of your configure step. No installation required. Slower first build, faster CI.

16.4 Sandboxing your program (Linux / macOS)

maya itself does not sandbox; that is a feature of programs like agentty that wrap shell calls. If your TUI shells out, look at agentty's *src/tool/util/sandbox.cpp* as a reference. The relevant building blocks:

- **Linux:** `bwrap` (bubblewrap) with read-only binds for `/usr`, `/etc`, etc., a fresh `tmpfs` for `/tmp`, a shared network namespace.
- **macOS:** `sandbox-exec` with a sandbox profile file written to a temp path and removed on exit.

The pattern is to detect the backend by attempting a `<exe> -version` probe, fall back to no-sandbox if absent, and surface the decision to the user.

16.5 Producing a static binary

If you want one binary you can scp to any compatible machine, build with static linking. The shape (Linux) is:

```
cmake -B build-static \
    -DCMAKE_BUILD_TYPE=Release \
    -DCMAKE_EXE_LINKER_FLAGS="-static-libstdc++ -static-libgcc"
cmake --build build-static -j
```

This statically links libstdc++ and libgcc, but leaves libc dynamic (fully-static glibc breaks DNS resolution via NSS). For a 100%-static binary on Linux you need a musl toolchain (`x86_64-linux-musl-g++` from Alpine or `musl-cross-make`). On macOS, only `libSystem` is dynamic — that is Apple's only supported ABI.

On Windows, link the static MSVC runtime (`/MT`) and use the `x64-windows-static` vcpkg triplet for any third-party dependencies. agentty's release pipeline does exactly this; you can crib from its `CMakeLists.txt`.

16.6 Packaging

For distribution, see agentty's *packaging/* directory and *scripts/release.sh*. The script builds:

- A `.deb` (Debian / Ubuntu).
- An `.rpm` (Fedora / RHEL).
- A `.pkg.tar.zst` (Arch).
- A Homebrew formula.
- A Scoop manifest (Windows).
- An AUR PKGBUILD.
- Raw static binaries.

The whole pipeline runs locally; no CI required. The script is a worked example for any project that wants a similar one-command release flow.

16.7 Debugging

16.7.1 Terminal stuck in raw mode

If your program crashed and the shell is acting weird, run `stty sane` or `reset`. Avoid this in the future by using RAII for the terminal state (which maya does internally).

16.7.2 Rendering looks corrupted

First, check that your terminal supports truecolour and the modes maya uses. `echo $TERM` should show a modern value (`xterm-256color`, `kitty`, `wezterm`). Some old terminals (Windows command prompt) need explicit enablement of ANSI processing.

Second, capture a frame to a file:

```
script -q -c './my_program' frames.txt
```

The `frames.txt` file contains every byte your program sent; inspect it with `cat -v frames.txt` to see the raw ANSI.

16.7.3 Slow rendering

If frames are dropping, the bottleneck is almost certainly your view function or your event handler, not maya. Profile with `perf record` (Linux), `Instruments` (macOS), or `Tracy` (cross-platform). maya's render path itself measures in the tens of microseconds.

16.8 Going further

This book has been a guided tour. The repository contains a great deal more:

- *maya/docs/*: 14 markdown chapters that overlap with this book but are more reference-style.
- *maya/examples/*: 26 fully-worked programs ranging from *counter.cpp* (60 lines) to *messenger.cpp* (3000+ lines, a real chat UI).
- *maya/docs/internals/*: deep engineering writeups on the corner cases of inline rendering, the witness chain, and scrollback integrity.
- agentty's repository: the largest production user of maya, with its own design discipline that complements the library's.

If you finished this book and built something with maya, you have joined a small club of people who write terminal UIs by composing typed values rather than by typing escape sequences. That is the right way to build them; the rest of the world will get there eventually.

Recap

Build maya with CMake against GCC 15+ or Clang 18+ (or MSVC 14.40+ on Windows). Use `find_package(maya)` or `FetchContent` to depend on it. Static binaries need `-static-libstdc++ -static-libgcc` on Linux (musl for fully static) or `/MT` on Windows.

Packaging is one script away in agentty's *scripts/release.sh*. When something looks wrong, `script(1)` the byte stream and read the ANSI.

Workshop

1. Set up a new project, vendor maya via `FetchContent`, and write a one-screen TUI for something you actually want — a directory size visualiser, a small log tail, an HTTP-endpoint poller.
2. Build it static. Scp it to a different machine. Run it. Note how the dependency story stays exactly: zero.
3. Contribute back. Pick a widget that does not yet exist (a kanban board, a piano-roll, a syntax-highlighted JSON viewer) and PR it. The maintainers are friendly; the architecture makes the work tractable.

Colophon

This book was produced from the agentty repository *(commit at time of writing)* using `pdflatex` on Arch Linux. The body is set in Latin Modern; code listings use Latin Modern Mono.

The four parts mirror four levels of zoom: foundations, architecture, rendering, and reactivity. Each chapter is its own `.tex` file under *chapters/*, included from *main.tex*. Re-ordering, extending, or shortening the book is a matter of editing `include` lines and re-running `make`.

The illustrative code snippets are simplified for exposition. Where the simplification could matter — the bit layout of a packed cell, the exact arms of the `Cmd<Msg>` variant, the loop body of the SIMD differ — the prose points at the file and section in maya where the real version lives. The real version always wins.

If a passage in this book disagrees with the source, the source is right. Please file an issue against the agentty repository so we can fix the book.

Source layout for the book:

```
book/
  main.tex           - root document, includes everything
  preamble.tex       - packages, colours, code styles, boxes
  Makefile           - ‘make‘ builds maya-book.pdf
  chapters/
    01-introduction.tex   - Hello, maya
    02-terminal.tex       - The terminal as a state machine
    03-cpp.tex            - Modern C++ you need
    04-elm.tex            - The Elm loop and algebraic effects
    05-dsl.tex            - The compile-time DSL
    06-elements.tex       - Elements, style, colour, border
    07-layout.tex         - Flexbox in integers
    08-canvas.tex         - The canvas and packed cells
    09-simd.tex           - SIMD frame diffing
    10-ansi.tex           - ANSI serialization and the writer
    11-inline.tex         - Inline mode and the witness chain
    12-signals.tex        - Signals, computeds, effects
    13-events.tex         - Events, focus, input parsing
    14-cmdsub.tex         - Cmd, Sub, and the runtime
    15-widgets.tex        - The widget catalogue
    16-build.tex          - Building, packaging, shipping
    99-colophon.tex       - this page
```